

UNITY STUDY



필기_1 : 20/06/23 ~ 20/08/27

필기_2 : 21/01/29 ~ 21/xx/xx

기존 필기 검토 및 수정 : 21/05/24 ~

박지원

<목차>

- Unity 기초 지식
- Unity 중급 기능
- Unity 고급 기능
- 최적화
- Debugging
- Unity Profiler
- ERROR
- Prefab
- 2D Sprite
- Tilemap
- Animation
- Layer & Tag & Name

- UI
- Sound
- Scene
- 슈팅 게임 만들기
- 그룹으로 묶인 게임 오브젝트
- Open Source Study

<Unity 기초 지식> 20/06/23

1. Unity Object의 생명주기

1) Awake()

- **GameObject가 최초로 생성 될 때 호출**되므로 프로젝트에서 단 한번만 호출됩니다.
- GameObject의 **활성화 여부와 상관없이** 호출됩니다.
- Start()보다 먼저 호출됩니다.
- 활성화 여부와 상관없이 호출되는 함수는 Awake() 말고 , 생성자가 존재합니다. 이 둘을 제외하면 아마 대부분 활성화 되어야 호출될 듯?

2) Start()

- **Update() 직전에 호출**됩니다.
- 프로젝트에서 단 한번만 호출됩니다.
- GameObject가 **활성화 되어야만** 호출됩니다.

[...] Awake is called after all objects are initialized so you can safely speak to other objects or query them using eg. `GameObject.FindWithTag`. Each GameObject's Awake is called in a random order between objects. Because of this, you should use Awake to set up references between scripts, and use Start() to pass any information back and forth. Awake is always called before any Start functions. This allows you to order initialization of scripts. Awake can not act as a coroutine.

and about `Start()` :

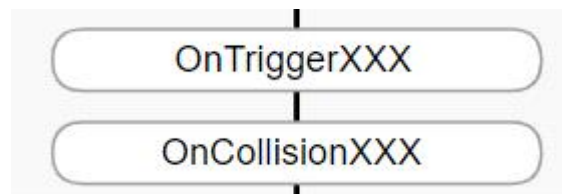
Start is called on the frame when a script is enabled just before any of the Update methods is called the first time.

Like the Awake function, Start is called exactly once in the lifetime of the script. However, Awake is called when the script object is initialised, regardless of whether or not the script is enabled. **Start may not be called on the same frame as Awake if the script is not enabled at initialisation time.**

3) FixedUpdate()

= Unity Engine이 물리 연산을 하기 전에 시행되는 함수

- 컴퓨터 사양(프레임)에 상관없이 언제나 1초당 50번 (=20ms) 호출되는 규칙적인 함수입니다.
- 정확한 계산이 필요한 **물리 기능에서 이용**합니다.
- FixedUpdate는 고정된 간격으로 동작하는 것이 유리한 행동들을 계산하는 데 적합합니다.
- '0.02초 -> 0.04초 -> 0.06초 -> ~ -> 1초'의 방식으로 감지하기 때문에 0.03초에 이벤트가 발생하면 감지하지 못합니다.
- FixedUpdate의 물리 연산을 하는데 0.02초를 초과한다면, 다음에도 연산 하는데 0.02초가 넘어갈 것이고 화면이 멈춥니다. 이를 방지하기 위해 Maximum Allowed Timestep을 설정하여, 이 시간을 넘기면 FixedUpdate를 중지하고 다른 연산을 처리합니다.
- FixedUpdate는 최적화가 중요합니다. 왜냐하면 FixedUpdate에 소모되는 시간이 많을수록 게임 입력, 로직, 렌더링에 사용되는 시간이 적어지며, 이 때 발생하는 병목현상은 게임의 프레임 드랍 현상을 발생시킵니다. (최적화 파트에서 다룹니다.)



- 생명주기에 따르면, FixedUpdate는 언제나 좌측의 두 함수보다 먼저 호출됩니다.

All of the OnCollision*() functions are called after the physics simulation is completed.

4) Update()

- Unity에서 가장 프레임에 대한 개념을 근접하게 구현한 방법이며, 프레임에 따라 1초당 N번 호출되기 때문에 사용자의 컴퓨터 사양(프레임)마다 호출 되는 횟수가 달라지는 불규칙적인 함수입니다.
- Update 관련 함수는 프레임 기반으로 호출되는 함수를 복잡도가 높게 작성한다면 method 내부에서 부하가 발생하므로 **게임이 매우 느려집니다**. 그러므로 반드시 필요한 기능만 넣고 매우 적은 이벤트를 넣어야 합니다. -> 이벤트 주도적 프로그래밍으로 해결합니다.
- Update문이 길면 가독성이 좋지 않으므로 Update에는 Method 호출 위주로 작성하고, 아래에 Method를 작성합니다.

5) LateUpdate()

- Update가 끝난 이후에 호출이 됩니다.
- Update와 동일한 횟수로 호출됩니다.

6) OnDestroy()

- GameObject가 파괴된 이후에 단 한 번 호출됩니다.
- GameObject는 보통 Destroy() OR 씬 종료에 대한 응답으로 파괴 될 수 있습니다.

7) 활성화 / 비활성화 함수

- GameObject는 활성화 , 비활성화가 될 수 있습니다.
- SetActive()로 활성화 여부를 관리합니다.
- 활성화 , 비활성화는 반복할 수 있으므로 , 1회성 호출이 아니라 활성화 , 비활성화 될 때 마다 알맞은 함수가 호출됩니다.

① OnEnable : GameObject가 활성화 될 때 마다 실행됩니다.

② OnDisable : GameObject가 비활성화 될 때 마다 실행됩니다.

<Reference>

- Unity Docs : <https://docs.unity3d.com/kr/2019.4/Manual/ExecutionOrder.html#FirstSceneLoad> >
- 골드메탈 : <https://www.youtube.com/watch?v=PyN3JkPTpAI>

2. 월드/시간 업데이트

1) Frame

= 연속된 움직임의 최소 단위

- 게임에서 일반적으로 1/60초 (60 fps)
- Unity에서는 1 frame(=1/60초) 마다 주어진 일을 수행합니다. 1 frame 보다 빠른 시간 내에 일 처리를 완료해도 다음 frame 까지 대기하지만 , 일 처리가 1 frame 보다 더 걸리면 다음 동기화 타이밍을 기다립니다. 다시 말해 주어진 일이 1.3 frame 정도 걸린다면 2 frame을 사용하게 됩니다. 이런 현상을 Frame Drop 이라고 합니다.
- Frame Drop을 유발하는 방식은 많은 ERROR 와 버그를 발생시키므로 반드시 해결해야 합니다.

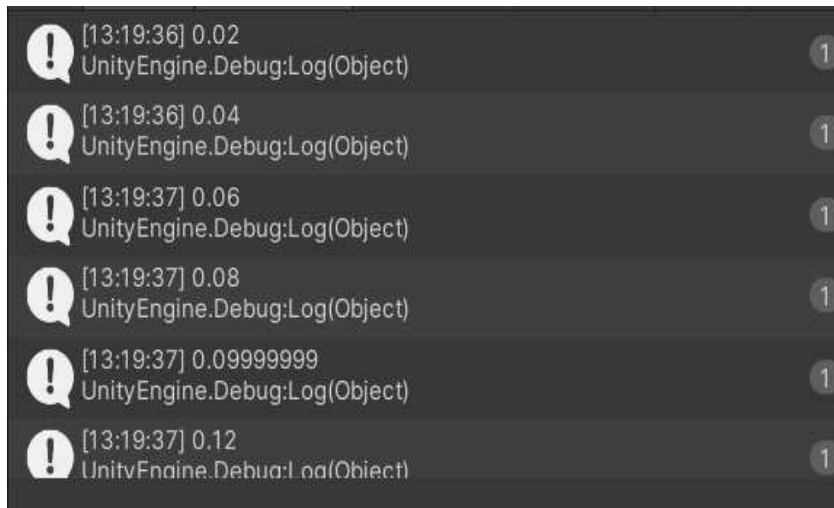
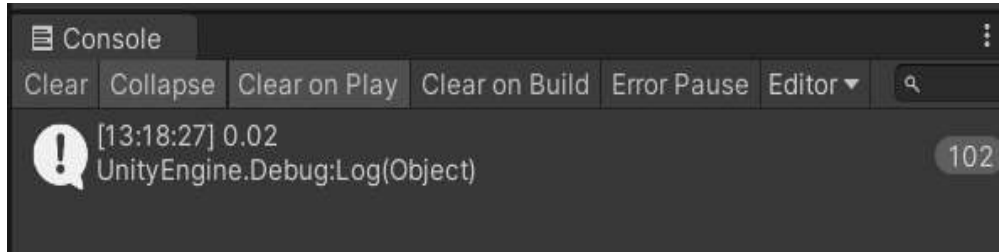
2) FPS(Frame Per Second)

= 1초 동안의 frame 개수

- = 1초 동안 Unity가 카메라에서 화면으로 새로 Rendering 하는 코드를 몇 번이나 반복할 수 있는지.
- 컴퓨터의 성능과 환경에 영향을 많이 받습니다.

3) Time.deltaTime : 바로 직전의 1 Frame이 수행된 시간

① FixedUpdate에서의 Time.deltaTime



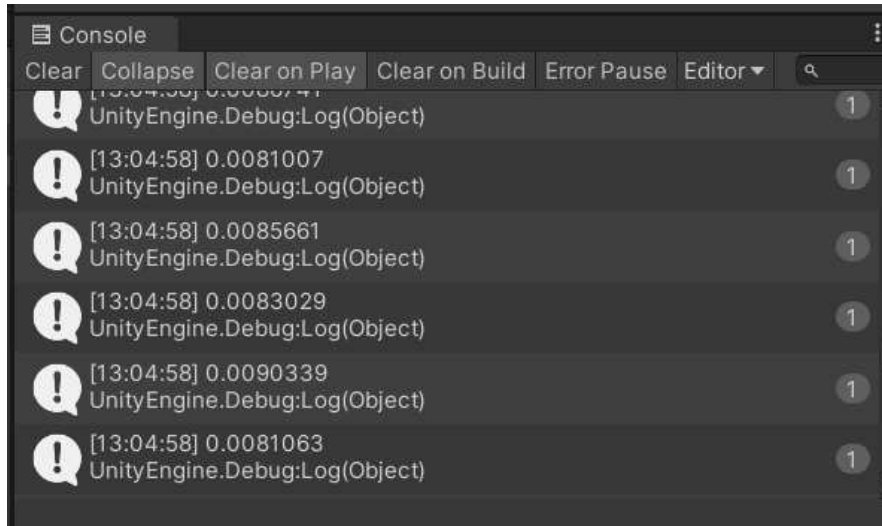
<그림 1>

- FixedUpdate는 통상 50fps이므로 Time.deltaTime이 언제나 0.02초가 됩니다.
- 현재 Frame 기준 바로 직전의 Frame은 항상 0.02초이기 때문입니다.

<그림 2>

- 하지만 애석하게도 , 시간 측정을 위해 time_span += Time.deltaTime으로 작성하고 time_span을 작성하면 FixedUpdate 조차 0.00001초 정도의 오차는 발생시킵니다.

② Update에서의 Time.deltaTime



- Update는 통상 60fps라고 배웠지만 모든 컴퓨터마다 Game창의 stats에서 확인한 fps의 값은 다르고, 그마저도 고정되어 있지 않고 동일한 컴퓨터에서조차 fps는 시시각각 변화합니다.

- 좌측의 그림처럼 Time.deltaTime을 update에서 출력해보면 0.01보다 작은 값이 나오며, 이들은 각각 고정되지 않은 1 Frame의 시간을 의미합니다.

- Time.deltaTime은 매우 작은 값이므로 상수를 곱해서 물체의 이동에 사용합니다.

ex) 제 컴퓨터는 690 ~ 1500 fps가 나오고, 강의에 나온 컴퓨터에서는 3000 fps 정도가 나옵니다.

3. GameObject

= 게임에서 존재하는 물체입니다. 객체라고 생각하여도 됩니다.

- 사운드, 매니저 등등 보이지 않는 객체까지 포함합니다.

- GameObject 자체는 빈 깡통입니다. 거기에 많은 Component를 추가시켜서 GameObject를 진화시킵니다. 그러므로 GameObject는 component의 집합이라고 할 수 있습니다.

- 모든 GameObject는 반드시 Transform Component는 갖고 있어야 합니다.

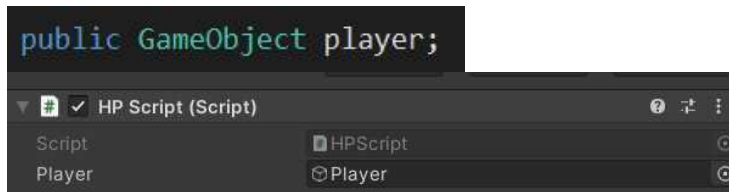
- GameObject는 모두 Unity의 Hierarchy에 저장 되어있습니다. 앞으로 제가 Unity라고 적는 용어는 원래는 World라고 불리는 용어이며, 이는 Unity(응용프로그램) 공간을 의미합니다.(Unity 실행하면 나오는 공간)

ex) Player, Monster

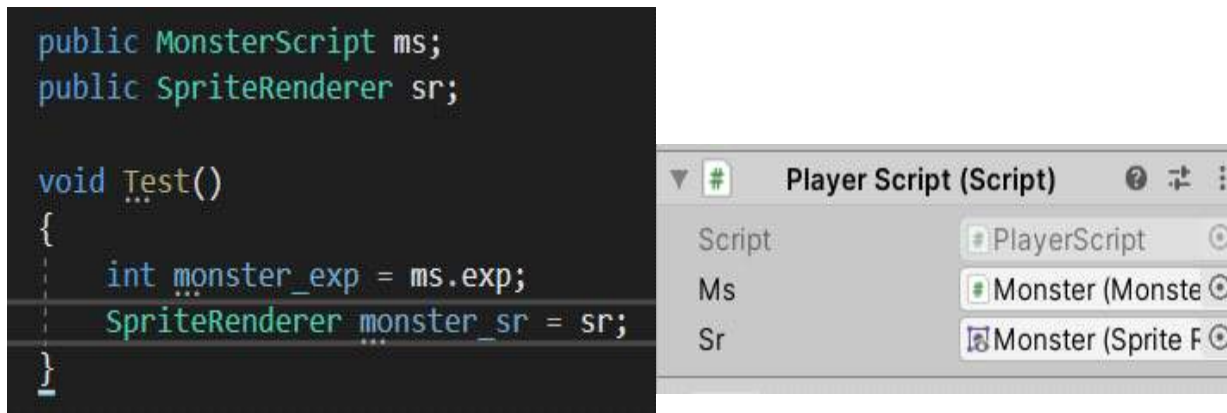
4. 스크립트에서 Unity Editor에 존재하는 GameObject 연결하는 방법

- Unity 공식 문서 : < <https://docs.unity3d.com/kr/2019.4/Manual/ControllingGameObjectsComponents.html> >
- Reference : 유니티 C# 스크립팅 마스터 P139

1) public으로 존재하는 GameObject 드래그로 연결하기 (prefab 불가능)



2) GameObject에 존재하는 특정한 Component만 연결하기



- ① Script에서 해당 컴포넌트에 맞는 클래스로 객체를 생성합니다.
- ② Unity Editor에서 **Script가 아닌 GameObject를 드래그** 하면 자동으로 Unity의 GameObject에서 해당 타입에 맞는 component를 연결해줍니다. (Script는 백날 드래그 해봤자 금지 표시가 발생하면서 연결이 되지 않습니다.)

3) Find("string") & FindGameObjectWithTag("string")

- 여기서 사용하는 함수들은 유용하지만 **호출 비용이 비싸기 때문에 Awake나 Start처럼 일회성 이벤트에서만 호출하도록** 합니다.
- private 멤버의 경우에 드래그가 불가능하므로 해당 방식을 사용합니다.
- Unity Editor에 GameObject가 무수히 많다면 드래그 하는 것이 불편하므로 해당 방식을 사용합니다.

① Find("Name")

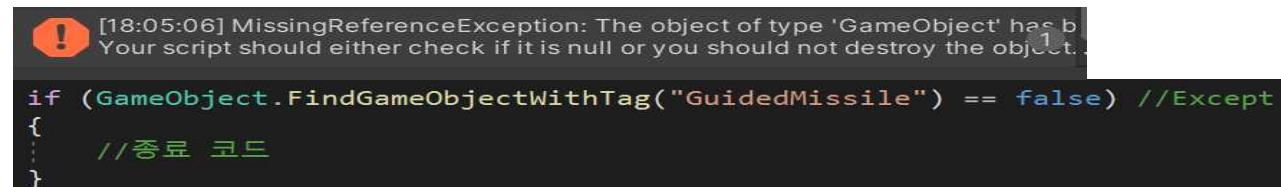
- 문자열 비교로 Unity에서 GameObject를 찾기 때문에 매우 느리게 동작하는 method 입니다.

② FindGameObjectWithTag("Tag") & FindWithTag("Tag")

- 태그를 반드시 할당해야 하며 , Script 내에서도 할당이 가능하나 보통은 Unity에서 할당할 것 입니다.
- Unity Editor의 Scene에서 일치하는 태그를 가진 GameObject를 검색해 **처음 발견되는 1개의 GameObject를 반환**합니다.
- **인자로 문자열을 넣긴 하지만 , Unity가 내부적으로 문자열을 숫자 형태로 변환해서 비교하므로 Find("Name")보다 속도가 빠릅니다.**
- **그러므로 FindGameObjectWithTag("Tag")를 사용하여 GameObject를 Unity에서 찾아 Script로 가져옵니다.**

<ERROR>

- 어떤 GameObject가 Unity Editor에 존재하지 않을 때 발생합니다.
- 아래의 코드와 같이 예외처리를 해야 합니다.



```
[18:05:06] MissingReferenceException: The object of type 'GameObject' has been destroyed. Your script should either check if it is null or you should not destroy the object.

if (GameObject.FindGameObjectWithTag("GuidedMissile") == false) //Except
{
    //종료 코드
}
```

③ FindGameObjectsWithTag("Tag")

- = Unitor Edtior에서 일치하는 태그를 가진 GameObject 들을 검색해 , 해당 태그를 가진 모든 GameObject 들을 반환합니다.
- 배열에 저장해서 사용합니다.

4) Parent GameObject를 통해 Child GameObject 연결하기

① 부모를 찾고 , Find로 자식 찾기

```
blind = GameObject.FindGameObjectWithTag("Canvas").transform.Find("Blind").gameObject;
```

- 부모를 먼저 Tag로 찾고 , 하위 자식을 이름으로 찾습니다.
- 성능은 좋지 않을 것이지만 , Index보다 안전하다고 생각합니다.

<Warning Code : FindChild>

```
blind = GameObject.FindGameObjectWithTag("Canvas").transform.FindChild("Blind").gameObject;
```

 class System.String

CS0618: 'Transform.FindChild(string)'은(는) 사용되지 않습니다. 'FindChild has been deprecated. Use Find instead (UnityUpgradable) -> Find([mscorlib] System.String)'

잠재적 수정 사항 표시 (Alt+Enter 또는 Ctrl+.)

② Index를 활용하는 방식

```
//게임 오브젝트 중에서 첫번째 자식 가져오기  
GameObject child = transform.GetChild(0).gameObject;  
//게임 오브젝트 중에서 첫번째 자식의 SpriteRenderer 가져오기  
SpriteRenderer sr = transform.GetChild(0).gameObject.GetComponent<SpriteRenderer>();
```



- Player가 부모면 Body가 0번 , WheelLeft가 1번 , Triangle이 4번 Index가 됩니다.
- Index를 이용해서 호출할 수 있으나 , 자식의 순서가 변경되는 순간 다른 자식 GameObject를 참조하므로 , 예기치 못한 ERROR가 발생합니다. 그러므로 사용을 지양합니다.

5) 비활성화(=SetActive(false))된 GameObject를 찾는 방법

① ERROR



[00:13:58] NullReferenceException: Object reference not set to an instance of an object
MenuManager.ActivatedMenuBar () (at Assets/Scripts/StartGameScripts/MenuManager.cs:17)

- 비활성화 된 GameObject는 Unity Editor에서 존재하지 않는 것으로 처리하기 때문에 찾지 못하여 , 찾으려 하면 ERROR가 발생합니다.

② 해결 방법 : 활성화 된 부모 이용

근데 이러면 문제가, 통장을 눌렀을 때 회색, 처리, 거절 오브젝트를 find하고 SetActive(true);를 해야 하는데 false인 오브젝트는 find가 안 된다

해결 방법은 **활성화된 부모를 찾아서 비활성화된 자식을 찾으면 된다**

gray오브젝트는 image, 처리와 거절은 button으로 셋 다 UI로써 부모를 Canvas로 두고 있다

Canvas는 활성화되어 있으니까 Canvas를 find하고 findchild로 비활성화된 자식들을 찾으면 된다

5. Component

1) 정의 및 예시

= Unity에서 미리 만들어 놓은 , GameObject에 필요한 모든 기능(Library)

- 상속은 부모의 모든 내용을 상속받아 사용하므로 필요 없는 기능까지 받아버리므로 오히려 해당 기능을 변경하거나 삭제해야 하는 추가 작업이 발생합니다. 이를 보완하기 위한 것이 component입니다.

- Component 간에 독립성으로 인해 기능의 추가와 삭제가 간편하고 , 불필요한 기능은 상속 받지 않습니다.

- GameObject는 Component 들로 이루어져 있기 때문에 GameObject가 객체라면 , Component는 Method로 생각할 수 있습니다.

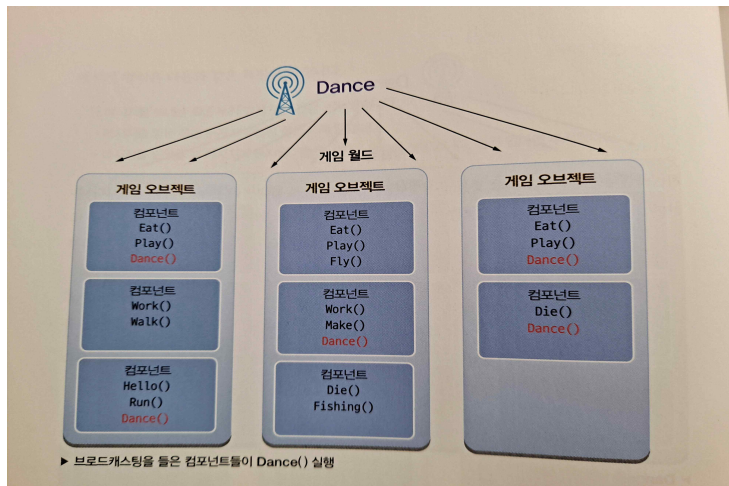
- Unity에 존재하는 모든 Script가 상속받는 MonoBehaviour에 우리가 사용하는 Component가 저장되어 있습니다.

- 개발자가 만든 Script도 하나의 Component 입니다.

- 모든 GameObject에서 transform component는 삭제할 수 없습니다.

ex) transform , rigidbody , box collision

<Broadcasting 방식으로 Component 호출>



- Unity에서 Scene에 존재하는 모든 GameObject가 춤을 추기를 바랍니다. 그러면 , Unity 는 모든 GameObject에게 Dance() 함수를 실행하도록 message를 broadcasting 합니다.
- 모든 GameObject들은 message가 어디서 왔는지는 따지지 않고 , 본인이 Dance() 함수를 가지고 있다면 실행하고 , 없다면 무시합니다.
- 이는 Start(), Update()같은 유니티 이벤트 method들의 호출 원리입니다. Start()를 예로 들면 , 모든 GameObject는 component(Script)에 Start()가 있으면 GameObject가 생성된 순간 모두 실행합니다.

2) GetComponent<> ()

= 원하는 타입의 component(스크립트 포함)를 Unity 내부에서 스크립트로 찾아오는 method입니다.

- Reference Type 이므로 메모리 낭비를 줄일 수 있습니다.

- 비용이 큰 작업인지, 작은 작업인지에 대해서는 Unity의 버전마다 다릅니다. 그러므로 안전하게 Awake나 Start에서 변수에 캐싱(찾아가는 경로)을 생략하기 위해 변수에 저장하는 기법을 하여 사용하고 Update에서는 사용하지 않습니다.

<사용법 및 특징>

① GameObject에서 자신이 원하는 타입의 component를 가져옵니다.

```
GameObject bullet = Instantiate(bullet_a, transform.position, transform.rotation);  
//bullet의 rigidbody를 가져옵니다.  
Rigidbody2D rigid = bullet.GetComponent<Rigidbody2D>();
```

② 단독으로 사용하면 default로 스크립트와 연결된 GameObject의 해당 타입의 component를 가져옵니다.

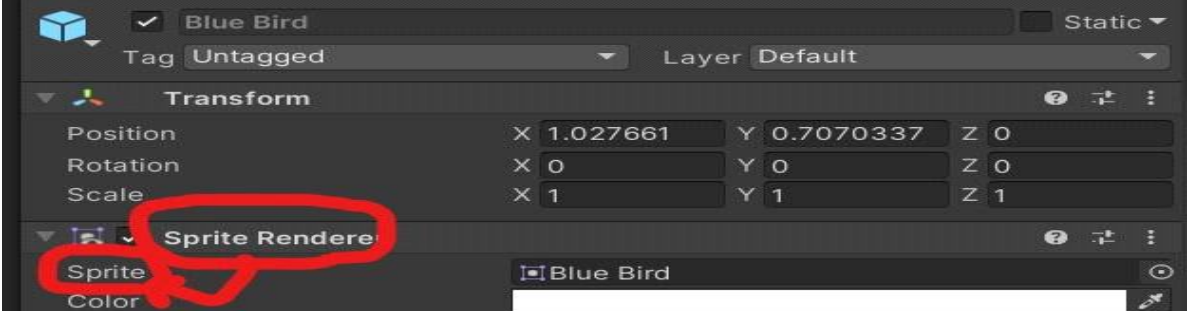
```
SpriteRenderer spriterenderer = GetComponent<SpriteRenderer>();
```

③ 복수의 컴포넌트 가져오기

= 배열과 GetComponents<Component>()로 가져옵니다. (위에서 부터 순서대로 읽어 옵니다.)

④ Component를 가져올 때 고려할 주의사항 (ERROR가 나지 않아 더욱 찾기 어려운 ERROR)

```
//profile_player1.sprite = InitPlayerCharacter.Instance.player1_character.GetComponent<Sprite>();  
profile_player1.sprite = InitPlayerCharacter.Instance.player1_character.GetComponent<SpriteRenderer>().sprite;
```



<Component와 속성을 잘 구분합니다.>

- Sprite Renderer Component의 하위 속성인 Sprite를 가져 와야 합니다.
- transform.position 대신에 그냥 position을 사용한 것과 동일한 실수입니다.

6. Instantiation (인스턴스화)

= Unity에서는 스크립트에 클래스를 정의합니다. 그리고 이 스크립트를 이용하기 위해 GameObject에 component로 추가합니다.

- 클래스는 틀이기 때문에 이것만으로는 아무런 일이 일어나지 않습니다. 아직도 스크립트는 그냥 코드를 적은 텍스트 일 뿐입니다. 이를 GameObject의 component로 연결 해주어야 합니다.

- 이렇게 GameObject에 작성한 스크립트(클래스)를 넣어서 GameObject 숨을 붙여넣는(=기능을 추가하는) 과정을 Script Instance 라고 합니다.

***** GameObject & Prefab = 객체 // Script = Class OR Interface*****

<주의사항 : 하나의 Script는 수많은 GameObject에 인스턴스화 시킬 수 있습니다.>

① 문제점

- 여러 개의 GameObject에 똑같은 Script를 넣는다면 Script에 존재하는 모든 멤버는 각각의 GameObject에서 1개씩 만들어져 게임 내부에서 독립적으로 여러 개가 존재하고 , Method는 여러 번 실행됩니다. 예를 들어 Script에 자료구조 List가 존재하고 , 해당 Script를 3개의 GameObject에 인스턴스화 시키면 3개의 게임 전체에 독립적인 List가 3개 존재하게 됩니다.

② 해결책

- 1개의 GameObject에만 존재해서 게임 내에서 멤버 변수 , Method가 독립적으로 한 번씩만 존재합니다.
- 우리는 이것을 **Singleton Design Pattern** 이라고 부릅니다.

7. 자료구조

C++	C#
Vector	List
Map	Dictionary
Stack	Stack
Queue	Queue
String	String

- C++와 C#의 자료구조의 이름과 Method는 다르지만 원리는 모두 동일합니다.

- Reference : <https://engineer-mole.tistory.com/174>

8. Character Controller

- = 캐릭터를 간단하게 움직이는데 사용하는 Component
- Rigidbody의 물리를 많이 사용하지 않는 캐릭터의 이동 처리에 사용합니다.
- Reference : <https://medium.com/ironequal/unity-character-controller-vs-rigidbody-a1e243591483>

캐릭터 컨트롤러(Character Controller)는 Rigidbody 물리를 활용하지 않는 3인 또는 1인 플레이어에 주로 사용됩니다.

하지만! 유니티에서는 플레이어 캐릭터가 물리에 의해 영향을 받도록 하고 싶다면
캐릭터 컨트롤러 대신 Rigidbody를 사용하는 것이 좋다고 권장되고 있습니다.

- 반응도 하지 않고 , 물리학도 적용되지 않습니다.
- 물리에 반응 하지 않는 기본 collider가 내장됩니다. 그로 인해 transform으로 움직일 수 있으나 , 충돌로는 움직일 수 없습니다.
- 플레이어의 동작을 많이 기술할 수 있으나 , 모든 걸 스크립트에서 개발자가 직접 적어야 합니다.
- OnTrigger & OnCollision 대신에 이 클래스에 내장된 Method를 사용합니다.

OnControllerColliderHit

OnControllerColliderHit is called when the controller hits a collider while performing a Move.

9. Raycast

- 해당 GameObject에서 발사한 Ray(광선)와 다른 물체의 충돌을 감지하기 위해 사용합니다.
- Ray는 처음으로 감지한 단 하나의 collider만 Return 합니다. Ray를 발사한 GameObject도 default로 collider에 포함됩니다.

① Ray를 Unity에서 눈으로 확인하기

```
Debug.DrawRay(transform.position, new Vector3(0,-5,0), Color.green);
```

- 매개변수 : (Ray발사 지점 , Ray발사 방향 및 크기 , Ray의 색상)
- Unity에서 Ray를 눈으로 확인하기 위한 Log일 뿐 Ray의 물리적인 능력은 존재하지 않습니다.

② Ray로 GameObject 감지하기

```
private RaycastHit hit;
```

```
if (Physics.Raycast(transform.position, Vector3.down, out hit, 5f) == false)
```

- 매개변수 : (Ray발사 지점 , Ray발사 방향 , RaycastHit Object , Ray발사 크기)
- 이 method는 bool type 이며 , 아래로 5만큼의 크기로 hit라는 RaycastHit 객체를 발사했을 때 만나는 collider가 존재하면 true를 return 하고 hit에 만난 collider를 초기화시키고 , 존재하지 않으면 false를 return 합니다.
- 일단은 , 자기 자신은 ray의 collider로 감지 하지 않는 듯?

10. MonoBehaviour

- MonoBehaviour는 모든 스크립트가 상속 받는 기본 클래스입니다.
- C#을 사용하는 경우에는 명시적으로 MonoBehaviour를 상속받아야 합니다.
- 이를 상속받아 start(), Awake(), Update()등의 method를 사용할 수 있습니다. 또한 위에 적힌 5개의 특수한 변수를 사용할 수 있습니다.
- **gameObject는 스크립트가 component로 저장 된 GameObject를 나타내고 , transform은 스크립트가 component로 저장 된 GameObject의 위치를 나타냅니다.**
- 상속을 받는 다면 Unity의 메시지를 받을 수 있습니다.

gameObject 현재의 컴포넌트가 첨부된 게임 오브젝트를 나타냅니다. 컴포넌트(component)는 항상 게임 오브젝트에 첨부되어 있다.

tag 현재 게임 오브젝트의 태그를 나타냅니다.

transform 현재 GameObject에 첨부된 Transform를 나타냅니다. (첨부되지 않은 경우에 null)

hideFlags 오브젝트가 숨겨져있는 상태인지, 씬에 저장된 상태인지, 또는 사용자에게 의해서 수정가능한 상태인지를 확인합니다.

name 오브젝트의 이름을 나타냅니다.

11. C#의 포인터 : ref

```
private int tmp1 = 1;
private int tmp2 = 1;

private void Awake()
{
    NotRef(tmp1);
    Ref(ref tmp2);
    Debug.Log(tmp1 + " " + tmp2);
}

void NotRef(int a)
{
    a = 2;
}

void Ref(ref int a)
{
    a = 2;
}
```

- C#은 포인터가 없기 때문에 인자로 받은 변수의 값을 바꾸려면 ref를 사용합니다.

```
IEnumerator ShootGuidedMissile(ref GameObject guided_missile)
{
    Vector3 guided_missile_position = guided_missile.position;
    Vector3 enemy_player_position;
```

☞ (매개 변수) ref GameObject guided_missile

CS1623: 반복기에는 ref, in 또는 out 매개 변수를 사용할 수 없습니다.

12. Var Type & foreach 구문

1) Var Type

= 컴파일러에 의해 실행 과정에서 타입이 결정됩니다.

- 컴파일러가 타입을 자동으로 결정해주므로 개발자는 더욱 편해집니다.
- 지역변수에만 사용이 가능합니다.
- 선언과정에서 반드시 초기화를 해주어야 합니다.
- C++의 auto와 같습니다.

2) foreach

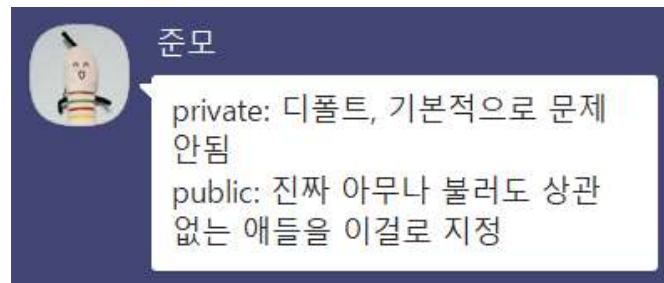
= C++의 범위 기반 for문입니다.

- var과 함께 사용합니다.

```
foreach(var i in obstacles_will_be_destroyed)
{
    Destroy(i);
}
```

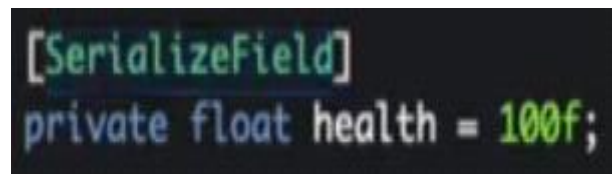
13. 접근지정자 : 미래의 나를 위해 Private를 적극적으로 사용합시다.

- Unity는 public 변수가 생성될 때 자동으로 Inspector 인터페이스(Unity Editor)와 연결해서 값을 노출합니다.
- public 변수는 코드의 어디에서든 값을 변경하고 저장할 수 있으므로 소프트웨어 디자인 관점에서 위험해 보입니다. 왜냐하면 예기치 않게 값이 변경되면(2인 개발 이상) 여러 가지 버그가 발생하기 때문입니다.
- C#은 멤버변수들의 기본 속성이 private이기 때문에 private 키워드를 꼭 붙여줄 필요는 없습니다.



- 한 달 정도가 지났을 때 자신의 코드를 읽을 경우나 타인의 코드를 읽는 경우 , public으로 선언 된 것에 대한 분석을 할 때는 프로젝트 전체를 뒤져야 합니다. (ctrl + f12를 눌러서 어딘지 확인할 수 있음)
- private은 무조건 현재의 스크립트에만 관련 구문이 존재함.
- 솔직히 UI나 특정 GameObject를 public으로 연결해도 내가 혼자 게임을 만들면 변경을 하지 않겠지만 2명 이상의 프로젝트에서는 누가 그걸 변경할지도 모르고 , 변경했을 때 발생하는 ERROR에 대한 위험부담이 생깁니다. 그걸 미연에 방지하는 것이 private입니다.
- **private으로 선언하면 유지보수가 간편하며 , 미래의 내가 할 일이 적어집니다.**

<SerializeField : Private의 기능은 유지하지만 Unity Editor의 Inspector에서만은 예외로 값을 변경할 수 있는 기능>



- 원래는 private로 설정한 객체는 Unity Editor의 해당 Inspector에 보이지 않습니다.
- 하지만 , **SerializeField를 붙인 private 객체는 Inspector에서 보이고 , 수정이 가능 합니다.**
- 당연히 private이므로 Script(visual studio)에서는 수정할 수 없습니다.

<좋은 가독성>

```
[SerializeField] private DialogueLineChannelSO _openUIDialogueEvent = default;
```

14. 매우 중요한 개념이지만 자주 실수하는 2가지 개념

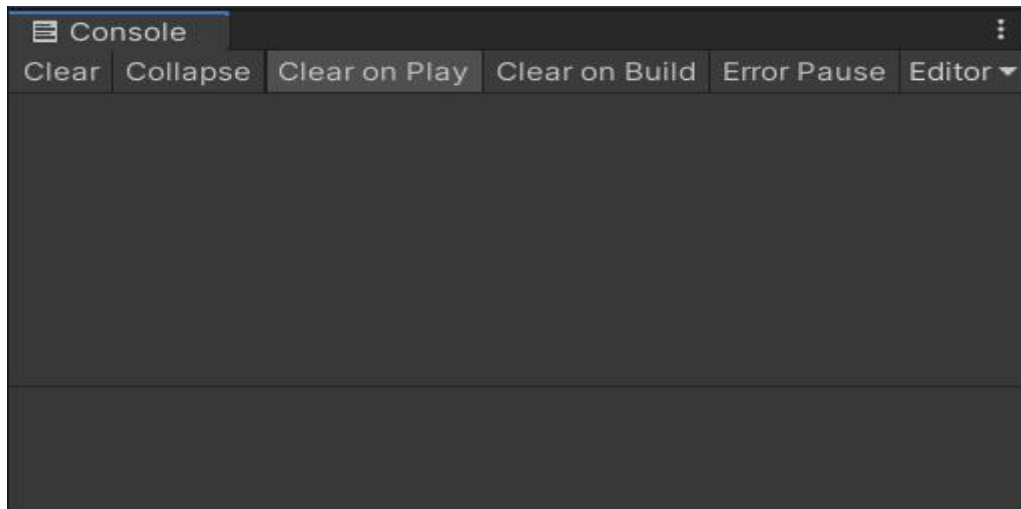
1) Script의 객체로 GameObject에 연결하면 Script도 Component가 됩니다.

- Prefab에 script를 넣어주면 , 해당 script는 언제든지 prefab으로 만든 GameObject에서 GetComponent를 통해 사용할 수 있습니다. 다시 말해 해당 script는 연결한 prefab의 다른 script에서 Component로 사용할 수 있습니다. 그러므로 굳이 해당 Script를 Singleton으로 만들 필요가 없습니다. (GameObject도 마찬가지)
- prefab 자체가 객체가 여러 개가 되므로 Singleton으로 만들 생각을 절대 하면 안 되기도 합니다.

2) 두 GameObject p1과 p2에 둘 다 연결된 스크립트에서 p1이 장애물을 만났을 때 p1에서만 발동하도록 하는 경우

- 지금까지는 if를 써서 이게 p1인지 검사하고 실행시켰으나 그럴 필요가 없습니다.
- p1에서 발생했으므로 이 이벤트가 발생한 gameObject는 p1이므로 스크립트의 gameObject가 p1이 됩니다. 그러므로 gameObject를 통해 연결된 component(다른 스크립트 , rigidbody 등등)을 자유롭게 private로 가져올 수 있습니다.

15. Console Tip



- **Collapse**를 비활성화 시켜서 동일한 로그여도 시간에 따라 출력되게 합니다.
- Collapse가 활성화되면 동일한 로그라면 오른쪽에 출력될 때마다 숫자가 증가하기만 하므로 , 언제 어떤 상황에 발생한 로그인지 알 수 가 없습니다.
- **Clear on Play**를 활성화 시켜서 게임을 시작할 때 마다 로그를 초기화 합니다.

16. 배열

1) 기본 배열

```
int arr[5];  
arr[2] = 2;  
  
int[] arr2 = new int[5];  
arr2[2] = 2;
```

- C#은 배열을 반드시 참조형으로 선언해야 합니다.
- arr2는 참조하는 변수일 뿐 , 5개의 배열 instance를 만들어야 합니다.

2) 2차원 배열

2차원 배열의 선언 방법

```
데이터형식[,] 배열이름 = new 데이터형식 [2차원길이, 1차원길이];
```

2 x 3 크기의 int형 2차원 배열 선언

```
int[,] array = new int[2,3];  
array[0,0] = 1;  
array[0,1] = 2;  
array[0,2] = 3;  
array[1,0] = 4;  
array[1,1] = 5;  
array[1,2] = 6;
```


17. C#은 Struct와 Class가 완벽히 동일하므로 언제나 Class를 사용합니다.

1) C++ Struct

```
struct str
{
    int a;
    int b;
};

str obj;
obj = { 1, 2 };
cout << obj.a << " " << obj.b;
```

<- C++ 코드

- C++에서는 구조체를 사용하는 것과 클래스를 사용하는 것의 차이가 분명히 존재합니다. 클래스는 조금 복잡하지만 구조체는 데이터를 모아둔 집합체에 불과하여 매우 간단하고 제약도 적었습니다.

2) C# Struct

```
public struct str
{
    public int a;
    public int b;
};

str obj;
obj = { 1, 2};
```

<- C# 코드

- 하지만 C#은 C++처럼 구조체의 사용이 자유롭지 않습니다.
- 구조체의 멤버에 값을 할당하려면 구조체와 구조체의 모든 멤버에 public을 적용해야 하며 , 초기화도 클래스와 비슷한 형식으로 해야 합니다.
- 다시 말해 , **C#의 구조체는 간단하지 않고 클래스와 비슷하게 불편합니다.**
- 구조체와 클래스의 차이가 모호한데 굳이 구조체를 써야 하는 지에 대한 의문이 생깁니다.

3) C# Class

```
public struct records
{
    public string p1_name;
    public string p1_score; //int형 데이터도 string으로 이용
    public string p2_name;
    public string p2_score;

    public records(string a , string b , string c , string d)
    {
        this.p1_name = a;
        this.p1_score = b;
        this.p2_name = c;
        this.p2_score = d;
    }
};
```

```
public class records
{
    public string p1_name;
    public string p1_score; //int형 데이터도 string으로 이용
    public string p2_name;
    public string p2_score;

    public records(string a , string b , string c , string d)
    {
        this.p1_name = a;
        this.p1_score = b;
        this.p2_name = c;
        this.p2_score = d;
    }
};
```

```
records tmp = new records(row[0] , row[1] , row[2] , row[3]);
```

- 좌측의 struct를 우측의 class로 바꾸어도 코드의 아무런 변화가 없습니다. 즉 , C#에서 struct와 class는 동일합니다.
- 그러므로 구조체로 만들 것을 클래스로 만들고 , 선언도 객체를 생성하며 하고 , 그 과정에서 생성자로 초기화합니다.

18. Const

```
public int BLOCK_CNT_FOO { get; private set; }
public const int BLOCKS_CNT = 5;
```

- = Const는 기본적으로 Static 상태이므로 Static을 붙일 수 없습니다.
- Const로 선언한 상수는 애시 당초 값을 바꿀 수 없기 때문에 private set보다 강력합니다.
- 게임 전체에서 사용해야 하는 상수는 Singleton으로 제작된 Constant Script로 만들어서 public const 형태로 관리합니다. public으로 선언함에도 불구하고 const의 고유속성으로 인해 값을 변경할 수 없습니다.
- 이 스크립트에서만 사용하고 , 다른 스크립트에서 자동완성에 나타나지 않도록 하려면 private로 선언하는 것이 좋습니다.
- Singleton에 const로 선언한 상수는 '클래스이름.Instance.상수'가 아닌 '클래스이름.상수'로 참조할 수 있습니다.

19. Enum이 아닌 상수 배열

Yes, but you need to declare it `readonly` instead of `const`:

```
public static readonly string[] Titles = { "German", "Spanish", "Corrects", "Wrongs" };
```

The reason is that `const` can only be applied to a field whose value is known at compile-time. The array initializer you've shown is not a constant expression in C#, so it produces a compiler error.

Declaring it `readonly` solves that problem because the value is not initialized until run-time (although it's guaranteed to have initialized before the first time that the array is used).

Depending on what it is that you ultimately want to achieve, you might also consider declaring an enum:

```
public enum Titles { German, Spanish, Corrects, Wrongs };
```

```
private static readonly int[] path_const = { 7, 8, 9, -1, 1, -9, -8, -7 };
```

- `const` 대신에 `static`과 `readonly`를 이용해서 `const`와 비슷한 효과를 냅니다.
- `const`는 언제나 값을 변경할 수 없지만, `readonly`는 값을 변경할 수도 있습니다. 그래서 왜 사용해야 하는지 모르겠습니다.
- 지역함수, 지역변수에서 사용을 하면 아래의 ERROR가 발생하며 사용할 수 없으므로, 클래스의 멤버 변수로만 사용해야 합니다.

CS0106: 이 항목의 'readonly' 한정자가 유효하지 않습니다.

20. C# Enum

1) 기본형

- 가로로 나열할지 , 세로로 나열할지는 개발자가 직접 가독성을 고려하여 결정합니다.

```
enum WEAPON
{
    DEFAULT,
    POWER = 777,
    TRIPPLE_SHOT,
    GUIDED_MISSILE
}
```

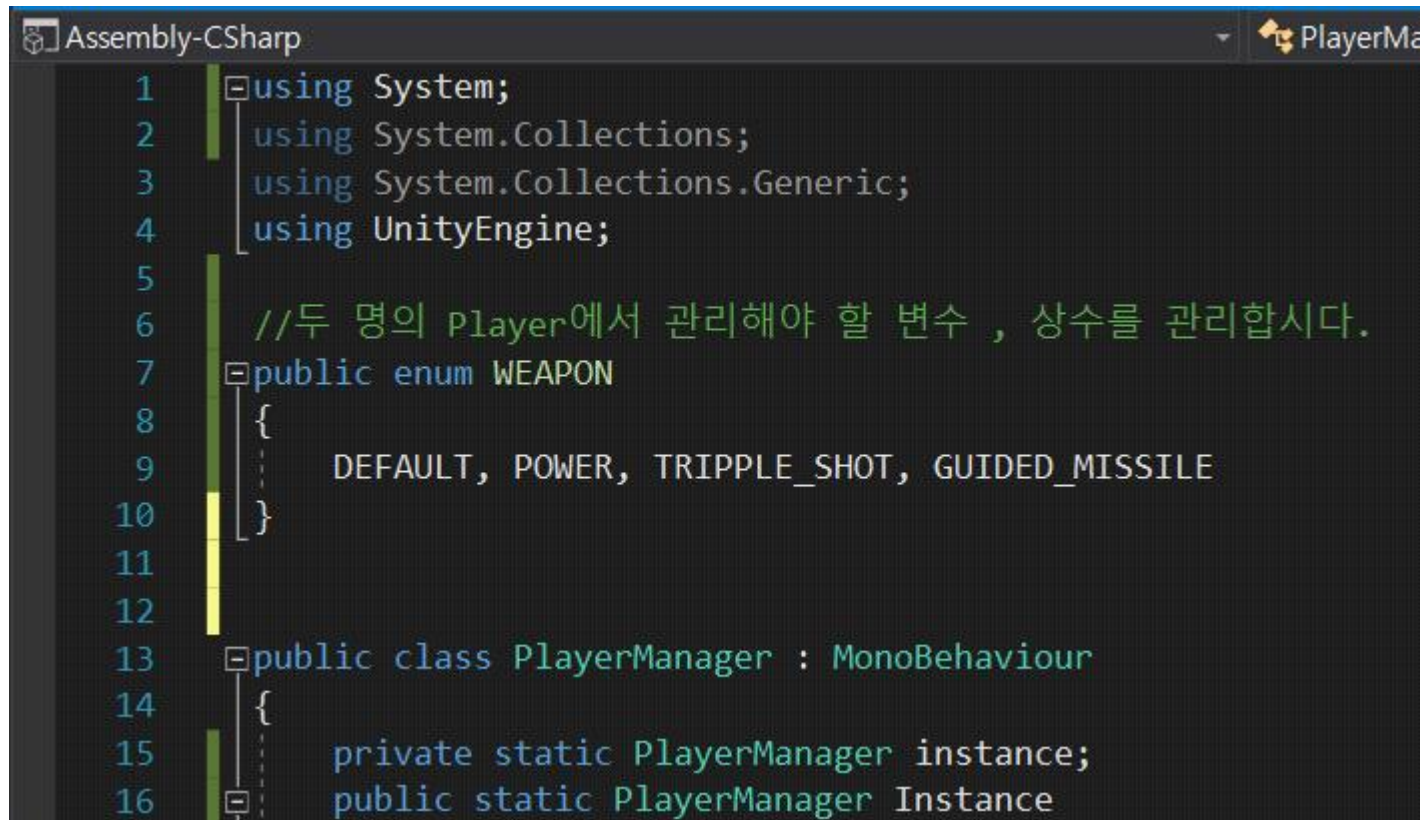
```
enum WEAPON
{
    DEFAULT, POWER, TRIPPLE_SHOT, GUIDED_MISSILE
}
```

```
(int)WEAPON.DEFAULT
```

- 값을 지정할 수 있으며 , 값을 지정하지 않으면 이전의 값 + 1로 자동으로 지정됩니다.
- Return Type이 모호하여 cast를 통해 값을 지정합니다.
- Enum에서는 정수만 사용가능 하므로 , Float을 사용할 수 없습니다.

Nice share . but enums type cannot be of floats and doubles . it can only be type of complete numbers. make correction.

2) Unity의 모든 스크립트에서 사용 할 Enum (전역)

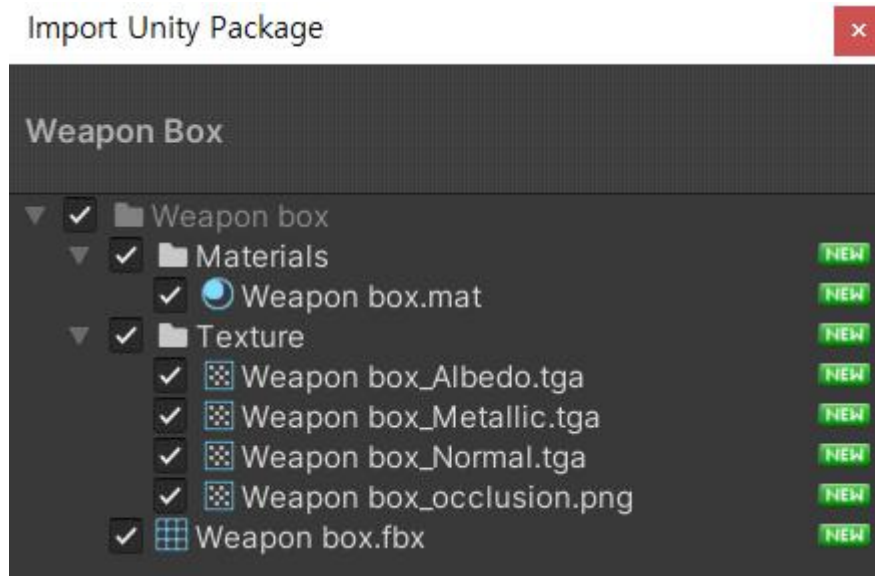


```
Assembly-CSharp PlayerMa
1  using System;
2      using System.Collections;
3      using System.Collections.Generic;
4      using UnityEngine;
5
6      //두 명의 Player에서 관리해야 할 변수 , 상수를 관리합니다.
7      public enum WEAPON
8      {
9          DEFAULT, POWER, TRIPPLE_SHOT, GUIDED_MISSILE
10     }
11
12
13     public class PlayerManager : MonoBehaviour
14     {
15         private static PlayerManager instance;
16         public static PlayerManager Instance
```

- 클래스 외부에 public으로 선언하면 모든 Script에서 enum을 사용할 수 있습니다.

21. Unity Asset에서 타인이 제작한 Texture 가져와서 적용시키기

<Import 받을 Weapon Box 데이터>



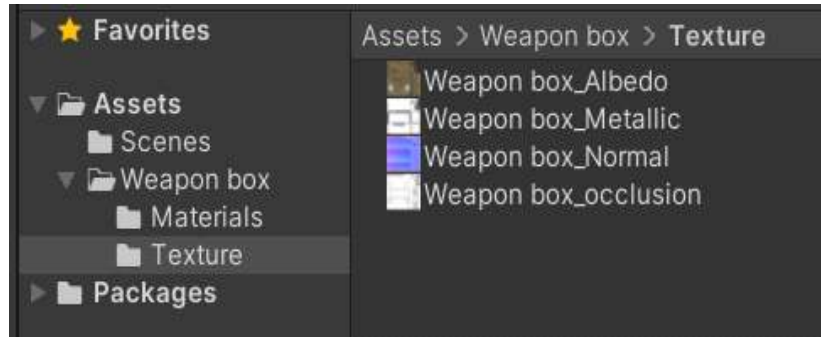
1) Mesh 설정



- 기존 GameObject는 Cube , Sphere 같은 기본 물체의 Mesh를 갖고 있지만 , 디자이너의 사용자 설정 Mesh를 사용합니다.
- 가져온 데이터 중에서 “Weapon box”라고 되어있는 Mesh를 적용합니다.

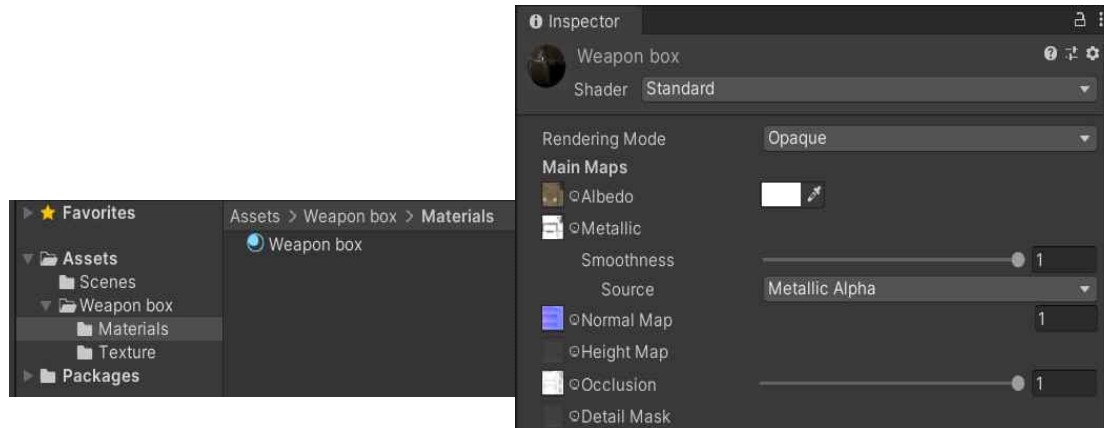
2) Material 설정

① Texture



- Import 한 데이터는 Materials와 Texture로 구성되어 있습니다.
- Texture는 Material의 구성요소입니다.

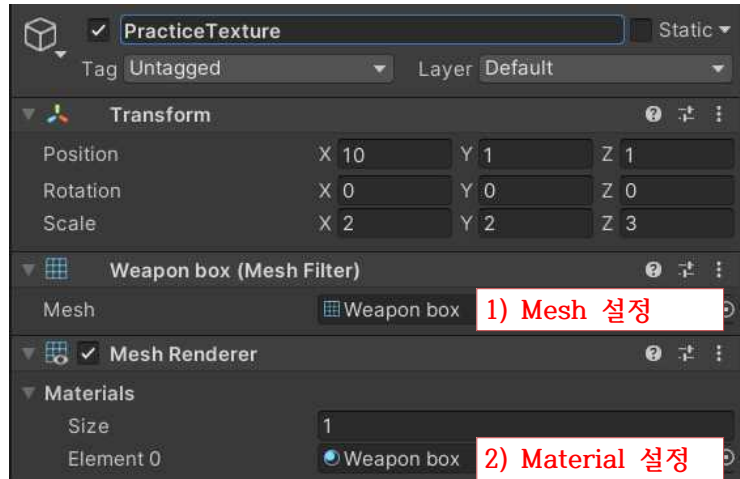
② Material : 실질적으로 개발자가 필요한 것



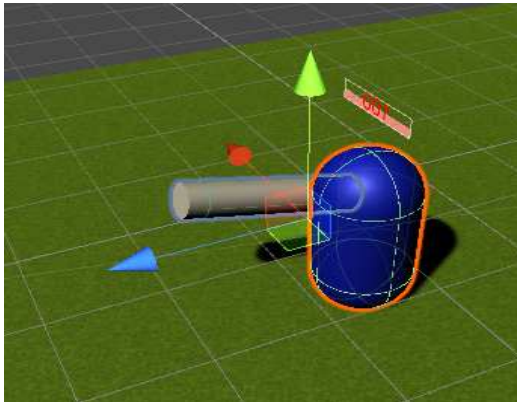
- Texture들을 디자이너가 직접 Material에 넣어서 1개의 완성된 Material을 만들어주었습니다. 만약 Texture만 존재하는 것을 Import 해오면 제가 직접 Material을 만들고 Texture를 추가하면 됩니다.

- “Weapon box” Material을 제가 원하는 GameObject의 Material에 추가합니다.

3) GameObject에 Import로 받아온 이미지(Mesh + Material) 입히기



22. 객체의 방향

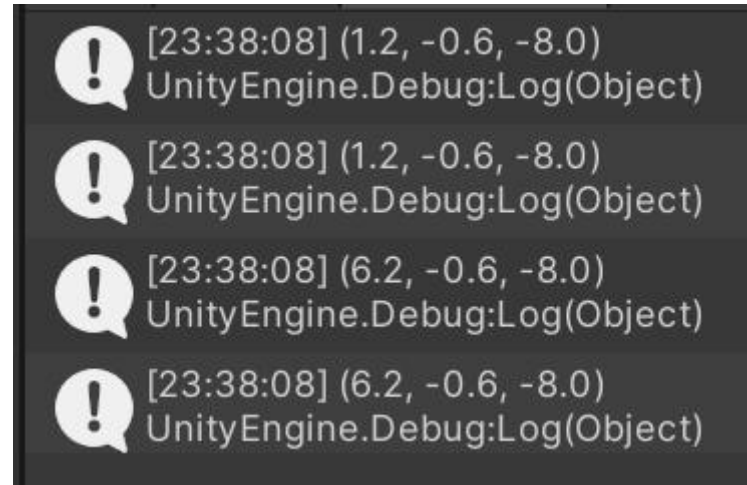


- transform의 멤버 변수를 이용합니다.
- Vector3의 forward , up , right와는 다른 개념입니다.

- ① transform.forward : 바라보는 방향 (파란색 화살표)
- ② transform.right : 오른쪽 방향 (빨간색 화살표)
- ③ transform.up : 위 방향 (초록색 화살표)

23. 게임오브젝트 복사 (동일시)

```
public GameObject a;  
private GameObject b;  
  
private void Awake()  
{  
    b = a;  
    Debug.Log(a.transform.position);  
    Debug.Log(b.transform.position);  
  
    a.transform.position += new Vector3(5, 0, 0);  
    Debug.Log(a.transform.position);  
    Debug.Log(b.transform.position);  
}
```



[23:38:08] (1.2, -0.6, -8.0)
UnityEngine.Debug:Log(Object)
[23:38:08] (1.2, -0.6, -8.0)
UnityEngine.Debug:Log(Object)
[23:38:08] (6.2, -0.6, -8.0)
UnityEngine.Debug:Log(Object)
[23:38:08] (6.2, -0.6, -8.0)
UnityEngine.Debug:Log(Object)

- public을 이용해서 Unity Editor에 존재하는 GameObject a를 Script에 추가했습니다.
- 'b = a'를 통해 GameObject b를 a로 초기화시켰습니다.
- 직접 테스트 해 본 결과 b와 a는 같게 되며 , a가 움직이면 b도 움직인 것으로 결과가 나옵니다.
- 예전에 애송이 시절에 이렇게 실패한 적이 있어서 요즘 계속 그런데 , 게임 개발을 해보면서 좀 더 확실하게 내용을 추가해보자.
- 이게 포인터처럼 주소 자체를 동일시시키는 것인지 , 참조자 인지 , 아니면 다른 방식인지 알고 싶은데 뭐라고 검색해야 할까.
- 21/08/18 :

```
if (position_z < MAP_LINE)  
{  
    victim_player = player1;  
}  
else  
{  
    victim_player = player2;  
}
```

- GameObject 변수인 victim_player에 GameObject인 player1 , player2를 넣고 동작시켜 본 결과 조건에 따라서 알맞은 GameObject를 복제해서 올바르게 코드가 수행됩니다.

24. One-Line Function

- 성능 감소는 거의 없고 , 가독성은 항상 시키므로 한 줄 짜리 함수를 적극적으로 사용합시다.

한줄짜리 함수의 첫번째 유용성은 리팩토링 책에서 소개하듯이 가독성때문이다.

1. if (date.before(SUMMER_START) || date.after(SUMMER_END))를
2. if (notSummer(date)) 와 같이 바꾼다면 (Decompose Conditional)

Reading하기가 좀더 쉽다. 아마도 notSummer는 private 함수일것이고 함수명으로 좀더 의미를 명확하게 전달하기 때문에 굳이 함수를 들여다보아야 해서 복잡해지진 않는다.(이런 가독성에 대해서는 이전글에서 언급했듯이 동전의 양면이다. 만약 함수명이 적절하게 지어지지 않는다면 좀더 복잡해질 것이다. Framework에서 구조의 문제도 마찬가지이다.)

그럼에도 한줄짜리 함수를 싫어하는 사람들은 꽤 있었지만 현재는 그 수가 많이 줄었다. 이는 툴의 발전에도 많이 영향을 받았다. 이전에 text editor에서 코드를 작성하던 주라기 시대에는 툴에 함수 바로가기 등의 assist 기능이 없어서 text search로 함수 내용을 확인해야 하던때라 가능한 하나의 함수에서 해주는게 미덕인 시절이 있었다.

한줄 함수를 싫어하는 다른 이유로는 C 시절에는 Function Call Stack이 쌓이게 됨으로써 발생하게 되는 비효율을 이유로 들었으나 자바에서는 C와 달리 이러한 비효율이 성능상에서 차지하는 부분이 아주 작다. 이미 자바 라이브러리 자체가 수십개의 Function Stack Call을 하고 있으므로 거기에 몇개쯤 더한다고 거의 영향이 없다. (그러나 C나 C++은 영향이 있다. 그 영향 자체는 상대적으로 적기에 많다 적다고 할수 없지만 확실히 자바보다는 영향이 많다.)

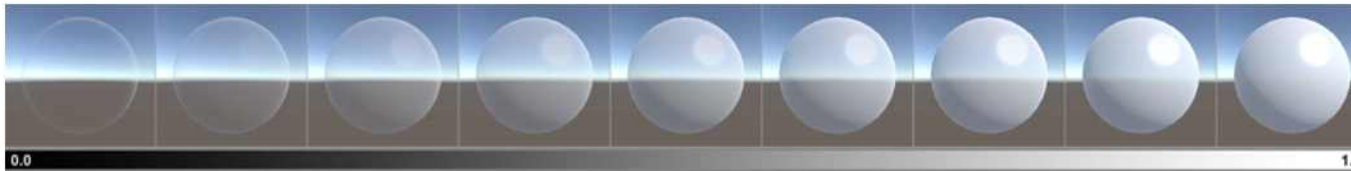
<Reference>

- <https://bleujin.tistory.com/139>
- <https://www.codereadability.com/are-one-line-functions-a-good-idea>

25. Color에서 alpha값을 변경해도 투명도가 변경되지 않는 이유

투명도(Transparency)

Albedo 컬러의 알파 값에 따라 머티리얼의 투명도 레벨이 조절되지만, 머티리얼에 대한 렌더링 모드가 투명 모드 중 하나로 설정되어 있고 **불투명**이 아닌 경우에만 그렇습니다. 위에서 설명한 것처럼 올바른 투명도 모드를 선택하는 것이 중요합니다. 투명도 모드에 따라 반사 및 스페큘러 하이라이트가 전체 값으로 계속 표시될 것인지, 아니면 투명도 값에 따라 페이드 아웃될 것인지 결정되기 때문입니다.



사실적인 투명 오브젝트에 적합한 투명 모드를 사용한 0에서 1까지의 투명도 값 범위

26. 자주 사용하는 기초 함수 및 로직

*** Google에서 검색하면 바로 나오지만 , 자주 사용하기 때문에 암기를 해놓으면 편리할 것 같습니다. ***

1) 키보드로 입력 받기

```
Input.GetKeyDown(KeyCode.DownArrow)
```

- GetKey : 해당 키를 누르고 있을 때 계속 True를 return.
- GetKeyDown : 해당 키를 눌렀을 때 True를 return.
- GetKeyUp : 해당 키를 눌렀다 뺐을 때 True를 return.

2) 두 Vector 사이의 거리

```
Vector3.Distance (cube1.transform.position, cube2.transform.position);
```

3) 게임 전체 관리하기 (빌드한 게임에서 동작하는 지 확인 하자)

① 게임 일시정지

```
if(Input.GetKeyDown(KeyCode.Escape))
{
    Time.timeScale = 0;
    UIManager.Instance.pause_screen.gameObject.SetActive(true);
}
```

- 'Time.timeScale = 0' : 게임이 일시정지 됩니다.
- 'Time.timeScale = 1' : 게임이 재시작 됩니다.

② 게임 종료

Application.Quit()

```
public class ExampleClass : MonoBehaviour {
    void Update() {
        if (Input.GetKey("escape"))
            Application.Quit();
    }
}
```

빌드한 어플리케이션에서는 정상적으로 종료가 실행되지만, 유니티 에디터에서는 게임 플레이가 멈추지 않는다.
따라서 에디터와 프로그램 실행을 구분지어서 코딩해줘야 한다.

③ 게임 재시작

*** 추후에 ***

4) Material의 색상 변경

```
Renderer capsuleColor;  
  
void Start()  
{  
    capsuleColor = gameObject.GetComponent<Renderer>();  
}  
  
void Update()  
{  
    if (Input.GetMouseButtonDown(0))  
    {  
        if(capsuleColor.material.color != Color.blue)  
        {  
            capsuleColor.material.color = Color.blue;  
        }  
        else  
        {  
            capsuleColor.material.color = Color.red;  
        }  
    }  
}
```

```
GameObject explosion = Instantiate(explosion_prefab, new Vector3(0, 3, 0), transform.rotation);  
Material explosion_material = explosion.GetComponent<Renderer>().material;  
Color explosion_color = explosion.GetComponent<Renderer>().material.color;
```

```
explosion_material.color = new Color(1, rgb_g, rgb_b, 0); //Red -> Orange  
explosion_color = new Color(1, rgb_b, rgb_b, 0);
```

- explosion_material.color를 이용해서 변경하는 코드는 올바르게 동작하지만 , explosion_color를 이용해서 변경하는 코드는 동작하지 않습니다.
- renderer class에 대해서 공부합시다.

<Unity 중급 기능> 20/07/31

1. 프로퍼티

= C#에서 클래스의 멤버 변수를 선언할 때 권장하는 정보 은닉 선언 방법

- 다른 스크립트에서 현재 스크립트의 값을 읽고 싶지만, 값을 변경할 수 없게 합니다. 다른 스크립트에서 값을 변경할 수 있으면 예상치 못한 변경이 발생하기 때문에 프로퍼티의 사용은 필수입니다.
- 기본적으로 property로 설정한 멤버 변수는 public 형태입니다.
- get은 기본적으로 private로 설정하면 ERROR를 발생시키므로 접근 지정자를 지정하지 않습니다. 그러면 public 상태로 남아 어디서든 프로퍼티 멤버 변수의 값을 확인 할 수 있습니다.
- set을 private로 선언하면 해당 클래스 이외의 공간(다른 클래스 OR Unity)에서는 값을 설정하거나 변경할 수 없습니다. Unity에서는 아예 inspector창에서 해당 스크립트의 프로퍼티 멤버 변수의 값을 조절하는 기능이 사라집니다.
- set 까지 public으로 하면 두 method가 모두 public이 되므로, public 멤버 변수와 차이가 없기 때문에 프로퍼티를 설정하는 의미가 없습니다.
- get과 set은 access method 이며, 반드시 호출된다는 보장은 없지만 호출될 가능성이 매우 높기 때문에 미리 정의해 놓습니다.

1) 간단한 형태

```
public int simple_property { get; private set; }
```

- get은 public이므로 어디서든 프로퍼티 변수를 확인할 수 있으나, set이 private이므로 자신의 클래스 이외의 공간에서는 프로퍼티 변수 값을 설정하거나 변경할 수 없습니다.
- C# 3.0 이상부터 가능합니다.

2) get과 set에 내용 추가

- 아래의 두개는 동일한 형식이지만 Enter(개행)의 차이만 주었습니다.

① 가독성_1 : 코드가 짧을 때 간단하게 구현

```
public int weapon_cnt{get{return weapon_cnt;} set{weapon_cnt = GetWeaponCount();}}
```

② 가독성_2 : 코드가 길 때 복잡하게 구현

```
public int b
{
    get
    {
        Debug.Log("Getting obj.b");
        return b;
    }
    private set
    {
        b = 10;
    }
}
```

```
Debug.Log(obj.b);
obj.b = 3;
```

- 프로퍼티로 설정한 멤버 변수 b의 값을 다른 클래스에서 확인하는 get은 가능합니다.
- 하지만 프로퍼티로 설정한 멤버 변수 b의 값을 다른 클래스에서 설정하는 set은 불가능합니다.

3) 불편한 프로퍼티

= private로 int a를 설정하고 그 아래에 public int A{get , set}으로 private 변수의 get과 set을 가능하게 해주는 방식도 있지만 위의 2 방식이 훨씬 간편한 것 같습니다.

2. Static

1) 기본 개념

- Static 변수 = 클래스 변수
- Static Method = Class Method
- 다른 클래스(Script)에 존재하는 변수나 함수를 사용할 때 현재 스크립트에서 해당 스크립트의 객체를 만들지 않고 클래스를 이용해 직접 바로 사용하기 위해 사용하는 구문
- Static을 사용하면 전역화(Global) 되어 버리기 때문에 귀찮게 다른 스크립트를 현재 스크립트로 연결하고 , 다른 스크립트의 객체를 만들지 않고 사용할 수 있습니다.
- 이걸 확장해서 Class 전체에 적용시키는 것이 Singleton Pattern입니다.

<Example : 다른 스크립트에서 PlayerManager 스크립트에 존재하는 변수와 함수를 클래스 이름(=스크립트 이름)을 이용해서 사용합니다.>

```
Debug.Log(PlayerManager.good);  
PlayerManager.GetWeaponCount();
```

2) 좋은 선언 방식

```
public static int good { get { return good; } private set { good = 777; } }  
  
public static int GetWeaponCount()  
{  
    return Enum.GetValues(typeof(WEAPON)).Length;  
}
```

3) 좋지 못한 선언 방식

```
public static int good { get { return good; } private set { good = 777; } }
```


3. Singleton Pattern (싱글턴)

1) 정의

- 일반적인 Class(Script)는 Unity World에 존재하는 복수의 GameObject에 Component가 될 수 있습니다(=인스턴스화).
- 하지만 , Class(Script) 1개가 Unity World에 존재하는 단 1개의 GameObject에서만 인스턴스화가 되는 경우에 이를 Singleton GameObject 라고 부릅니다.

ex) GameManager Script는 Unity World에 존재하는 GameManager(GameObject)에만 연결하고(인스턴스화) 다른 GameObject에는 절대 연결시키지 않습니다.

2) 특징

```
public static GameManager Instance
{
    get
    {
        if (instance == null) instance = FindObjectOfType<GameManager>();

        return instance;
    }
}
```

- Singleton으로 만든 클래스는 get 멤버만을 가지고 있는 읽기 전용 프로퍼티인 Instance를 이용해서 다른 GameObject에서 접근하기가 매우 간단합니다.
- 다른 GameObject에서 Unity에서 드래그를 해서 연결하거나 GetComponent를 사용해서 연결하는 번거로운 일을 수행하지 않아도 됩니다.
- 이제는 “GameManager.Instance.멤버변수”처럼 클래스 이름과 Instance로 다른 스크립트에서 GameManager의 멤버에 접근할 수 있습니다.
- Singleton으로 만든 클래스의 모든 멤버들은 다른 GameObject에서 접근이 가능하도록 하여야 하므로 public으로 설정합니다. 당연히 프로퍼티 , 멤버(변수 + method)를 private로 설정하면 다른 GameObject에서 사용이 불가능 합니다.
- 싱글턴의 멤버 변수를 public으로 설정했을 때 , 해당 값을 다른 스크립트에서 변경 가능한지 확인해보자 , Instance가 set은 설정을 안했지만 멤버 변수는 어떻게 반응하는지 궁금함.

3) 사용방법

① GameManager 클래스의 객체로 instance를 할당합니다.

```
public static GameManager instance;
```

- Static으로 선언한 instance는 메모리의 Code 영역 OR Data 영역에 프로그램 종료 직전 까지 사라지지 않습니다.
- 선언만 하면 instance에는 null이 저장됩니다. public static GameManager instance = null; 과 동일
- 보안을 위해 객체 instance를 private로 선언하고 프로퍼티 Instance로 접근권한을 설정할 수 있지만 일단은 위의 방식으로 간단하게 선언합니다.

② 예외처리 및 객체 형성

```
private void Awake()
{
    if(instance != null)
    {
        Destroy(gameObject);
        return;
    }
    instance = this;
}
```

- static 객체는 초기화 하지 않고 선언하였기 때문에 아무 것도 저장하지 않아 null값이 저장됩니다.
- Singleton은 객체가 반드시 1개이므로 instance에 null이 아닌 다른 객체가 저장되어 있다면 예외처리를 통해 현재의 gameObject(GameManager)를 삭제합니다.
- 반면에 instance에 초기화 된 GameManager(GameObject) 객체가 없다면, this는 GameManager(Class = Script)를 담고 있는 GameManager(GameObject)를 가리키고 이를 instance에 초기화 시킵니다.
- 그 결과 게임 내에는 GameManager(Class = Script)로 만들어진 단 1개의 GameManager(GameObject)가 탄생합니다.

③ 다른 GameObject에서 Singleton으로 만든 GameObject 사용하기

```
//멤버 변수  
GameManager.instance.a = 3;  
Debug.Log(GameManager.instance.a);  
  
//method  
GameManager.instance.DoSomething();
```

- instance는 static으로 선언했으므로 클래스 명인 GameManager를 붙여서 쉽게 사용할 수 있습니다.

4. Coroutine(루틴) VS Thread(스레드)

1) 동기식 처리 모델 vs 비동기식 처리 모델

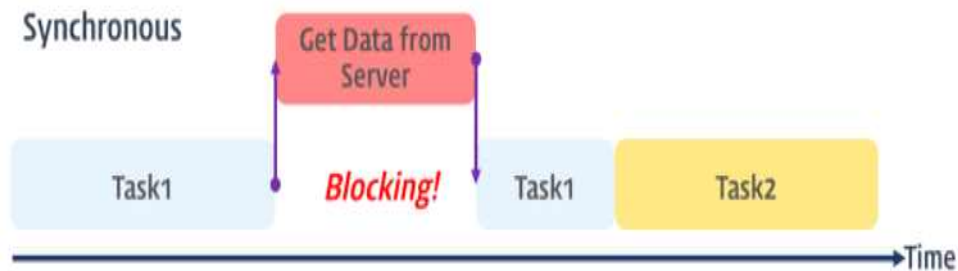


(a) 동기



(b) 비동기

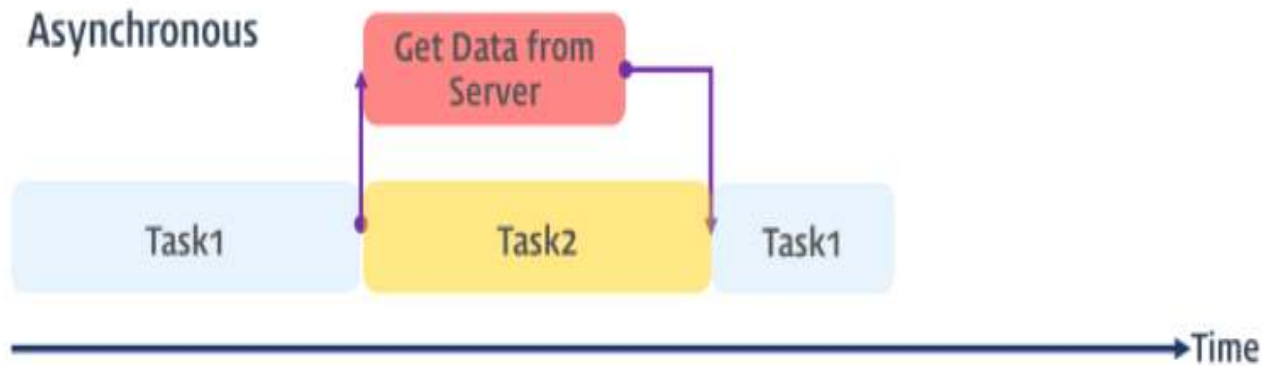
① 동기식(Synchronous) 처리 모델



동기식 처리 모델(Synchronous processing model)

- 직렬적으로 태스크(task)를 수행합니다. 태스크는 순차적으로 실행되며 어떤 작업이 수행 중이면 다음 작업은 대기하게 됩니다.
 - 서브루틴 간에 **명확한 실행 순서가 존재**하기 때문에 작업이 A,B,C 순으로 실행되기를 기대한다면 반드시 A->B->C의 순서로 실행되어야 합니다. 그러므로 B는 A가 실행이 완료되기를 기다리며 , C는 A와 B가 실행 완료되기를 기다립니다.
 - **Coroutine**을 동기식 처리 모델이라고 확실히 말할 수는 없으나 적어도 비동기식 처리 모델은 절대 아니며 , 명확한 실행 순서가 존재합니다.
- ex) 서버에서 데이터를 가져와서 화면에 표시하는 작업을 수행할 때, 서버에 데이터를 요청하고 데이터가 응답될 때까지 이후 태스크들은 블로킹(blocking, 작업 중단)됩니다.

② 비동기식(Asynchronous) 처리 모델



비동기식 처리 모델(Asynchronous processing model)

- 병렬(Parallel)적으로 태스크(task)를 수행합니다. 태스크가 종료되지 않은 상태라 하더라도 대기하지 않고 다음 태스크를 실행합니다.
- 서브루틴 간에 명확한 실행 순서가 존재하지 않기 때문에 A->B->C의 순서로 실행될 수도 있고, B->C->A의 순서로 실행될 수도 있습니다. 따라서 프로그래밍 시에 반드시 이점을 고려해야 합니다. 이러한 경우에는 서브루틴이 완료되었다는 것을 전달하기 위해 callback 패턴을 사용하기도 합니다.
- 대표적인 비동기식 처리 모델로는 Thread Programming과 Firebase가 있습니다.

This means that any Firebase API that needs to deal with data on disk or network is implemented in an *asynchronous* style.

ex) 서버에서 데이터를 가져와서 화면에 표시하는 태스크를 수행할 때, 서버에 데이터를 요청한 이후 서버로부터 데이터가 응답될 때까지 대기하지 않고(Non-Blocking) 즉시 다음 태스크를 수행한다. 이후 서버로부터 데이터가 응답되면 이벤트가 발생하고 이벤트 핸들러가 데이터를 가지고 수행할 태스크를 계속해 수행합니다.

2) Coroutine

= 특정 작업을 시간 혹은 다른 작업의 처리가 완료되었을 때 단계적으로 진행 시키는 함수 입니다. 다시 말해 , 다른 작업이 완료될 때까지 현재 작업을 기다리는 루틴을 사용할 수 있습니다. 결론적으로 작업 처리의 순서(Scheduling)를 개발자가 제어할 수 있습니다.

- Coroutine은 Thread가 아닙니다.
- Coroutine은 비동기가 아니기 때문에 작업들이 동시에 처리되지 않습니다.
- 매 Frame 마다 Update 구문을 확인하지 않기 때문에 프로그램의 성능 향상에 큰 영향을 미칩니다.
- 코드의 가독성이 좋아집니다.

3) Thread

= 한 프로세스 내에 존재하는 실행 흐름.

- 프로세스를 쪼갠 미니 프로세스라고 이해했습니다.
- Thread는 비동기적으로 처리하기 때문에 다른 Thread와 함께 각각의 작업을 동시에 처리 할 수 있습니다.
- 현대 CPU는 다중 코어로 발전해서 OS는 자원들을 충분히 활용하기 위해 병렬 프로그래밍을 하게 되는데 , 병렬 처리의 핵심도구가 Thread 입니다.
- 코드가 복잡합니다.
- 완벽해보이지만 Thread 간의 자원 경쟁(상호 배제)과 데드락이라는 문제점이 존재합니다.

4) Reference

- <https://3dmpengines.tistory.com/2035>
- <https://silisian.tistory.com/44>
- <https://showx123.tistory.com/62>
- 동시성(Concurrency) vs 병렬성(Parallelism) : <https://seamless.tistory.com/42>

5. Unity에서 Coroutine 사용하기

- 반복문 + **yield return**으로 시간 제어를 한 코루틴은 **update**와 **fixed_update**와 동일한 기능을 수행하나 , 시간을 제어할 수 있습니다.
- 함수를 호출하게 되면 , 함수는 자신의 역할을 모두 수행한 이후 자신을 호출했던 메인 루틴으로 제어 권한을 넘기고 자신은 서브루틴이 됩니다.
- 모든 코루틴은 IEnumerator 형식을 반환해야 하는데 , 최소한 하나의 yield문을 포함하고 있어야하고 StartCoroutine으로 실행되어야 합니다.
- yield문은 조건이 충족될 때 까지 코루틴의 실행을 미루는 특별한 구문 입니다.
- yield break로 코루틴 내부의 반복문을 종료 시킬 수 있습니다.
- Coroutine을 public으로 구현하면 다른 스크립트에서도 실행 시킬 수 있습니다.
- **coroutine**에서도 스크립트 내부의 다른 일반 method를 호출할 수 있습니다.

<StartCoroutine의 2가지 매개변수 : 함수명(String), 함수명(Function Type)>

Declaration

```
public Coroutine StartCoroutine(string methodName, object value = null);
```

Description

Starts a coroutine named `methodName` .

In most cases you want to use the StartCoroutine variation above. However StartCoroutine using a string method name lets you use [StopCoroutine](#) with a specific method name. The downside is that the string version has a higher runtime overhead to start the coroutine and you can pass only one parameter.

```
public Coroutine StartCoroutine(IEnumerator routine);
```

Description

Starts a Coroutine.

<매개변수가 1개 존재하는 코루틴 호출>

```
IEnumerator ChangeBombColor(GameObject spawned_object)
{
    int max_change_cnt = 9;
    for(int i=0; i<max_change_cnt; i++)
    {
        if(spawned_object.GetComponent<Renderer>().material.color == Color.black)
        {
            spawned_object.GetComponent<Renderer>().material.color = Color.red;
        }
    }
}

StartCoroutine("UpperHeight",spawned_object);
```

<매개변수가 2개 이상 존재하는 코루틴 호출 : method 이름을 string으로 호출하는 것이 아닙니다.>

```
StartCoroutine(ChaseEnemy(ball , enemy_player.transform.position));
```

<yield Return : 코루틴을 호출한 루틴으로 개발자가 지정한 시간만큼 제어 권한을 돌려주기>

IEnumerator를 사용해야 하고, yield return 구문이 적어도 한번 이상은 사용되어야 합니다.

```
IEnumerator TestCo(){  
    //수행할 작업  
    yield return null;  
}
```

yield는 함수의 코드가 실행한 결과를 반환합니다. 코루틴이 함수에 정의된 작업을 실행하다가 yield return 구문을 만나면, yield return 구문으로 지정한 명령을 실행시키고, 코루틴을 호출했던 루틴으로 잠시 제어의 권한을 넘겨줍니다. 코루틴의 실행이 일시적으로 중지되는 것이구요. 위 코드에서 yield return 이 지정한 "null"은 1프레임을 코루틴의 호출자에게 양보하고 대기하라는 뜻입니다. 즉 다음 프레임까지 대기를 해야할 경우에 사용할 수 있지요. 만약 예를 들어 3초 동안 대기했다가 다음 작업을 실행하려면, yield return 구문을 아래와 같이 지정할 수 있습니다.

```
yield return WaitForSeconds(3.0f);
```

3초가 지나면 프로그램의 제어 권한은 코루틴 함수에서 yield return으로 중단되었던 코드의 위치로 복귀하여 코루틴 함수의 작업을 계속 이어갑니다. yield 구문으로 반환되는 값이 코루틴 함수가 다시 실행(재개)되는 시점을 지정하는 것입니다. 이것이 바로 서브 루틴과 코루틴의 결정적인 차이입니다. 서브 루틴은 호출될 때마다 코드의 첫 부분부터 다시 실행 하지만, 코루틴은 중단되었던 코드의 다음 위치부터 실행 됩니다. 함수가 시작되는 위치를 진입 지점(Entry Point)이라고 하는데요? 진입 지점이 1개인 서브 루틴에 비해서, 코루틴은 진입 지점을 여러개 정의할 수 있습니다. 아래는 코루틴에서 사용하는 반환 타입의 목록입니다. (코루틴이 코루틴의 호출자에게 양보하는 조건이라고 생각하시면 됩니다.)

<yield return null>

- 코루틴을 탈출하는 구문이 아니라 , 1 프레임 정도를 메인 스레드에 돌려주는 기능
- yield return은 시간을 제어하는 구문이며 , 해당 시간동안 지연을 시키는 동작입니다. 그러므로 return이 사용되기 때문에 코루틴이 해당 구문에서 종료되는 것이 아닌 , 코루틴 내부의 모든 동작이 종료되면 코루틴이 종료되고 , 다시 메인 루틴으로 제어권을 넘겨줍니다.

<좌측 예시의 오류>

- yield return new WaitForSeconds(3.0f); 로 적어야 실행됩니다.

<Coroutine 강제종료하기>

```
public class Example : MonoBehaviour {
    private IEnumerator coroutine;

    void Start()
    {
        coroutine = TestCoroutine();

        // 동작x
        StartCoroutine(TestCoroutine());
        StopCoroutine(TestCoroutine());

        // 동작o
        StartCoroutine(coroutine);
        StopCoroutine(coroutine);
    }

    IEnumerator TestCoroutine()
    {
        yield return new WaitForSeconds(2f);
        Debug.Log("테스트");
    }
}
```

- 중지해야 하는 코루틴을 객체로 선언하여 명확하게 참조합니다.

StartCoroutine(TestCoroutine());

StopCoroutine(coroutine);

처럼 혼용해서 사용하면 중지되는 코드가 실행이 되지만 중지되지 않고 계속 코루틴을 진행시킵니다.

그러므로 반드시 '동작o'의 코드로 객체를 명시하여 실행시킵시다.

```
public class Example : MonoBehaviour {
    private IEnumerator coroutine;

    void Start()
    {
        coroutine = TestCoroutine();

        // 동작x
        StartCoroutine(TestCoroutine());
        StopCoroutine(TestCoroutine());

        // 동작o
        StartCoroutine(coroutine);
        StopCoroutine(coroutine);
    }

    IEnumerator TestCoroutine()
    {
        yield return new WaitForSeconds(2f);
        Debug.Log("테스트");
    }
}
```

<Coroutine VS Invoke>

- 복잡하고 다양하게 시간(지연)을 관리할 때는 Coroutine을 사용하고 , 간단한 단발성 지연일 때는 Invoke를 사용합니다.

코루틴(Coroutine)

- 여러 개의 루틴이 동시에 실행되며 서로 제어를 넘겨주는 방식.
- yield return을 사용하여 현재 위치를 기억하고 다른 루틴에게 수행 권한을 넘겨서 여러개의 쓰레드가 동시 동작하는 것과 같은 효과를 제공한다.
- 실제로는 단일 쓰레드 이기 때문에 멀티쓰레드가 가지는 교착 상태 경합 등의 문제에서 자유롭다는 장점이있다.
- 특정 조건이 충족될때까지 실행 상태를 지연시킬 수 있다.
- GameObject의 Active가 off가 된다면 이미 실행 중이었던 코루틴은 정지되며, StartCoroutine은 오류가 발생한다.
- 정지된 코루틴은 GameObject가 다시 Active가 되더라도 재 실행되지 않는다.

Invoke

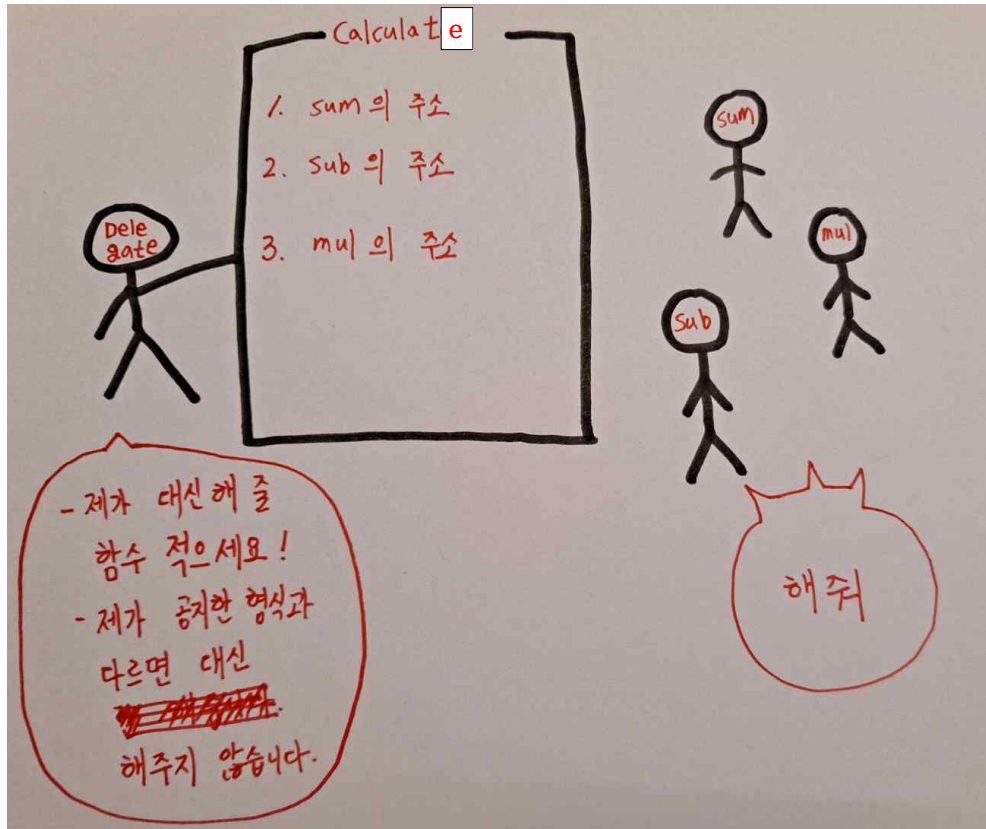
- 매개변수로 넘겨주는 시간 만큼 함수를 지연호출할 수 있다.
- GameObject의 Active가 off 상태에서도 호출된다.
- InvokeRepeat를 사용할 때 지연시간을 변수로 두고 변경하더라도 최초로 넘겼던 파라미터의 값으로 적용된다.
- 지연시간 이외의 파라미터를 전달할 수 없다.

Invoke is useful if you want to do something one time only.

구분	Coroutine	Invoke
GameObject Off 상태 실행여부	X	O
파라미터 전달	O	X

- 포탄에 코루틴을 달았는데 , 코루틴이 종료되기 전에 포탄이 사라지면 코루틴이 종료됩니다.

6. Delegate (=대리인 =위임자)



- Delegate는 자신이 대신해 줄 함수들의 형식을 지정합니다.
- Delegate는 Object를 만듭니다. 해당 Object는 대신해 줄 함수를 적을 종이라고 생각하면 이해가 빠릅니다.
- 대신해 줄 함수를 적게 되면, 실제로는 해당 함수들의 주소가 Delegate의 object에 저장되며, 해당 함수를 찾아가서(참조) 실행시킵니다.
- Delegate는 object에 적힌 순서대로 실행하도록 합니다.
- Delegate는 object에 적힌 마지막 함수의 리턴 값을 리턴 합니다.

1) Delegate 형식 만들기

```
public class Calculator : MonoBehaviour {  
  
    delegate float Calculate(float a, float b);  
  
}
```

- Return 형이 float이고 , 매개변수 2개가 float인 함수만 Calculate delegate가 대신하겠다는 형식을 지정하며 Delegate를 선언합니다.

2) Delegate Object 만들기

```
Calculate onCalculate;
```

3) Delegate object에 적을 형식에 맞는 함수 만들기

```
public float Sum(float a, float b)  
{  
    Debug.Log(a + b);  
    return a + b;  
}  
  
public float Subtract(float a , float b)  
{  
    Debug.Log(a - b);  
    return a - b;  
}  
  
public float Multiply(float a, float b)  
{  
    Debug.Log(a * b);  
    return a * b;  
}
```

- 형식은 '매개변수 + 리턴 형'을 의미합니다.
- Delegate는 형식이 다른 함수는 대신 해주지 않습니다.

4) Delegate Object에 최초로 함수 1개 추가하기

① 일반적인 추가 (참조형)

- 함수의 포인터를 받아 와서 delegate object에 추가합니다.

```
onCalculate = Sum;
```

② 잘못된 추가

- delegate object에 추가하지 않고 , 함수를 직접 실행시켜버리므로 delegate의 의미가 퇴색됩니다.

```
onCalculate = Sum();
```

5) Delegate Object에 함수 추가

```
onCalculate += Subtract;
```

6) Delegate Object에 함수 제거

```
onCalculate -= Subtract;
```

7) 실행

```
void Update()
{
    if(Input.GetKeyDown(KeyCode.Space))
    {
        onCalculate(1,10);
    }
}
```

- Delegate object에 존재하는 함수에 사용자가 입력한 인자를 넣어 **순서대로 실행** 시킵니다.

7. 인터페이스

1) 정의

= 설계도

- Script = Class를 확장시켜 Script = (Class OR Interface)로 정의할 수 있습니다.
- Interface를 구현하는 Script의 이름 앞에는 'I'를 붙이기로 약속했습니다.
- method 이름만 구현하고 , 아무 기능을 구현하지 않기 때문에 반드시 다른 Script(=Class)에 상속 받아서 사용해야 합니다.
- 어떤 Script(=Class)가 Interface를 상속 받았다면 해당 Script는 Interface에 존재하는 모든 Method를 구현해야 합니다. 이 과정이 Override이지만 C#에서는 Override를 붙이지 않아도 됩니다.

```
public class GoldItem : MonoBehaviour, IItem {
```

- GoldItem(=Class)은 MonoBehaviour(=Class)와 IItem(=Interface)을 상속받습니다.
- GoldItem은 IItem에 존재하는 모든 Method를 구현해야 합니다.

2) 인터페이스를 사용해야 하는 이유 (게임을 예시로 설명)

- 이 게임에는 아이템이 1,000 종류가 있고 플레이어와 닿으면 해당 아이템이 자동으로 사용됩니다.
- 아이템이 자동으로 사용되는 기능은 Use 라는 Method로 구현합니다.

① 기존의 허접한 방식 : If문 내부에 1,000개의 분기 생성

= Player Script에 OnTriggerEnter()함수를 만들고 , 그 안에 1,000개의 if문을 이용하여 조건에 맞는 아이템을 작동시킬 것 입니다. 1,000개의 아이템마다의 Script에는 당연히 Use 함수를 구현해야 합니다.

② Interface를 사용하는 훌륭한 방식 : If문 내부에 단 1개의 분기 생성

= Interface의 다형성과 Interface를 상속 받은 Class는 반드시 Interface의 Method를 구현해야 하는 방식으로 인해 분기 없이 코드를 작성할 수 있습니다. 1,000개의 아이템마다의 Script에는 당연히 Use 함수를 구현해야 합니다.

<허접한 기존의 방식>

```
void OnTriggerEnter2D(Collider2D other)
{
    GoldItem goldItem = other.GetComponent<GoldItem>();

    if(goldItem != null)
    {
        goldItem.Use();
    }

    HealthKitItem healthKitItem = other.GetComponent<HealthKitItem>();

    if(healthKitItem != null)
    {
        healthKitItem.Use();
    }
}
```

<Interface를 사용한 훌륭한 방식>

```
void OnTriggerEnter2D(Collider2D other)
{
    IItem item = other.GetComponent<IItem>();

    if(item != null)
    {
        item.Use();
    }
}
```

- Reference : <https://www.udemy.com/course/retr0-unity/learn/lecture/8609448#overview>

<Unity 고급 기능> 20/08/21

1. Unity Memory

1) Native 영역

= Texture , Mesh , Asset Data 같은 하부시스템 , GameObject와 Component 같은 기본 클래스의 위치 데이터와 물리 객체 요소
<Native Code>

= 해당 플랫폼에 맞게 곧바로 컴파일 되어 실행 과정에서 번역이 필요하지 않은 프로그래밍 언어/코드

2) Managed 영역

① Stack

- = 용량이 작기 때문에(MB단위) 일반적으로 작고 금방 사라질 데이터가 스택 형태로 쌓여서 저장됩니다.
- 특별한 작업을 하지 않은 **일반 변수**가 생성되면 스택구조로 스택 메모리에 스택 구조(LIFO)로 쌓입니다.
- method가 호출 된다면 **매개변수** , **return address** , **static link** , **dynamic link** 모두가 스택에 쌓입니다.
- 재귀함수를 이용하다 보면 스택에 쌓이는 데이터들이 기하급수 적으로 쌓이는 경우가 있는데 이를 Stack Overflow라고 합니다.

② Heap (=Managed Heap)

- Stack을 제외한 Managed 영역을 차지합니다.
- 배열은 C와 C++에서 스택과 힙 모두에서 생성이 가능했으나 C#은 무조건 힙에서만 생성 됩니다.
- 최초의 Heap은 1MB로 매우 작지만 , Script에 의해 메모리가 호출될 때마다 크기가 조금씩 커집니다.
- Profiler에서 'Mono'라고 표시됩니다.
- **Garbage Collector에 의해서 관리** 됩니다.
- **전역 변수** , **Static 변수** , **모든 배열** , **클래스** , **객체** , **String(=문자열)**, 자료구조(ex : List)가 **힙 메모리**에 생성됩니다.

3) 외부 DLL을 위한 Native Memory 영역

- 어려운 내용
- Profiler에서 'Unity'로 표기된 영역으로 나타납니다.

2. Garbage Collection (=GC)

1) 파일처리 과목에서의 Garbage Collection

- 파일은 추가하고 수정하고 지우기를 반복합니다. 그러다 보면 저장 공간에서 순차적으로 저장 되지 않고 빈 공간을 발생 시킬 것 입니다. 이를 Garbage라고 하며 Garbage에 새로운 파일을 저장하는 것이 Garbage Collection입니다.
- Garbage의 공간과 추가할 파일의 크기를 비교하여 Garbage에 파일을 추가합니다.

2) Unity에서 Garbage Collection

- 메모리 관리자는 항상 힙에서 사용되지 않고 있는 블록 영역을 추적합니다. 힙에서 새로운 메모리 블록이 요구될 때 메모리 관리자는 블록이 할당되지 않은 공간을 선택하고 사용하지 않는 것으로 인식하고 있는 메모리를 제거합니다. 요청된 블록의 크기를 할당 할 때까지 동일한 방식으로 처리됩니다. 이 시점에서 힙에 할당된 모든 메모리가 사용되는 경우는 극히 드뭅니다.
- 메모리 관리자가 사용하지 않는 메모리를 처리하고 해제하는 것을 가비지컬렉션, 줄여서 GC 라고 부릅니다.
- C와 C++에서는 프로그래머가 동적 할당으로 힙 영역에 데이터를 생성하면 반드시 수동적으로 free()를 통해 메모리를 해제하여 메모리 낭비를 줄여야 합니다. 하지만 C#은 메모리 관리자가 자동으로 특정 규칙에 의해 해제해도 될 것 같은 메모리를 해제 하여 줍니다.
- 프로그래머에게 보이지 않지만 수집 과정은 실제로는 상당한 CPU 시간을 필요로 합니다. Garbage Collection이 많아지면 게임 자체가 종료되거나 멈추는 현상이 발생합니다. 그러므로 프로그래머는 과학계 자동으로 실행 되는 Garbage Collection을 막아야합니다.

3) 과도한 Garbage Collection 해결 방법

① Garbage Collection 끄기

- 가비지 컬렉션을 끄면 메모리 사용량의 감소(이로운 기능)는 절대 일어나지 않습니다. 그러므로 메모리의 사용량은 일정하거나 계속 증가하게 됩니다. 따라서 이상적으로 이용하기 위해서는 가비지 컬렉션을 비활성 상태로 만들면 메모리의 추가할당을 피해야 합니다.

② 작은 메모리를 빈번하게 할당하고 해제하는 방법

- 자주 발생하는 환경에 적합합니다.
- 작은 메모리를 할당하고 해제할 때 걸리는 시간이 확연히 줄기에 이 방법을 이용합니다.

- 실전에서 테스트할 때는 반드시 프로파일러를 이용하여 비교해 봐야합니다.

③ 큰 메모리를 가끔 할당하고 해제하는 방법

- 가끔 발생하는 환경에 적합합니다.
- 이 전략은 메모리 할당과 가비지 콜렉션이 비교적 자주 발생하지 않아 게임플레이 중간에 처리할 수 있는 게임에 적합합니다. 힙 용량은 최대한 큰 것이 좋습니다. 다만 너무 커서 운영체제가 시스템 메모리를 확보하려고 앱을 강제종료하는 경우는 피해야 합니다. Mono 런타임은 자동으로 힙을 확장하는 것을 최대한 피합니다. 따라서 수동으로 힙을 확장해야 하는데, 시작할 때 일부 플레이스홀더를 미리 할당하면 됩니다. 예를 들어, 메모리 관리자에서 메모리를 할당받기 위해 “무의미한” 오브젝트를 인스턴스 처리하면 됩니다.

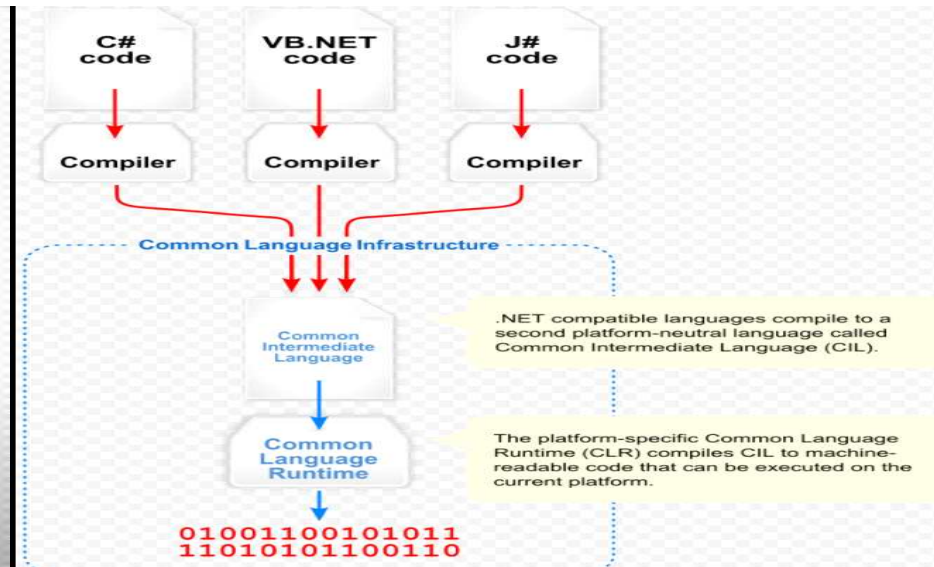
④ Object Pooling

- = GameObject의 생성과 삭제의 경우 많은 부하가 발생하고 Garbage Collection의 호출로 인해 게임 내에서 렉이 발생합니다. 그러므로 게임이 시작되는 순간 필요한 만큼의 GameObject들을 Object Pool에 미리 만들어 놓습니다. 필요할 때 꺼내서 사용하고 필요가 없어지면 다시 Object Pool로 GameObject를 이동시킵니다. 게임이 종료되면 Object Pool 전체를 삭제시킵니다.
- 현대에는 메모리가 풍족하지만 게임의 최적화를 위해서 Object Pooling 기법을 사용합니다.
- 큰 메모리를 가끔 할당하고 해제하는 방법으로 봐도 무방하다고 생각합니다.

공식 문서 : <https://docs.unity3d.com/kr/2019.4/Manual/UnderstandingAutomaticMemoryManagement.html>

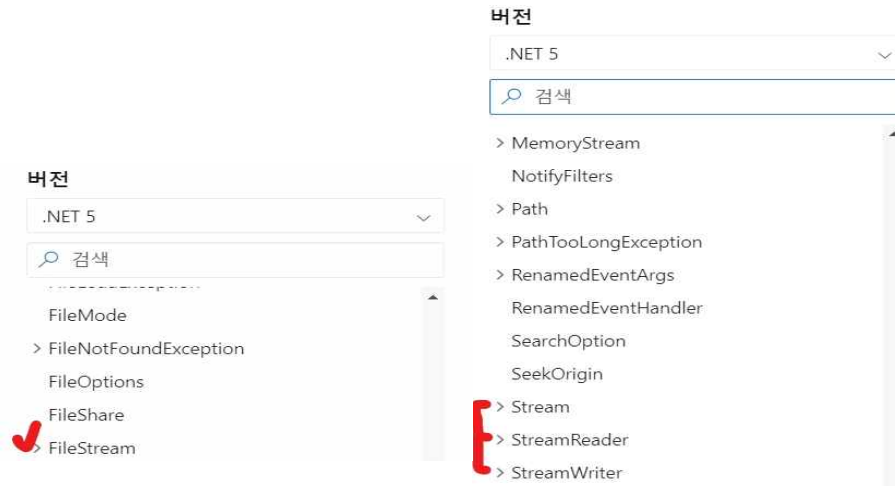
3. Net Framework(닷넷 환경)

- = Unity에서의 닷넷 환경 : Unity는 C#언어를 사용하므로 Unity를 개발하기 위한 토대입니다.
- = 자바를 사용할 때 JVM이 필요하듯이 C#은 닷넷 프레임워크가 필요합니다.
- = 닷넷 환경에서의 프로그램들을 개발하기 위한 기반이자 가상머신 입니다. 즉 해당 프로그램들을 개발하거나 사용하기 위해서는 닷넷 환경을 반드시 다운로드 하여야 합니다.
- = 메모리를 자동으로 관리하는 언어(가상머신 기반인 자바와 C#)의 특성상 수동으로 관리하는 C와 C++보다 느립니다.
- = 윈도우에서만 닷넷 환경을 이용할 수 있었지만 모노 또는 .NET CORE로 인해 MacOS와 리눅스에서도 사용할 수 있게 되었습니다.
- = winForms = 윈도우 기술 , ASP.NET = 웹 기술 , ADO.NET = 데이터베이스 기술 (상위 내용들도 모두 비슷합니다.)
- = 닷넷 환경의 대표적인 고급언어인 C#(닷넷의 근본), C++, VB.NET code, J# Code입니다. 이 들을 컴파일 시키면 기계어로 바로 번역 되는 것이 아니라 닷넷에서 사용할 수 있는 중간언어common intermediate language(CIL)로 번역 되고 common language runtime(CLR)을 거친 후에 최종적으로 기계어가 됩니다.
- = .Net Framework 에는 유용한 API가 존재합니다.



4. 데이터베이스 (2021/07/16 최신화)

1) 라이브러리



```
using System.IO;
```

FileStream 클래스는 Stream 클래스에서 파생됩니다.

2) 파일의 경로

Assets

Assets 폴더는 Unity 프로젝트에서 사용하는 에셋이 포함되는 주 폴더입니다. 에디터 프로젝트 창의 콘텐츠는 에셋 폴더의 콘텐츠와 직접 연관됩니다. 대부분의 API 함수는 모든 것이 에셋 폴더에 있다고 가정하므로 명시적으로 경로를 알려줄 필요는 없습니다. 그러나 일부 함수는 경로 이름에 에셋 폴더를 반드시 포함해야 합니다(예를 들어 AssetDatabase 클래스의 특정 함수).

```
private const string PATH = "Assets/Resources/RecordGamesDatabase.txt";
```


3) 파일 접근하기

```
StreamReader reader = new StreamReader(PATH, true);
```

- 읽을 때는 StreamReader로 파일에 접근합니다.
- 쓸 때는 StreamWriter로 파일에 접근합니다.

```
StreamWriter writer = new StreamWriter(PATH, true);
```

<ERROR : 중복으로 경로에 접근할 때>

 [16:27:07] IOException: Sharing violation on path C:\Users\pjw96\OneDrive\바탕 화면\Othello\Assets\Resources\RecordGamesDatabase.txt
System.IO.FileStream..ctor (System.String path, System.IO.FileMode mode, System.IO.FileAccess access, System.IO.FileShare share, System.Int32 bufferSize, System.Boolean

```
StreamReader reader = new StreamReader(PATH, true);  
StreamWriter writer = new StreamWriter(PATH, true);
```

- 동시에 같은 PATH에 접근하므로 IO ERROR가 발생합니다.

```
StreamReader reader = new StreamReader(PATH, true);  
reader.Close();  
StreamWriter writer = new StreamWriter(PATH, true);
```

- 동일한 PATH에 접근할 때는 이전의 Stream을 닫아줍니다.

4) 파일 읽기_1 : 디스크에서 파일 읽어오기

① Unity에서 제공하는 **TextAsset**

TextAsset

class in UnityEngine / Inherits from: [Object](#)

Description

텍스트 파일 에셋을 나타냅니다.

You can use raw text files in your project as assets and get their contents through this class. Also, you can access the file as a raw byte array if you want to access data from binary files. See the [Text Asset](#) manual page for further details about importing text and binary files into your project as Text Assets.

Variables

[bytes](#)

텍스트 에셋의 raw 바이트를 나타냅니다. (읽기전용)

[text](#)

.txt 파일의 텍스트 콘텐츠(문자열)를 나타냅니다. (읽기전용)

```
public TextAsset item_db; //disk
```

: 이 변수에 폴더에 있는 text data를 연결만하면 잘 읽음 , 이게 전부

② StreamReader

- 위의 방식이 너무 편하기 때문에 공부하지 않습니다. (아래는 예전자료)

```
/*현재 읽으려는 파일에는
 * a
 * b
 * c
 * 로 저장되어 있다고 가정합니다.
 */
/*
 * sr.ReadLine()은 어떤 상황에서도 적는다면 한 줄을 읽습니다.
 * 1번 코드의 조건문은 str = sr.ReadLine()으로 초기화를 시키고
 * 이것이 null인지 검사합니다 그러므로 올바르게 a , b , c가 나옵니다.
 * 2번 코드는 조건문에서도 sr.ReadLine()을 하지만 Debug에서도 sr.ReadLine()을 하기 때문에 a일때 검사하고 b를 출력하고
 * c일때 검사하고 null을 출력하게 됩니다.
 * 1번 형식을 Default 형식으로 사용합시다.
 */
//1번
while((str = sr.ReadLine()) != null)
{
    Debug.Log(str);
}
//2번
/*while(sr.ReadLine() != null)
{
    Debug.Log(sr.ReadLine());
}*/
```

- 주석을 정독합시다.
- ReadLine()method는 더 읽을 데이터가 있다면 해당 문자열을 String으로 반환하고 없다면 null을 반환합니다.
- Read()method는 한글자씩 읽어오는데 반환형이 Int이므로 매우 불편합니다. 하지만 깊게 생각하면 Read()method는 사용할 필요가 없습니다.
- 파일을 읽을 때나 쓸 때 최대한 파일에 접근을 줄여야 disk i/o가 적어집니다. 그러므로 **한 글자 씩 읽고 쓰는 방식은 disk i/o를 급증시키므로 절대 사용하면 안됩니다. 한 줄 또는 한 블록 씩 읽고 써서 disk i/o를 감소시킵시다.**

5) 파일 읽기_2 : 읽어 온 파일을 메모리(자료구조)에 저장하기

```
private List<records> item_list; //memory
```

```
public class records
{
    public string p1_name;
    public string p1_score; //int형 데이터도 string으로 이용
    public string p2_name;
    public string p2_score;

    public records(string a , string b , string c , string d)
    {
        this.p1_name = a;
        this.p1_score = b;
        this.p2_name = c;
        this.p2_score = d;
    }
};
```

```
void ReadCurrentData()
{
    string[] line = item_db.text.Substring(0, item_db.text.Length - 1).Split('\n');

    for(int i=0; i<line.Length; i++)
    {
        string[] row = line[i].Split('\t');
        records tmp = new records(row[0] , row[1] , row[2] , row[3]);
        item_list.Add(tmp);
    }
}
```

- 디스크에서 읽어온 item_db.text를 개행 마다 자릅니다.
- 방금 자른 string을 다시 tab 마다 자릅니다.
- 그것들을 미리 만들어 놓은 records(class)형 자료구조에 저장합니다.

6) 파일에 쓰기

- WriteStream(Class)의 WriteLine(Method)을 사용합니다.

<code>WriteLine(String, Object, Object, Object)</code>	<code>Format(String, Object)</code> 와 동일한 의미 체계를 사용하여 서식이 지정된 문자열과 새 줄을 스트림에 씁니다.
<code>WriteLine(String, Object, Object)</code>	<code>Format(String, Object, Object)</code> 메서드와 동일한 의미 체계를 사용하여 서식이 지정된 문자열과 새 줄을 스트림에 씁니다.
<code>WriteLine(String, Object[])</code>	<code>Format(String, Object)</code> 와 동일한 의미 체계를 사용하여 서식이 지정된 문자열과 새 줄을 스트림에 씁니다.
<code>WriteLine(String)</code>	문자열과 줄 종결자를 차례로 스트림에 씁니다.
<code>WriteLine(ReadOnlySpan<Char>)</code>	스트림에 줄 종결자가 다음에 오도록 문자 범위의 텍스트 표현을 씁니다.
<code>WriteLine(String, Object)</code>	<code>Format(String, Object)</code> 메서드와 동일한 의미 체계를 사용하여 서식이 지정된 문자열과 새 줄을 스트림에 씁니다.

- Reference : <https://88-it.tistory.com/181>

7) 파일 접근자 위치

C#

```
public override long Seek (long offset, System.IO.SeekOrigin origin);
```

Parameters

offset `Int64`

검색을 시작할 `origin`에 상대적인 위치입니다.

origin `SeekOrigin`

`SeekOrigin` 형식의 값을 사용하여 시작, 끝 또는 현재 위치를 `offset`에 대한 참조 지점으로 지정합니다.

Begin	0	스트림의 시작 부분을 지정합니다.
Current	1	스트림 내의 현재 위치를 지정합니다.
End	2	스트림의 끝을 지정합니다.

`offset`이 `byte_offset`이고 , 개행에 대해서는 관대하지 않습니다.

```
fs_read.Seek(6, SeekOrigin.Begin);
```

 = 스트림의 시작부분에서 6바이트 떨어진 곳에 파일 포인터를 이동시킵니다.

5. 데이터베이스 (2021/07/15)

1) Excel

- Excel은 메모장에 저장할 데이터를 만들 틀 일 뿐.

3) 읽은 데이터를 메모리(자료구조)에 저장

1) .Split()

```
string[] row = line[i].Split('\t');
```

- Split을 통해 거대한 string 문치를 엔터나 탭을 기준으로 분해해서 작은 string들로 분해합니다.
- 각각의 작은 string은 알아서 String형 배열에 저장됩니다.
- 메모리에 저장하기 위한 자료구조 필요 : 강의에서는 class를 사용하지만 나는 struct , 하지만 c#은 struct와 class가 90%이상 동일

<ERROR>



[23:40:11] NullReferenceException: Object reference not set to an instance of an object
DataManager.ReadCurrentData () (at Assets/Scripts/DataManager.cs:43)

```
private List<records> item_list; //memory item_list = new List<records>();
```

- 객체를 만들고 , instance를 생성하지 않으면 위의 예러가 발생합니다.
- 가장 흔한 ERROR이므로 , 평소에도 객체가 instance를 잘 참조하고 있는지 확인해야 합니다.

<최적화> 21/06/01

<Reference>

- <https://learn.unity.com/tutorial/fixing-performance-problems#5c7f8528edbc2a002053b594>
- <https://rito15.github.io/posts/unity-opt-script-optimization>
- <https://www.youtube.com/watch?v=tT9LS53HRx4>

1. 잦은 호출과 반복문

- 많은 GameObject에 component로 존재하거나, Update문 같이 자주 호출되는 함수에서의 반복문 사용을 가능한 줄여야합니다.

```
void Update()
{
    for (int i = 0; i < myArray.Length; i++)
    {
        if (exampleBool)
        {
            ExampleFunction(myArray[i]);
        }
    }
}
```

With a simple change, the code iterates through the loop only if the condition is met.

```
void Update()
{
    if (exampleBool)
    {
        for (int i = 0; i < myArray.Length; i++)
        {
            ExampleFunction(myArray[i]);
        }
    }
}
```


2. 이벤트 주도적 프로그래밍

1) 이벤트

= Unity내에서 일어나는 어떤 행동은 다른 행동을 수반(더불어 생김)하는 식으로 서로의 행동들은 중요한 관계를 맺고 있고 , 이러한 연결이나 관계를 이벤트라고 부릅니다.

ex) 플레이어의 HP가 100 -> 0이 되면 , 플레이어는 죽습니다.

- 이벤트는 정해진 시간에 발생할 수도 있지만 , 특정 시간에 발생할 수도 있습니다. 그 결과 , 해당 이벤트(원인 이벤트)가 발생하자마자 즉각적으로 응답시켜야 할 이벤트(응답 이벤트)를 제대로 구현해야 합니다.

2) 이벤트 주도적 프로그래밍 정의 (유니티 C# 스크립팅 마스터하기 P163 ~ P166의 코드를 읽고 이해한 내용)

- 주도적이라는 뜻이 가지는 의미가 중요합니다. 이벤트에 focus를 두어 update문의 부하를 감소시킨다는 의미입니다.

- 게임에서 플레이어의 체력 , 총알 개수 같은 멤버 변수의 값의 변화를 추적할 때 update(가장 적합해 보이지만 그렇지 않음)를 사용하면 멤버 변수가 많아 질 때 cpu 성능 저하가 발생합니다. 이는 알고리즘에서의 brute force처럼 언제나 모든 경우를 추적하는 것과 유사하다고 생각이 됩니다. 하지만 이벤트에 focus를 두어 주도적으로 생각해보면 , 우리는 플레이어의 체력이 변화하는 시점에만 플레이어의 체력을 검사하면 되고 , 총알이 발사되거나 장전될 때만 총알의 개수만 검사하면 됩니다. 이렇게 이벤트 주도적으로 프로그래밍을 하게 되면 update문에 들어갈 method가 감소하여 성능 향상을 얻을 수 있게 됩니다.

- 이전에는 너무 어렵게 생각했고 , '주도적'의 의미를 해석하지 못하여 이해를 하지 못했습니다. 결론적으로 이벤트 주도적 프로그래밍은 update에서 매 프레임 마다 값의 변화를 추적하지 않고 , 검사해야 할 특정 상황(원인 이벤트)에서만 검사를 하는 방식입니다. 그러므로 프로그래머는 검사해야 할 특정 상황을 줄여서 응답 이벤트의 호출 횟수를 줄여서 성능의 향상을 도모해야 합니다.

3) Unity 자체 이벤트 함수

- Unity 내부에서 특정 상황(원인 이벤트)을 규정해 놓고 , 해당 상황이 발생했을 때 자동으로 호출되는 응답 이벤트를 method로 구현해놓았습니다.

- Update(), Awake()처럼 On이 붙지 않은 method와 OnTrigger~ , OnMouse처럼 On이 붙은 method가 존재합니다.

3. Caching (캐싱) : GetComponent

= CPU가 해당 작업을 처리하는데 비용이 많이 드는 작업을 자주할 때 , 개발자가 변수를 만들어 저장하고 , 해당 변수를 재사용하는 방법을 캐싱이라고 합니다.

- 변수에 저장하여 메모리에 로드시키고 이를 계속 유지시킵니다.
- 메모리를 추가로 소모하는 대신 CPU 부하를 감소시킵니다.

<바로 이해되는 Example>

```
void Update()
{
    Renderer myRenderer = GetComponent<Renderer>();
    ExampleFunction(myRenderer);
}
```

The following code calls *GetComponent()* only once, as the result of the function is cached. The cached result can be reused in *Update()* without any further calls to *GetComponent()*.

```
private Renderer myRenderer;

void Start()
{
    myRenderer = GetComponent<Renderer>();
}

void Update()
{
    ExampleFunction(myRenderer);
}
```

- GetComponent는 비용이 많이 들기 때문에 Update()에서 사용 할 때 적지 않은 오버헤드를 발생시킵니다.

- 그러므로 , Start()에서 'myRenderer' 변수에 caching을 하고 사용합니다.

- GetComponent는 GetComponent("문자열") 과 GetComponent<Type>()으로 오버로딩 되어 있는데 GetComponent<Type>()가 훨씬(7배~30배) 빠릅니다.

<Caching에서 자주 고민하는 2가지 ISSUE>

- Caching은 매우 간단한 최적화 기법임에도 불구하고 , 가끔 머뭇거리게 하는 2개의 Issue가 존재합니다.

1) 어느 정도의 호출 빈도를 갖는 객체에 대해 캐싱을 해야 할까요?

캐싱에 관해서 구글링을 하고 , 글을 읽어 보았지만 스스로 해결되지 않는 문제에 대해서 질문드립니다.

자주 호출되는 (ex : update) 함수에서는 GetComponent<>가 성능저하를 발생하므로 사용하지 않고

그와 버금가는 호출에서도 사용하지 않고 있습니다.

하지만 , 호출의 빈도가 정말 적은(10초에 1번 정도) 함수에서 굳이 캐싱을 해야 할까요?

코드가 길어지고 , 시간도 조금이지만 더 사용하게 됩니다.

선생님들의 경험에 대한 조언을 주시면 감사하겠습니다.



leumas

매프레임 하지말라고 할 뿐 필요한 곳에는 사용하시는게 개발속도와 편의에 도움이 됩니다.
개발에서의 모든 선택은 Trade off로 이득과 손해를 다른 곳 에서 절충하는 방식입니다.
매프레임 돌리지 말라는건 얻는 것에 비해 잃는게 너무 크기 때문일뿐입니다.

2021.08.20. 16:01 답글쓰기



컴맹

그정도라면 굳이 캐싱안해도 되죠.

2021.08.20. 16:03 답글쓰기



Jintthree

찝찝하고 궁금한 것이 있으실 땐 한번 간단하게 테스트를 해보시는 것도 좋아요.

GetComponent도 1초에 수천번 이상 호출하는게 아닌 이상 영향은 거의 없습니다. 어지간한 모바일 C
PU에서도 1000번 실행에 1ms도 안걸릴거예요.

2) Unity Editor에 존재하는 GameObject의 Component가 변경될 때도 있고 아닐 때도 존재합니다.

<Example : Player 객체의 Material에 존재하는 투명도를 알파 값을 통해 변경하길 원합니다.>

① 변경 X :

```
public Color player_body_color { get; private set; }
```

```
player_body_color = gameObject.GetComponent<Renderer>().material.color;
```

```
player_body_color = PLAYER_BODY_COLOR_DEFAULT - new Color(0, 0, 0, 0.8f);
```

② 변경 O

```
public Material player_body_color { get; private set; }
```

```
player_body_color = gameObject.GetComponent<Renderer>().material;
```

```
player_body_color.color = PLAYER_BODY_COLOR_DEFAULT - new Color(0, 0, 0, 0.8f);
```

- 위의 원인을 명확히 모르겠습니다. 뭔가 내부의 클래스와 , 속성 차이 인 것 같음.

- 매개변수로 받아온 인자에 대해서 캐싱을 하면 , 변화하지 않습니다. 하지만 transform은 이경우에도 되고 , transform.position은 안됨.

4. 자료구조

Choose a collection

I want to...	Generic collection options
Store items as key/value pairs for quick look-up by key	Dictionary<TKey,TValue>
Access items by index	List<T>
Use items first-in-first-out (FIFO)	Queue<T>
Use data Last-In-First-Out (LIFO)	Stack<T>
Access items sequentially	LinkedList<T>
Receive notifications when items are removed or added to the collection. (implements INotifyPropertyChanged and INotifyCollectionChanged)	ObservableCollection<T>
A sorted collection	SortedList<TKey,TValue>
A set for mathematical functions	HashSet<T> SortedSet<T>

- 각각의 상황에 가장 유용한 자료구조를 사용하면 게임 성능에 큰 향상을 이룰 수 있습니다.

5. Frustum Culling (프러스텀 컬링)

= 카메라에서 보이지 않는 영역의 코드는 실행시킬 필요가 없습니다. 그러므로 , 보이지 않는 영역에서 사용되는 코드를 애당초 실행 시키지 않아 CPU의 오버헤드를 감소시키는 방식입니다.

- Unity는 기본적으로 Frustum Culling을 자동으로 실행하고 , Occulusion Culling도 체크박스에서 체크하면 실행시킬 수 있습니다.
- 명언 : "the fastest code is the code that does not run".

```
void Update()
{
    UpdateTransformPosition();
    UpdateAnimations();
}
```

In the following code, we now check whether the enemy's renderer is within the frustum of any camera. The code related to the enemy's visual state runs only if the enemy is visible.

```
private Renderer myRenderer;

void Start()
{
    myRenderer = GetComponent<Renderer>();
}

void Update()
{
    UpdateTransformPosition();

    if (myRenderer.isVisible)
    {
        UpateAnimations();
    }
}
```

- 물체가 보이면 'UpdateAnimation'을 실행하고 , 보이지 않으면 'UpdateAnimation'을 실행하지 않습니다.

6. 비용이 큰 Unity API 사용을 회피해야 합니다.

= 절대 사용하지 말라는 식으로 이해하지 말고 , 자주 호출되는(ex : Update) 구문에서의 사용을 피하라는 이야기입니다.

It's important to understand that there is no list of Unity API calls that we should avoid. Every API call can be useful in some situations and less useful in others. In all cases, we must profile our game carefully, identify the cause of costly code and think carefully about how to resolve the problem in a way that's best for our game.

1) SendMessage() & BroadcastMessage()

- Reflection(런타임에서 처리함)을 사용해서 CPU를 많이 이용합니다.
- 극도로 비용이 큼니다. (일반 함수의 2,000배)
- Prototype 과정에서만 사용합시다.
- SendMessage() 함수 대신에 GetComponent<> 함수를 사용합시다.
- 두 개념 대신에 Events OR Delegates를 적극적으로 사용합시다.

2) Find() & Find 유형의 다른 Method

- 강력한 기능이지만 비용이 큼니다.
- 메모리에 로드된 모든 GameObject와 Component를 순회하여 찾습니다. 그 결과 해당 프로젝트 전체의 영역을 검사하게 됩니다.
- 자주 사용하지 않기를 권장하며 , 사용하더라도 반드시 caching을 해야 합니다.
- Inspector Panel
- Manage Reference(=caching) 전문 스크립트 만들기

Some simple techniques that may help us to reduce the use of Find() in our code include setting references to objects using the Inspector panel where possible, or creating scripts that manage references to things that are commonly searched for.

3) Transform

- GameObject의 transform을 참조할 때 , 반드시 해당 GameObject의 자식의 transform까지 모조리 참조하므로 비용이 많이 듭니다.
- 자주 사용하는 **Transform.position**이라면 반드시 **vector**에 캐싱 해서 사용합니다.
- transform.localPosition이 월드 좌표를 기준으로 하는 transform.position보다 부하가 적은 것은 사실이지만 , 월드 공간을 로컬 공간으로 바꾸는 연산은 말도 안 되게 복잡합니다. 역시 캐싱이 정답입니다.
- 아래의 내용이 transform.position을 캐싱하는 건지 transform을 캐싱하는 건지 공부

Transform

Setting the position or rotation of a transform causes an internal *OnTransformChanged* event to propagate to all of that transform's children. This means that it's relatively expensive to set a transform's position and rotation values, especially in transforms that have many children.

To limit the number of these internal events, we should avoid setting the value of these properties more often than necessary. For example, we might perform one calculation to set a transform's x position and then another to set its z position in *Update()*. In this example, we should consider copying the transform's position to a *Vector3*, performing the required calculations on that *Vector3* and then setting the transform's position to the value of that *Vector3*. This would result in only one *OnTransformChanged* event.

Transform.position is an example of an accessor that results in a calculation behind the scenes. This can be contrasted with *Transform.localPosition*. The value of *localPosition* is stored in the transform and calling *Transform.localPosition* simply returns this value. However, the transform's world position is calculated every time we call *Transform.position*.

If our code makes frequent use of *Transform.position* and we can use *Transform.localPosition* in its place, this will result in fewer CPU instructions and may ultimately benefit performance. If we make frequent use *Transform.position*, we should cache it where possible.

4) Update() & LateUpdate()

- 겉보기에는 간단해보이지만 engine code 와 manage code 사이에서 많은 대화를 시키기 때문에 오버헤드가 증폭합니다.
- 호출 전에 Safety Check(=GameObject가 유효한지 , 사라졌는지 검사)를 하는데 , 이게 큰 오버헤드를 발생시키지는 않지만 Update 같이 수천 번 호출되는 method에서는 성능 감소의 원인이 됩니다.
- Update 함수의 body가 비어있어도 Safety Check는 일어나기 때문에 엄청나게 많은 CPU 자원이 소모되므로 Empty Update() 조차 우리의 코드에 존재하면 안 됩니다.

5) Vector2 and Vector3

- CPU에게 생각 보다 불필요한 작업을 많이 시킵니다.
- Update문에서는 Vector보다는 int OR float을 계산에서 사용합니다.

6) Camera.main

- Unity 2020.2 Version에서 Camera.main을 사용해도 됩니다. : <https://www.youtube.com/watch?v=x1Hjt0D4fMs>

7. 문자열 검색 : CompareTag

- 문자열 검색은 보통 GameObject의 name OR tag에서 사용되는데 , 이 때 불필요한 메모리 재 할당이 발생하고 , Garbage Collector가 동작하여 CPU 오버헤드가 증가합니다.

<Tag로 찾기 vs CompareTag("문자열") 로 찾기>

```

void Update() {
    int numTests = 10000000;
    if (Input.GetKeyDown(KeyCode.Alpha1)) {
        for (int i = 0; i < numTests; ++i) {
            if (gameObject.tag == "Player") {
                // do stuff
            }
        }
    }
    if (Input.GetKeyDown(KeyCode.Alpha2)) {
        for (int i = 0; i < numTests; ++i) {
            if (gameObject.CompareTag("Player")) {
                // do stuff
            }
        }
    }
}
}

```

① Tag

= Garbage Collector가 동작 하여 오버헤드가 발생합니다.

② CompareTag

= **Garbage Collector가 동작하지 않아** 기존의 Tag로 탐색하는 것 보다 27%의 속도향상을 이뤄냈습니다.

- 어렵지도 않으니 반드시 **CompareTag**를 사용합시다.

8. Coroutine의 자원 사용

- Coroutine은 일반 함수 호출보다 부가 자원을 약 2배 더 많이 사용합니다.

9. Object Pulling

1) 강의를 듣고 생각해 낸 비유 : 무대가 존재하는 연극

① Prefab으로 GameObject를 만드는 기존 방식

- GameObject를 생성하는 과정은 배우가 집에서 대중교통을 타고 극장에 도착해서 자신의 차례에 무대에 오르는 것입니다.

- GameObject를 제거하는 과정은 배우가 자신의 차례를 마치면 극장에서 대중교통을 타고 집으로 돌아가는 것입니다. 자신의 차례가 되면 집에서 극장으로 다시 와야 합니다.
- 이는 매우 많은 시간이 걸리고 , 힘듭니다.

② Object Pulling을 사용하여 GameObject를 만드는 방식

- 이미 배우들은 무대 뒤에 존재합니다.
- GameObject를 생성하는 과정은 배우가 무대 뒤에 있다가 자신의 차례에 무대에 오르는 것입니다.
- GameObject를 제거하는 과정은 배우가 자신의 차례를 마치면 무대 뒤로 이동하여 대기하는 것입니다. 자신의 차례가 되면 다시 무대로 올라가기만 하면 됩니다.
- 이는 매우 효율적이며 , 힘들지 않습니다.

2) Object Pulling 정의

= **ObjectManager를 만들고 , ObjectManager의 Script에 GameObject들을 미리 생성해 놓고 , 비활성화 해놓습니다. 그리고 필요할 때 활성화시키는 기법입니다.**

- Unity에서는 Prefab을 통해 GameObject를 만들면 Garbage Collector (GC)가 실행되면서 ‘멈춤 렉’이 발생합니다.
- GC를 피하기 위해 Object Pulling은 필수입니다.
- 풀장에 Object들을 미리 풀어 놓는 느낌.
- Object Pulling 방식은 미리 생성해 놓는 방식이기 때문에 당연히 기존 방식에서 생성할 때 마다 발생하는 ‘멈춤 렉’이 한 번에 발생합니다. 그러므로 거대한 ‘멈춤 렉’이 발생하므로 게임의 로딩시간에 Object Pulling을 동작시켜야 합니다. 미리 생성하는 GameObject가 많으면 많을수록 ‘멈춤 렉’이 길어지므로 로딩시간도 길게 설정해야 합니다.
- 전형적인 Back-End 기법

Use object pooling

It's usually more costly to instantiate and destroy an object than it is to deactivate and reactivate it. This is especially true if the object contains start up code, such as calls to *GetComponent()* in an *Awake()* or *Start()* function. If we need to spawn and dispose of many copies of the same object, such as bullets in a shooting game, then we may benefit from [object pooling](#).

Object pooling is a technique where, instead of creating and destroying instances of an object, objects are temporarily deactivated and then recycled and reactivated as needed. Although well known as a technique for managing memory usage, object pooling can also be useful as a technique for reducing excessive CPU usage.

A full guide to object pooling is beyond the scope of this article, but it's a really useful technique and one worth learning. [This tutorial on object pooling on the Unity Learn site](#) is a great guide to implementing an object pooling system in Unity.

- 만들고 지우는 기술 보다 , 활성화(보임)하고 비활성화(보이지 않음) 하는 방식이 훨씬 CPU 오버헤드를 감소시킵니다.

3) Script

① GameObject에서 사용할 Prefab

```
public GameObject enemyLPrefab;  
public GameObject enemyMPrefab;  
public GameObject enemySPrefab;  
public GameObject itemCoinPrefab;  
public GameObject itemPowerPrefab;  
public GameObject itemBoomPrefab;  
public GameObject bulletPlayerAPrefab;  
public GameObject bulletPlayerBPrefab;  
public GameObject bulletEnemyAPrefab;  
public GameObject bulletEnemyBPrefab;
```

② Prefab으로 만든 GameObject 들을 저장할 배열(멤버 변수)

```
GameObject[] enemyL;      enemyL = new GameObject[10];  
GameObject[] enemyM;      enemyM = new GameObject[10];  
GameObject[] enemyS;      enemyS = new GameObject[20];  
  
GameObject[] itemCoin;     itemCoin = new GameObject[20];  
GameObject[] itemPower;    itemPower = new GameObject[10];  
GameObject[] itemBoom;     itemBoom = new GameObject[10];  
  
GameObject[] bulletPlayerA; bulletPlayerA = new GameObject[100];  
GameObject[] bulletPlayerB; bulletPlayerB = new GameObject[100];  
GameObject[] bulletEnemyA; bulletEnemyA = new GameObject[100];  
GameObject[] bulletEnemyB; bulletEnemyB = new GameObject[100];
```

- private로 prefab을 생성하여 저장할 멤버 배열 선언합니다.
- Awake()에서 배열의 크기를 설정합니다.

③ Instantiate를 이용해서 Prefab으로 만들어지는 GameObject들을 생성합니다.

```
for(int index=0; index < enemyL.Length; index++)  
    enemyL[index] = Instantiate(enemyLPrefab);
```

```
public static Object Instantiate(Object original);
```

```
public static Object Instantiate(Object original, Vector3 position, Quaternion rotation);
```

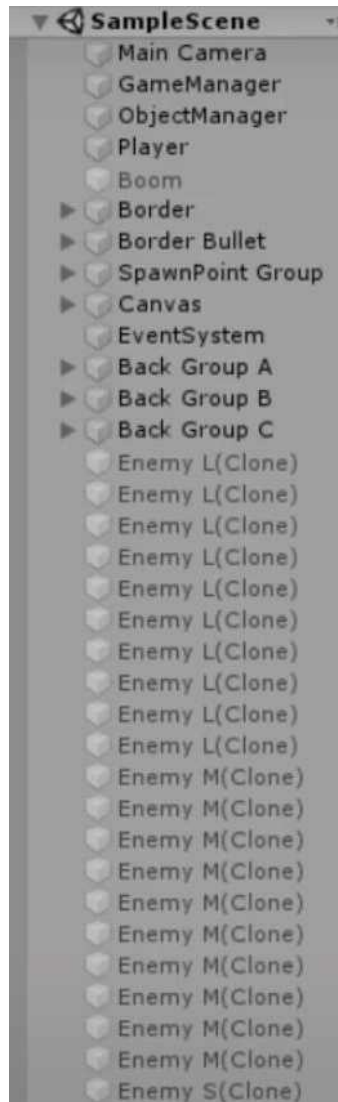
- Position과 Rotation이 필요 없기 때문에 인자는 1개만 필요합니다.
- Instantiate 이후에는 보이면 안 되므로(비활성화) 반드시 해당 GameObject를 SetActive(False) 해버립니다.

④ Public Method로 GameObject들을 활성화합니다.

- 미리 만들어 둔 비활성화된 GameObject들은 모두 private 상태이기 때문에 Public Method를 만들어서 접근할 수 있게 합니다.
- Method의 기능은 대부분 비활성화 된 GameObject 들을 활성화 시키는 기능일 것입니다.

⑤ About Singleton

- 골드메탈님은 입문 강의 특성상 Singleton을 구현하지 않으셨지만 , 나는 Singleton으로 구현하자



- 무수히 많은 GameObject를 Object Pulling으로 만들었지만 모두 SetActive 상태이므로 , Front에서는 변화가 없습니다.

10. 두 물체의 거리 구하기

= 완벽하게 정확한 거리를 요구하는 경우는 API의 거리 함수를 사용하지만 , 대충의 거리를 요구하는 경우는 속도를 위해 API의 거리 제공 함수를 사용하여 최적화를 합니다.

- 컴퓨터가 곱셈의 성능은 뛰어나지만 , 나눗셈에의 성능은 좋지 않은 개념을 적용한 것 입니다.

오브젝트간의 transform position 거리를 체크하고 싶을 때, 세 가지 방법이 있다.

1) Vector3.Distance

2) Vector3.magnitude

3) Vector3.sqrMagnitude

1,2번은 정확한 거리를 계산하게 되고, 3번은 계산된 거리의 제곱의 값을 리턴한다.

3번은 루트 연산을 하지 않으므로, 연산속도가 더 빠르기 때문에

정확한 거리는 몰라도 되고 거리를 비교하는 용도일 때 사용하라고 한다.

(편의를 위해서 Distance라는 함수를 제공하는 것 뿐, 1,2번 결과 및 계산 방식은 동일하다고 함.)

11. 물리

<Debugging>

1. 버그에 대한 과학자들의 생각

- 프로그램 테스트를 통해 버그의 존재를 증명할 수는 있지만 , 버그가 없음을 증명할 수 없습니다.
- 광범위하고 신중한 테스트를 하더라도 모든 경우 혹은 시나리오를 검증한 다는 것은 불가능합니다.
- 그러므로 , 다양한 일반적이고 적절한 환경에서 마주치는 여러 오류를 시간과 예산이 허락하는 한 가능하면 많이 찾아서 바로 잡을 수 있도록 체계적으로 테스트하는 것이 목표입니다.

2. 디버깅 방식_1 : Debug.Log

1) 개요

- 가장 일반적이고 만능의 디버깅 기법
- 일회성 디버깅에 사용합니다.
- 그대로 나두면 자원을 낭비하기도 하고 혼란을 초래하기도 합니다.
- 게임을 빌드해 배포할 시기가 되면 , 간결함을 유지하기 위해 Debug.Log 구문을 모두 지우거나 주석처리 해야 함을 잊지 맙시다.

2) 단점

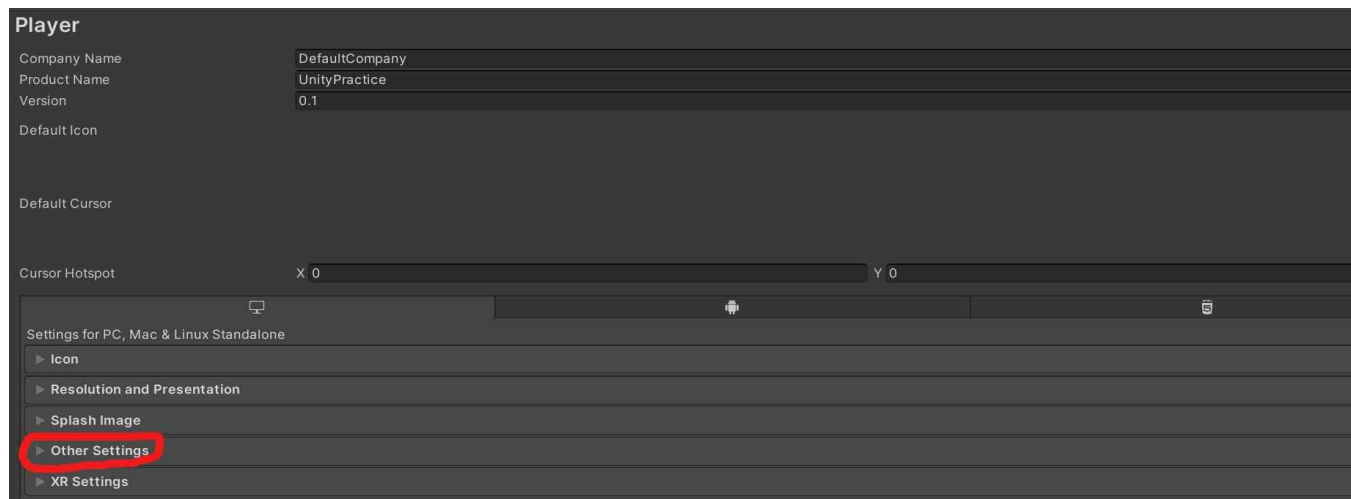
- ① 'Debug.Log' 구문을 명시적으로 코드에 추가해야 하고 , 디버깅을 마치면 일일이 찾아서 수동으로 제거해야 합니다.
- ② 개발자가 특정 코드에서 이제 'Debug.Log'가 필요 없다고 생각하여 지웠는데 다시 필요한 경우에 다시 수동으로 코드에 'Debug.Log'를 추가해야 합니다.

3. 디버깅 방식_2 : #if ~ #endif

- 전처리기 플래그를 이용해서 필요할 때만 디버깅 코드가 동작하도록 하는 방식
- 자주 사용하거나 다음에 사용할 것 같은 디버깅에 사용합니다.
- 대소문자를 구분합니다.
- 'Debug.Log의 단점 ②'를 완벽하게 보완할 수 있는 디버깅 방식

<Script Define Symbols>

- Unity Editor -> Edit -> Project Setting -> Player -> Other Settings -> Script Define Symbols
- 여기에 적는 이름 상수는 프로젝트 전역에서 True로 인식됩니다.
- False로 인식되게 하려면 보통 해당 이름 상수 앞에 '/'을 붙입니다.



```
Scripting Define Symbols
SHOW_DEBUG_MESSAGE
#if SHOW_DEBUG_MESSAGE
    Debug.Log("bye");
#endif
```

```
Scripting Define Symbols
/SHOW_DEBUG_MESSAGE
```

```
#if SHOW_DEBUG_MESSAGE
    Debug.Log("bye");
#endif
```

< #define 으로 할 수 있을까? NO >

솔로몬 ⌚ 2012.08.23 18:04

C# 에서 #define 전처리 지시자는 파일 범위안에서만 효력이 있습니다.

프로젝트 단위로 전처리를 하시려면, 사용하시는 IDE 의 (VS, MonoDevelop)

프로젝트 옵션 메뉴에서 원하시는 전처리 심볼등을 입력하셔서 사용 하시면 되지만,

현재 이 부분이 Unity 에서는 지원이 제대로 되지 않고 있는 것 같습니다.

현재로서는 제가 알고 있기로는 공식적인 방법이 없는 것 같습니다.

- #define은 해당 스크립트에서만 유효하며 프로그램 전체에서 작동하려면 , **스크립트마다 #define을 정의해야 하므로 불편**합니다.
- Singleton을 사용해도 전처리기는 다른 스크립트에서 동작하지 않습니다.

```
#define ONE 1 //C , C++
#define ONE //C# -> TRUE
#undef ONE //C# -> FALSE
```

- C OR C++은 전처리 구분에 값을 할당 할 수 있으나 , C#은 define , undef로 T OR F만 할당 할 수 있습니다.

4. C# 스크립팅 마스터 P107 ~ P127의 기능도 언젠가는 학습을 하겠지?

<Unity Profiler>

- Reference : <https://www.youtube.com/watch?v=WncETrdMG0Q>

1. 최적화에 대한 잡설

- 게임 개발 과정 초기인 prototype 단계에서는 절대로 최적화를 고려하지 않고 코드를 구현합니다.
- 최적화는 본격적인 게임 개발 단계에서 개발 초기인 prototype 단계를 제외하고 , 주기적으로 해야 합니다
- 최적화를 프로젝트 개발 단계 마지막에 몰아서 하면 프로젝트 전체를 갈아엎을 수 도 있습니다.

ex) Othello를 개발하는 과정에서 Room 설계를 1차원 배열로 하고 최적화를 하지 않은 결과 , 마지막이 되어서야 최적화를 진행했고 그 과정에서 2차원 배열로 구현함이 유리했음을 알게 되었지만 코드를 갈아엎지 못해 성능이 나쁜 1차원 배열로 개발을 마쳤습니다.

2. Unity Profiler 개요

- **최적화(=병목을 탐지하고 , 병목을 제거)**를 하는데 개발자에게 도움을 주는 도구입니다.
- profiler를 통해 성능을 측정하고 수치를 비교해서 어느 곳의 성능을 향상 할 수 있는지 평가할 수 있지만 코드에 특정 버그가 존재하는 알려주는 도구는 아닙니다.
- 그래프에서 급격한 증가는 무거운 동작을 표시하기 때문에 해당 지점을 유심히 살펴봐야합니다.
- 단축키 : Ctrl + 7

3. 용어

1) Editor Loop

EditorLoop in the profiler means the resources used by the UnityEditor itself. While running your game in the editor this overhead is added, but don't worry the standalone build won't have this overhead.

- 이 수치가 비정상적으로(80%이상) 높다면 , Unity Editor의 Scene Tap을 끄십시오.

4. 직접 공부해 본 Profiler

- 3D-fortress_refactoring의 PlayerMovement Script와 DestroyItem Script의 FixedUpdate를 분석해보았습니다.
- 편의상 PlayerMovement.FixedUpdate()를 PF , DestroyItem.FixedUpdate()를 DF라고 설명하겠습니다.
- Update 함수도 마찬가지로 분석할 수 있습니다.

1) Hierarchy 모드

<그림 1 : 전반적인 모습>

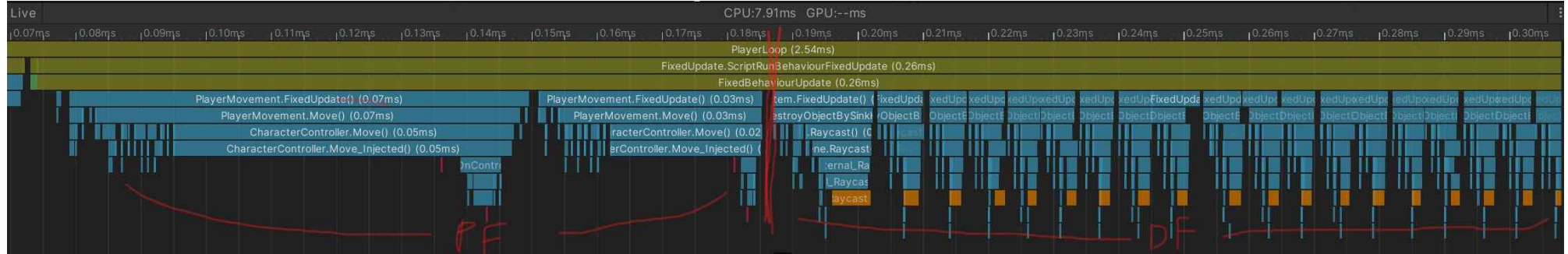
Overview	Total	Self	Calls	GC Alloc	Time ms	Self ms
EditorLoop	50.7%	50.7%	2	0 B	4.01	4.01
▼ PlayerLoop	46.4%	1.1%	2	0.6 KB	3.67	0.09
▶ FixedUpdate.ScriptRunDelayedFixedFrameRate	19.6%	0.0%	1	0 B	1.55	0.00
▶ Camera.Render	12.3%	1.9%	1	0 B	0.97	0.15
▶ FixedUpdate.PhysicsFixedUpdate	3.8%	0.1%	1	0 B	0.30	0.01
▼ FixedUpdate.ScriptRunBehaviourFixedUpdate	3.2%	0.0%	1	252 B	0.25	0.00
▼ FixedUpdateBehaviourUpdate	3.2%	0.2%	1	252 B	0.25	0.01
▼ DestroyItem.FixedUpdate()	1.6%	0.0%	23	0 B	0.13	0.00
▶ DestroyItem.DestroyObjectBySinkHole()	1.6%	0.1%	23	0 B	0.12	0.01
▼ PlayerMovement.FixedUpdate()	1.3%	0.0%	2	252 B	0.10	0.00
▶ PlayerMovement.Move()	1.2%	0.0%	2	252 B	0.09	0.00
▶ PlayerMovement.get_currentSpeed()	0.0%	0.0%	2	0 B	0.00	0.00
Behaviour.get_enabled()	0.0%	0.0%	4	0 B	0.00	0.00
PlayerInput.get_isfiredown()	0.0%	0.0%	2	0 B	0.00	0.00
▶ Vector2.op_Implicit()	0.0%	0.0%	1	0 B	0.00	0.00
PlayerInput.get_moveInput()	0.0%	0.0%	1	0 B	0.00	0.00
PlayerInput.get_jump()	0.0%	0.0%	2	0 B	0.00	0.00
PlayerInput.get_isfire()	0.0%	0.0%	2	0 B	0.00	0.00
Vector3.get_zero()	0.0%	0.0%	1	0 B	0.00	0.00
▶ WindManager.FixedUpdate()	0.0%	0.0%	1	0 B	0.00	0.00
▶ GameManager.FixedUpdate()	0.0%	0.0%	1	0 B	0.00	0.00

<그림 2 : GC>

▼ PlayerMovement.FixedUpdate()	1.4%	0.0%	2	252 B	0.10	0.00
▼ PlayerMovement.Move()	1.3%	0.0%	2	252 B	0.09	0.00
▼ CharacterController.Move()	1.0%	0.0%	2	252 B	0.07	0.00
▼ CharacterController.Move_Injected()	1.0%	0.8%	2	252 B	0.07	0.06
▶ ItemAndObstacleReact.OnControllerColliderHit()	0.1%	0.0%	2	92 B	0.00	0.00
GC.Alloc	0.0%	0.0%	2	160 B	0.00	0.00

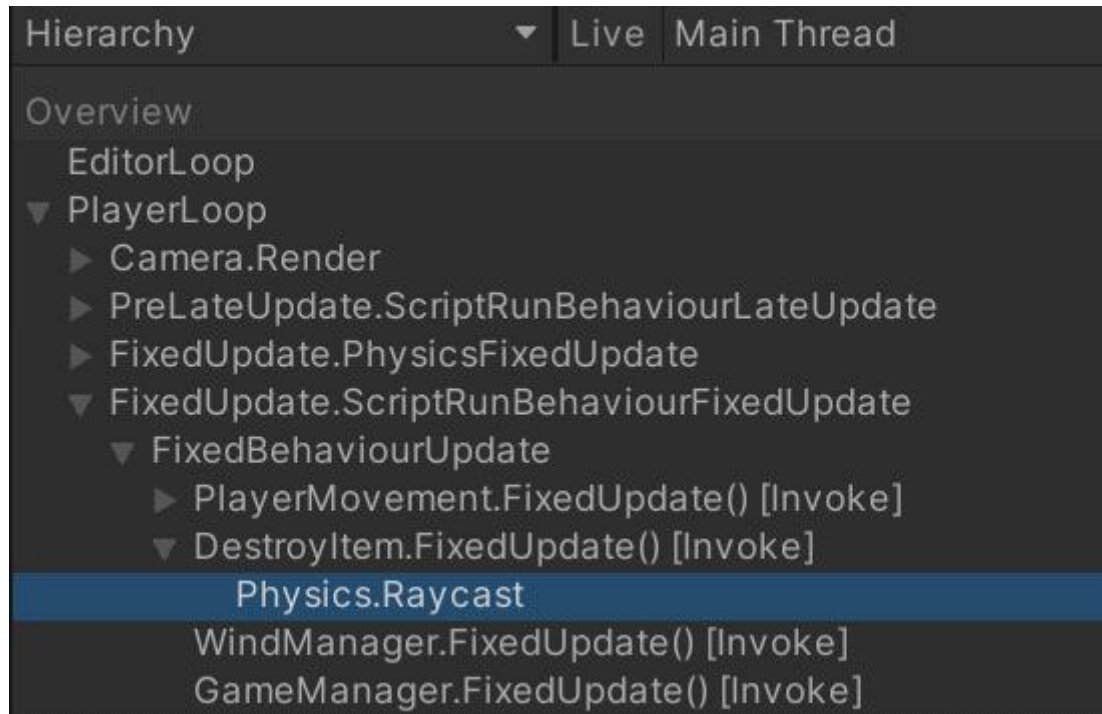
- DF안에는 DestroyObjectBySinkHole (내가 만든 함수)만이 존재하고 , 이것이 23번(이 게임에서 삭제 될 아이템이 23개이므로) 호출되어 총 1.6%를 점유하고 0.13ms의 지연을 시키고 있고 , GC는 동작하지 않습니다.
- PF안에는 Move (내가 만든 함수)와 다른 여러 변수가 존재하지만 Move를 제외하고는 0.1%만 차지 하므로 무시할 수 있습니다. Move함수는 고작 2번 (이 게임의 Player가 2개이므로)만 호출되지만 Garbage Collector가 동작하여 무려 1.3%를 점유하고 0.10ms를 지연시키고 있습니다.
- 이처럼 Unity Profiler로 분석된 결과를 바탕으로 어떤 함수로 인해 Overhead가 발생하여 , 게임의 성능을 저하시키는지 알 수 있고 , 이를 해결하여 게임의 성능을 향상 시킬 수 있습니다.

2) Timeline 모드



- Hierarchy 모드를 도식화 한 모드입니다.
- PF는 2번 호출되고 (이 게임의 Player가 2개이므로), DF는 23번 호출(이 게임에서 삭제 될 아이템이 23개이므로)됩니다.
- PF는 고작 2번 호출되지만 1.3%를 점유하고 , DF는 23번 호출되어 1.6%를 점유합니다. 그림에서도 알 수 있듯 PF 1개의 크기가 DF 1개의 크기보다 훨씬 큼니다.

<초보자는 Deep Profile을 키자>



end

- 상단의 그림은 Deep Profile을 활성화 시킨 것이고 , 좌측의 그림은 활성화 시키지 않은 것입니다.

- 활성화 시킨 그림에서는 DestroyItem.FixedUpdate() 내부의 어떤 함수(DestroyObjectBySinkHole)가 존재하는지 알 수 있습니다.

- 좀 더 자세하게 , 개발자가 사용한 함수를 알 수 있다는 장점이 있지만 , 리소스를 많이 잡아먹기 때문에 Unity Profiler가 능숙해지면 비활성화 시키는 것을 권장합니다.

<ERROR>

*** 작업할 때 마다 발생하는 ERROR를 기록해 놓습니다. ***

1. ArgumentOutOfRangeException



- 배열 OR 자료구조의 범위에 벗어난 인덱스를 참조할 때 발생하는 ERROR입니다.
- ERROR에서는 음수 인덱스 OR 사이즈를 초과한 인덱스를 참조할 때 발생한다고 명시했으나, 제가 테스트 해본 결과 음수 인덱스는 올바르게 출력이 되지만 사이즈를 초과한 인덱스를 참조할 때는 ERROR가 발생합니다.
- ex) 'List list' 의 크기가 5일 때 list[-1], list[-2]는 올바르게 출력이 됩니다. 하지만 list[5], list[6]을 요구할 때는 ERROR가 발생합니다. 그리고 list[-1]을 list[5-1] = list[4]로 바꾸는 C_Language Type이 아닌 진짜 list[0-1]의 list[-1]의 값을 출력시킵니다.
- Reference : <https://docs.microsoft.com/ko-kr/dotnet/api/system.argumentoutofrangeexception?view=net-5.0>

2. 클래스 타입의 배열을 만들 때 생기는 NullReference 오류 : 생성자를 통해 초기화 하여 해결

```
public class MapInfo
{
    public GameObject tile;
    public int state;

    public MapInfo(GameObject obj , int state)
    {
        this.tile = obj;
        this.state = state;
    }
};

private MapInfo[] front_map = new MapInfo[TILES_PER_MAP];
private MapInfo[] back_map = new MapInfo[TILES_PER_MAP];

private const int TILES_PER_MAP = 100;

for (int i=0; i<TILES_PER_MAP; i++)
{
    //PERFECT
    front_map[i] = new MapInfo(front_map_parent.transform.GetChild(i).gameObject, (int)OBJ.NONE);
    back_map[i] = new MapInfo(back_map_parent.transform.GetChild(i).gameObject, (int)OBJ.NONE);

    //MAKE ERROR
    front_map[i].tile = front_map_parent.transform.GetChild(i).gameObject;
    back_map[i].tile = back_map_parent.transform.GetChild(i).gameObject;
    front_map[i].state = (int)OBJ.NONE;
    back_map[i].state = (int)OBJ.NONE;
}
```

- 크기가 MapInfo형 배열을 만들어서 객체를 모두 생성했다고 생각하고 , MAKE ERROR 부분의 코드를 작성했습니다.
- 하지만 배열의 객체는 만들었지만 배열 안에 저장되는 MapInfo형 데이터는 객체를 만들지 않아 NullReference가 발생합니다.
- 생성자를 이용해 MapInfo형 객체를 생성하며 MapInfo 타입의 배열을 선언합니다.

<Prefab> 20/07/11

1. Prefab

- = 동일한 GameObject를 언제든지 재사용할 수 있도록 하기 위해 미리 제작된 GameObject입니다.
 - 그러므로 prefab도 당연히 GameObject입니다.
 - Prefab은 GameObject이므로 , script와 component를 모두 할당 받을 수 있습니다.
 - Prefab은 복잡한 게임 오브젝트를 run_time 시점에 인스턴스화 하려는 경우에 매우 유용합니다. 코드도 매우 간단합니다.
 - 다른 Scene에서도 사용이 가능합니다.
 - Prefab은 public OR [SerializeField]를 이용하여 Unity Editor에서 가져옵니다.(더 안전한 방법이 있을 수 있음)
 - 생성 되는 위치가 카메라에서 보이지 않은 경우 occlusion culling으로 인해 자동으로 삭제 OR 중력을 받아서 떨어지는 건지 , 생성이 안되는 경우를 경험했습니다.
- ex) 똑같은 탄알100개 , 똑같은 몬스터 10마리

2. Prefab을 Script 내에서 생성하기 : Instantiate()

```
//총알 쏘기
public void Fire()
{
    int speed = 10;
    //prefab들을 실제로 생성하기
    //인자 : (프리팹 , 생성될 위치(벡터) , 생성됐을 때 방향
    GameObject bullet = Instantiate(bullet_a, transform.position, transform.rotation);

    //bullet의 rigidbody를 가져옵니다.
    Rigidbody2D rigid = bullet.GetComponent<Rigidbody2D>();
    //위로 속도를 줍니다.
    rigid.AddForce(Vector2.up * speed , ForceMode2D.Impulse);
}
```

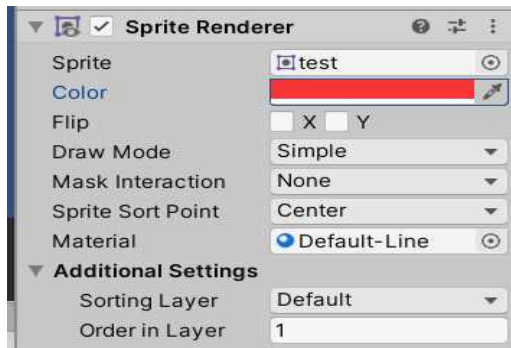
- Instantiate method : `Instantiate(GameObject , position , rotation)`

<2D Sprite> 20/06/27

1. 정의 : 그래픽 오브젝트(이미지)

2. Sprite Renderer & Material

1) Sprite Renderer



- 이 component에서 게임에서 렌더링 될 게임오브젝트의 스프라이트를 정할 수 있습니다.
- Mesh Renderer와 함께 사용이 불가능합니다.
- order in layer : 작을수록 뒤에 존재 합니다. (가장 큰 값이 화면에 보임)

2) Material

① 정의 및 특징

= **GameObject의 표면(Surface) 질감**을 어떻게 처리할 것인가를 설정하는 속성입니다. (컴퓨터 그래픽스)

- GameObject와는 별개로 Asset의 폴더에서 Material 폴더를 따로 만들어 , Material들을 저장합니다.

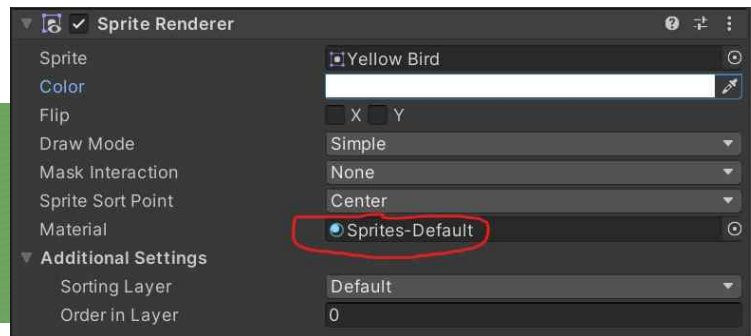
- Sprite가 '사람 그 자체'라면 Material은 '사람 전체를 덮는 불투명한 옷'으로 이해해도 괜찮습니다. 이렇게 때문에 Sprite의 모양과 기본 색상위에 Material을 입혀도 모양은 보존되며 , 색상이 변하지만 Sprite의 색상이 조금은 영향을 줍니다. 아래의 예시를 보고 이해하는 것이 가장 바람직합니다.

- Sprite 보다는 Mesh와 연관 관계가 더 큼니다.

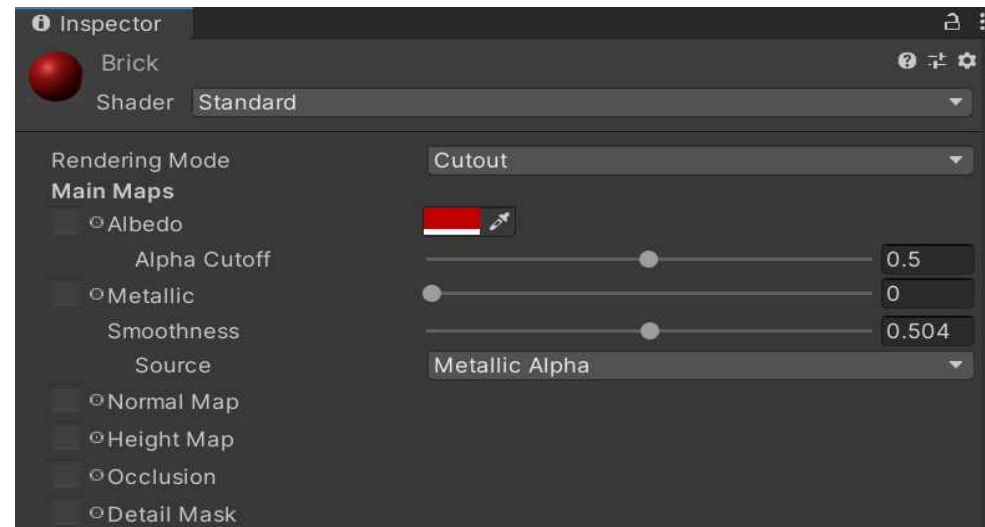
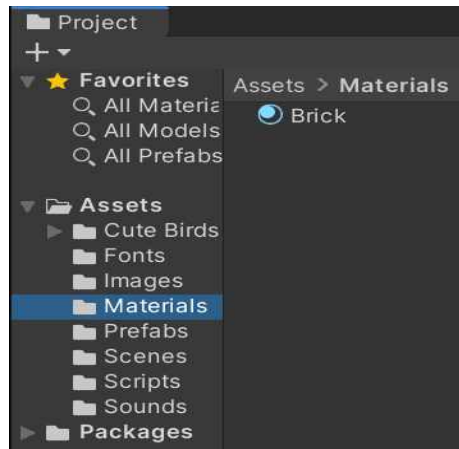
ex) : Material 안에는 벽돌 느낌의 질감 , 솜 느낌의 질감 같이 질감들만 모아놓습니다.

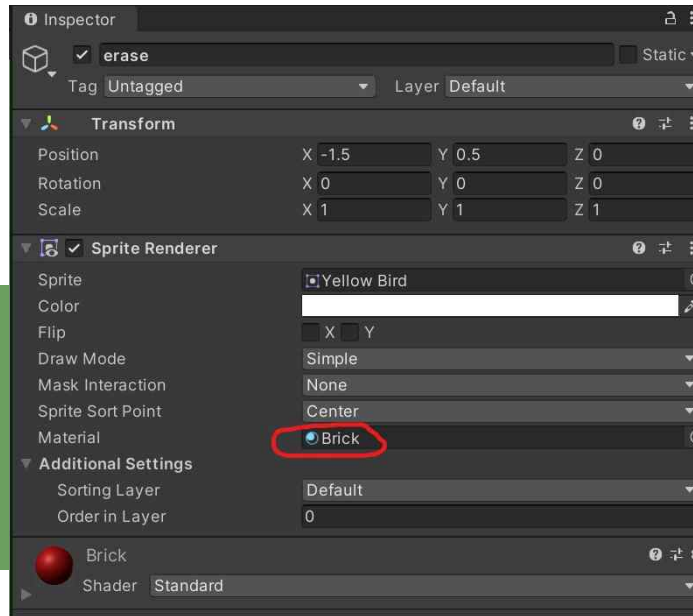
② Script

<기본 Sprite>



<“Brick” Material을 입힌 Sprite>





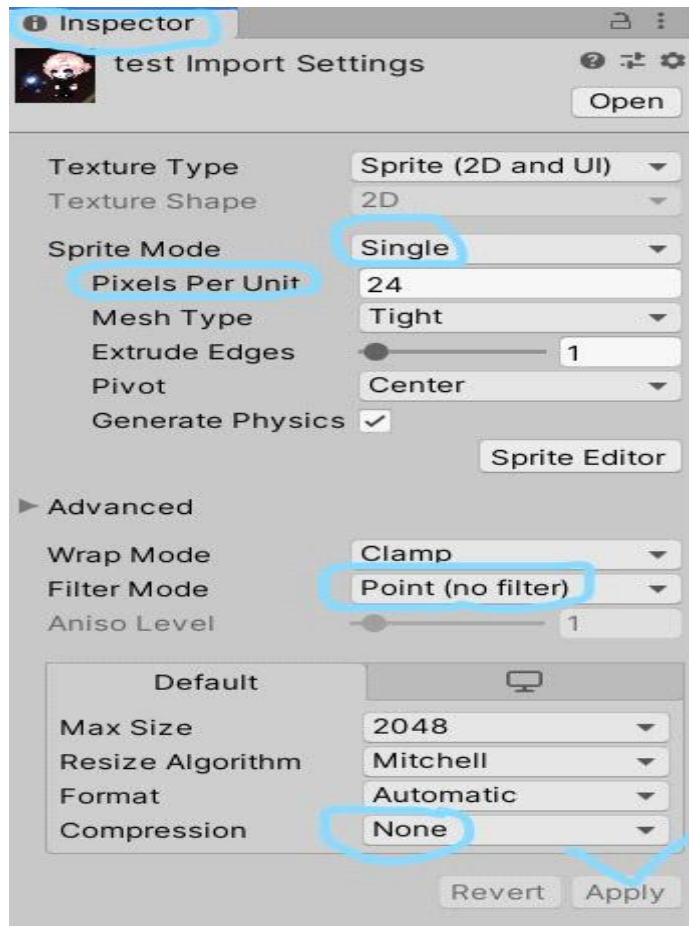
- 병아리의 모양은 유지되나 , 색은 Material의 질감에 영향을 받아 변했습니다.

③ Material 속성

Reference :

<http://blog.naver.com/PostView.nhn?blogId=dj3630&logNo=221462605089&from=search&redirect=Log&widgetTypeCall=true&directAccess=false>

3. Sprite's Inspector



1) advanced -> filter mode -> point(no filter)

2) advanced -> compression -> None

3) 이미지 여러 개가 묶인 이미지라면 Sprite Mode -> Multiple

4) pixels per unit

= 현재 이미지의 크기는 pixels per unit을 변경한다고 해도 언제나 변하지 않습니다. 하지만 pixels per unit을 통해 현재 Scene의 한 격자의 길이를 정할 수 있습니다.

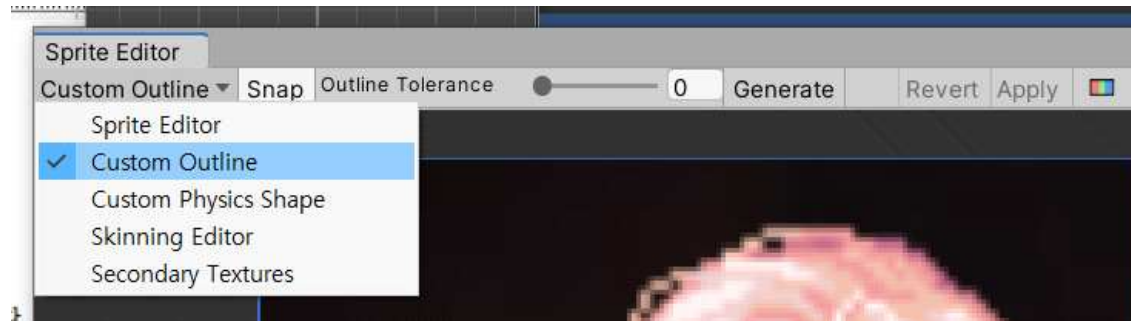
- Unit은 Unity의 Scene에서 1칸을 의미합니다.

- 이미지가 (16x16)일 때 pixels per unit(Unity 1칸)을 16으로 하면 Scene의 Unit의 가로가 16 , 세로가 16이 됩니다. 그러면 이미지는 격자 1칸에 알맞게 들어갑니다. 하지만 pixels per unit 을 32로 하면 Scene의 격자의 가로가 32 세로가 32가 됩니다. 그러면 이미지는 격자의 1/4칸만 차지하게 됩니다.

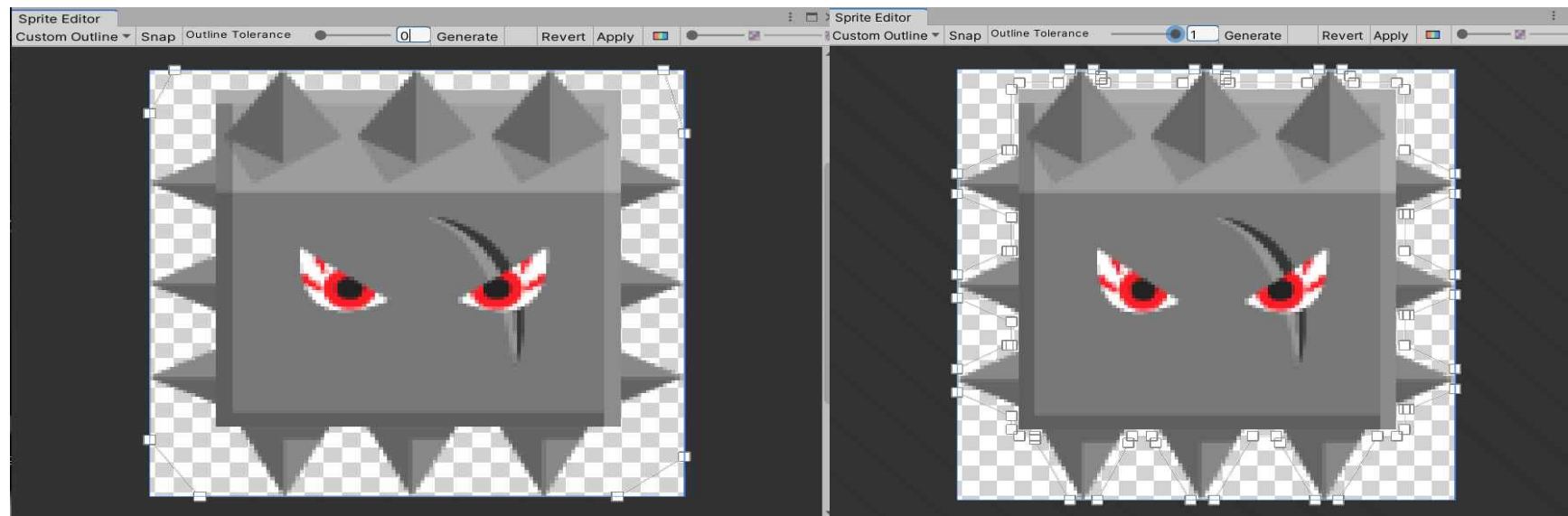
- 사용하고자 하는 png 파일의 속성을 보면 (1024*768)같이 해당 이미지가 몇 pixel로 구성되어 있는지 알 수 있으므로 참고하여 이용합시다.

4. Sprite Editor

= Sprite의 모든 것을 편집할 수 있는 탭입니다.



1) Custom Outline



- Mesh Outline : Sprite를 둘러싸고 있는 하얀 네모와 선을 종합한 윤곽선 입니다.
- Snap : 무조건 활성화 시킨다고 생각합니다. Outline Tolerance의 정도에 따라 자동으로 윤곽선을 그려줍니다.
- Outline Tolerance : 0부터 1까지 float으로 조절할 수 있습니다. 왼쪽그림이 0이고 오른쪽 그림이 1인데 숫자가 클수록 윤곽선을 Sprite의 실제 크기에 가깝게 맞춰줍니다. (Default로 1로 합시다)
- Generate를 켜고 Custom Outline을 설정합시다.

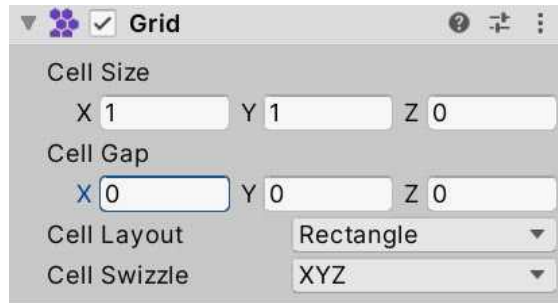
2) Custom Physics Shape

- Custom Outline과 적용방식은 완전히 일치합니다.
- Custom Outline은 Sprite의 이미지 부분이고 , Custom Physics Shape는 물리효과인 충돌의 윤곽선을 담당합니다.
- Polygon Collision을 입히면 여기서 만든 Custom Physics Shape이 적용됩니다.

3) 여러 개의 Sprite가 합쳐진 Sprite는 Multiple 속성으로 바꾸고 Slice를 할 수 있습니다.

<TileMap> 20/06/30

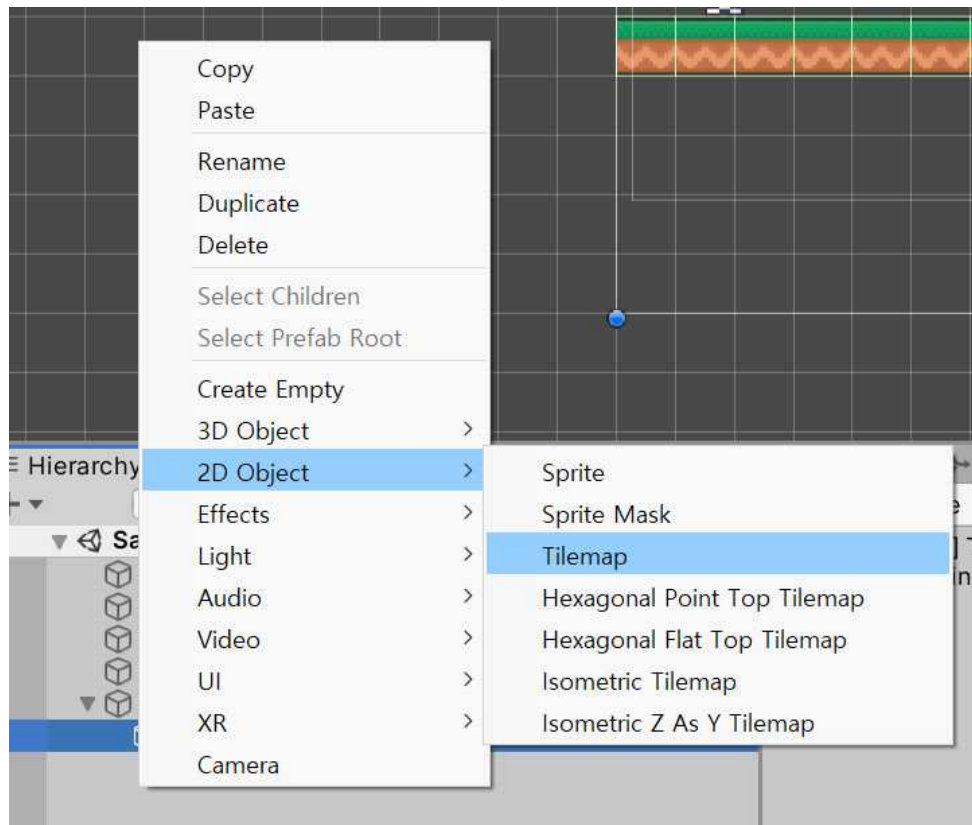
1. Grid



프로퍼티	기능
Cell Size	그리드의 셀 크기입니다.
Cell Gap	이 옵션을 사용하여 그리드 셀 사이의 간격 크기를 정의합니다. 이 옵션은 Cell Size 보다 낮은 음수 값으로 설정할 수 없습니다. 그럴 경우 Unity가 Cell Size 와 동일한 값으로 자동 재 설정하고 음수 값으로 변경합니다. 예를 들어 Cell Size가 1, 1, 0이고 Cell Gap 을 -2, -2, 0으로 설정하려고 시도하면 Unity가 Cell Gap 값을 자동으로 -1, -1, 0으로 변경합니다.
Cell Layout	이 드롭다운을 사용하여 그리드 셀의 모양과 배열을 정의합니다.
Rectangle	셀이 직사각형입니다.
Hexagon	셀이 육각형입니다.
Isometric	셀이 아이소메트릭 레이아웃에 대해 마름모 모양입니다.
Isometric Z as Y	Isometric Grid 레이아웃과 비슷하지만, Unity가 셀의 Z 위치를 해당하는 로컬 Y 좌표로 변환합니다.

2. Tilemap 기본 & Tilemap Renderer

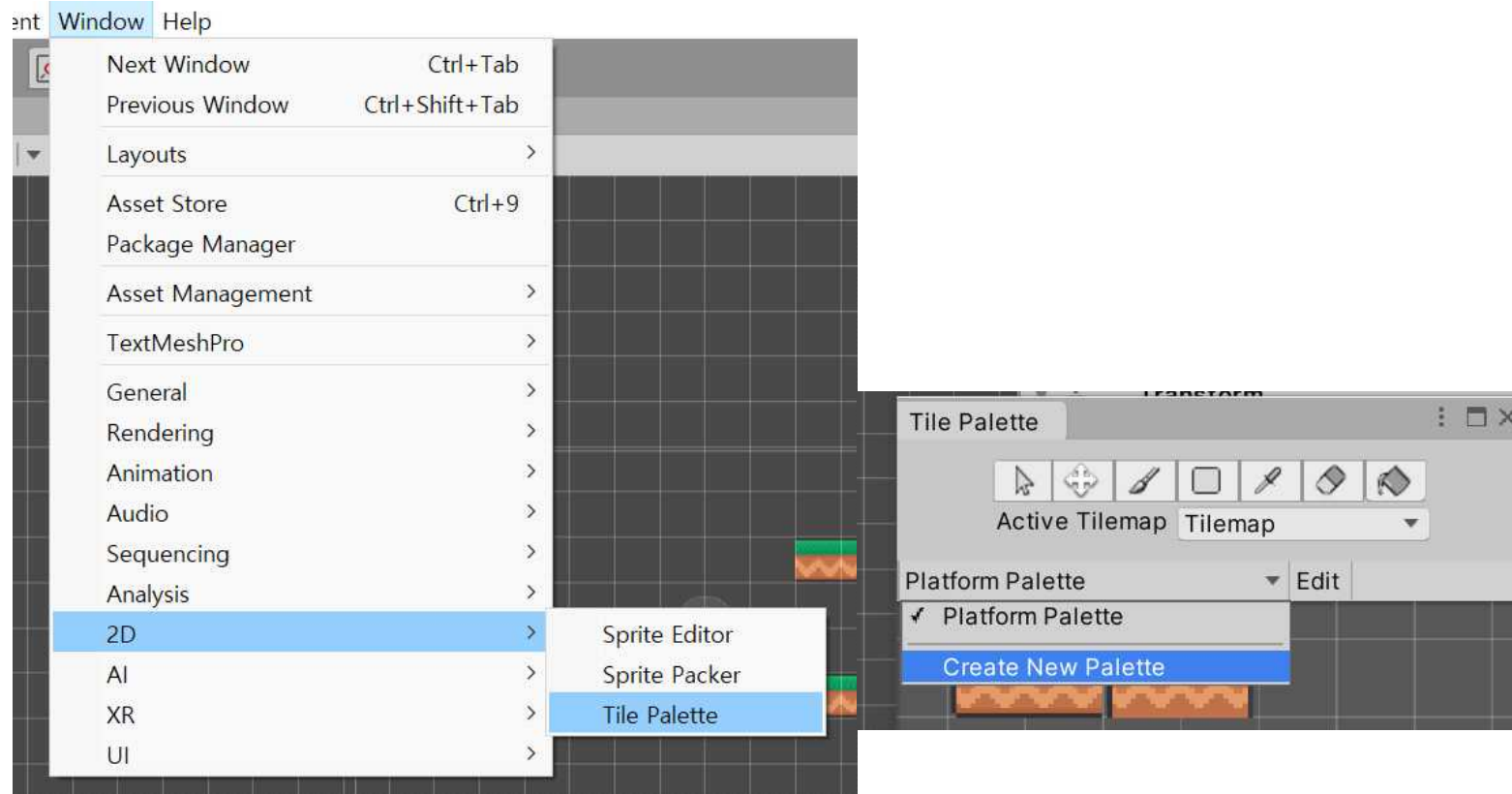
- = **Tilemap 전체가 하나의 GameObject**입니다.
- = Tilemap의 component를 지정하면 Tilemap 속의 모든 오브젝트에 똑같이 지정됩니다.
- = Tilemap은 TilePalette와 Grid가 존재하므로 Renderer에 중요한 기능이 적은 느낌입니다.
- = Tilemap을 만들면 default로 생성됩니다.



3. TilePalette(미술에서 사용하는 팔레트와 같은 개념)

1) TilePalette 생성

Create New Palette를 하면 어디에 저장할지 물어봅니다. Asset에 새로운 폴더를 만들어 자료들을 저장합니다.



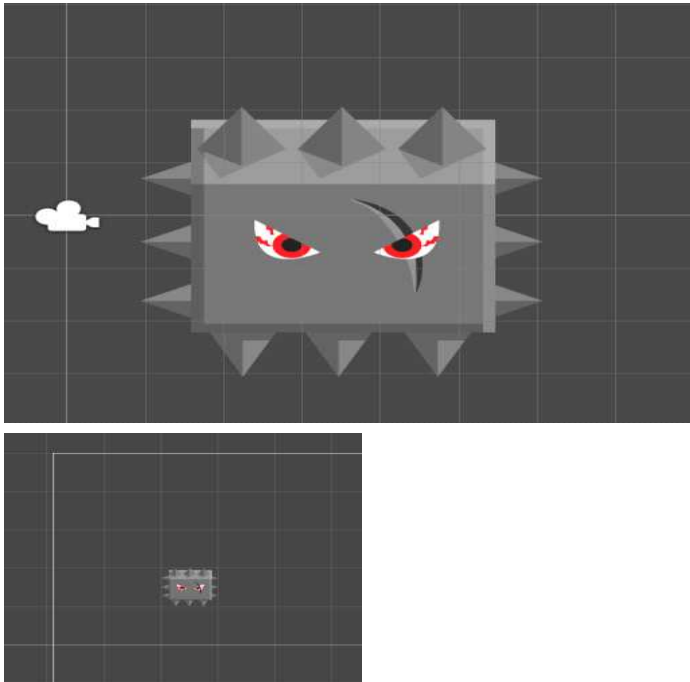
2) 타일 팔레트로 그림 그리기

- 반드시 Hierarchy -> 2D -> TileMap을 만들고 그 위에 그림시다.
- Edit을 켜면 스프라이트를 타일 팔레트에 올릴 수 있습니다.

(Edit 도구 : <https://docs.unity3d.com/kr/2019.4/Manual/Tilemap-Painting.html>)

- Edit을 끄면 TileMap에 그릴 수 있습니다. (그리기 도구 이용)

3) 타일 팔레트에서 Sprite 크기 맞추기



이 Sprite는 512 x 512 픽셀의 이미지입니다.

어떤 일이 있어도 Sprite의 절대적인 사이즈는 바꿀 수 없습니다.

위의 그림은 Sprite의 pixels per unit이 100입니다. 다시 말해 Tile Palette의 1칸을 100으로 보겠다는 뜻입니다. 그림에서 5.12칸씩을 차지하고 있습니다.

아래처럼 Fit하고 싶다면 pixels per unit을 512로 바꾸면 됩니다. 1칸을 512로 보겠다는 뜻이니 512 x 512의 Sprite는 1칸을 온전히 차지하게 됩니다.

언제나 이미지를 가져올 때 $m \times n$ 의 크기를 잘 봅시다.

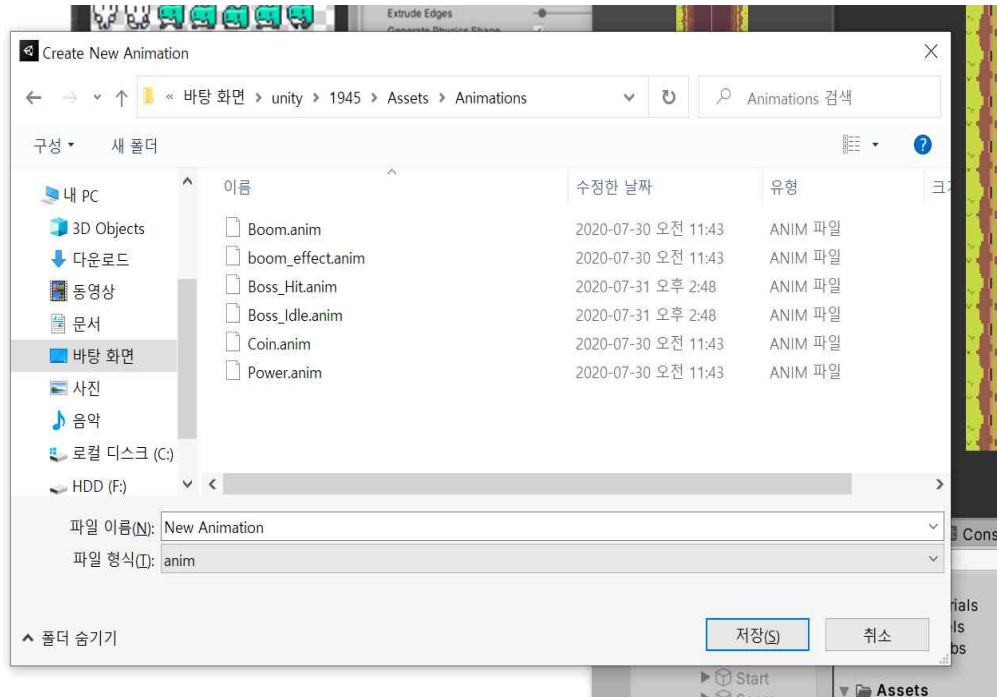
4. Tilemap Collider

= 타일맵의 타일 하나하나에 Collider가 생성됩니다.

5. 타일맵은 테트리스를 공부하면서 좀더 추가기능을 적어봅시다.

<Animation> 20/06/28

1.애니메이션 넣기



Sprites 들의 모임을 Hierarchy의 GameObject 로 드래그 합니다.

그러면 해당 그림이 나오게 되는데 Assets에서 내가 만든 Animations 폴더에 sprites들을 저장 합니다.

이렇게 하면 Default 애니메이션은 만든 것입니다.

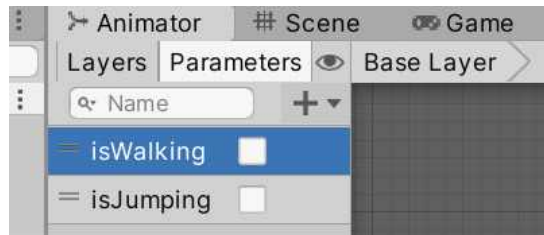
2. Animation 과 Animator

windows -> Animation 탭에서 관리합니다.

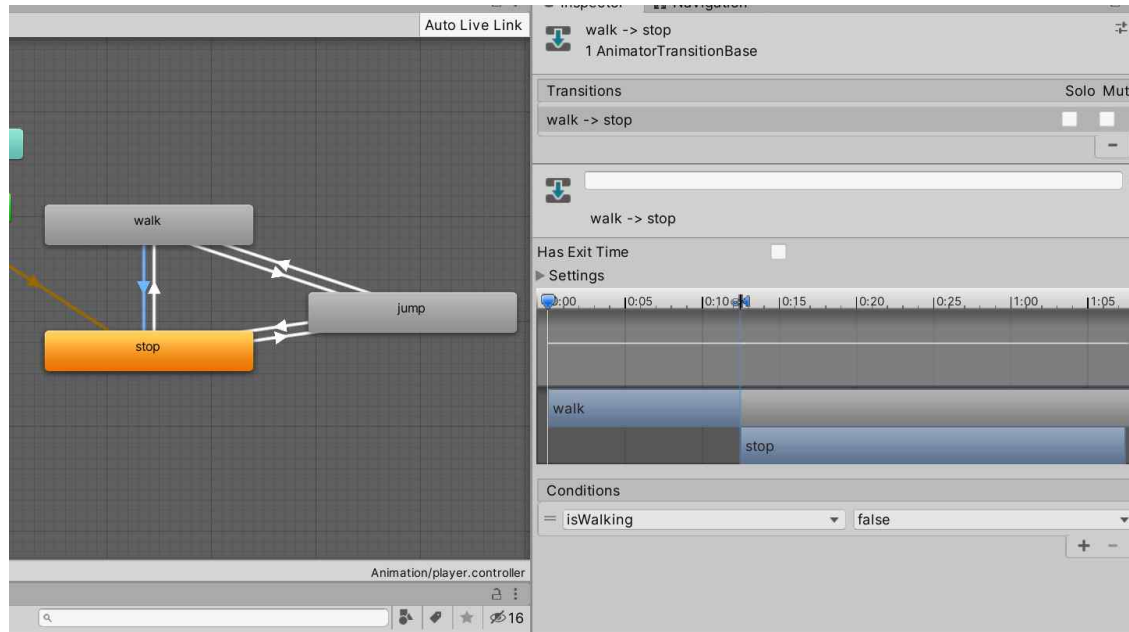
Animation = Scene에서 애니메이션이 들어 있는 GameObject를 누르고

Animator =

4) 플레이어 상황별 애니메이션 변경



parameter에서 bool을 선택하고 변수를 하나 만든다. -> script에서 이용



<Animator 작업>

animator에서 네모 박스에 make transition으로 다음 행동으로 연결한다.

settings -> 애니메이션의 겹치는 시간 -> 보통 0으로 한다.

settings -> has exit time (이전 애니메이션을 끝가지 하기) -> 끄기

화살표의 inspector에서 conditions의 bool변수 값을 변경시킵니다.

conditions가 의미하는 것은 우리는 반드시 script에서 우리가 만든 parameter의 값을

조건문을 통해 변경 시킬 것입니다. 그러면 conditions에서 값의 변경을 감지하고 그에 알맞은 애니메이션으로 변경시킵니다.

<script 작업>

```
//Animator ani = GetComponent<Animator>();
//normalized가 0이면 = 멈췄다. -> 내가 만든 iswalking을 false로 한다.
if (rigid.velocity.normalized.x == 0)
{
    ani.SetBool("isWalking", false);
}
else
{
    ani.SetBool("isWalking", true);
}
```

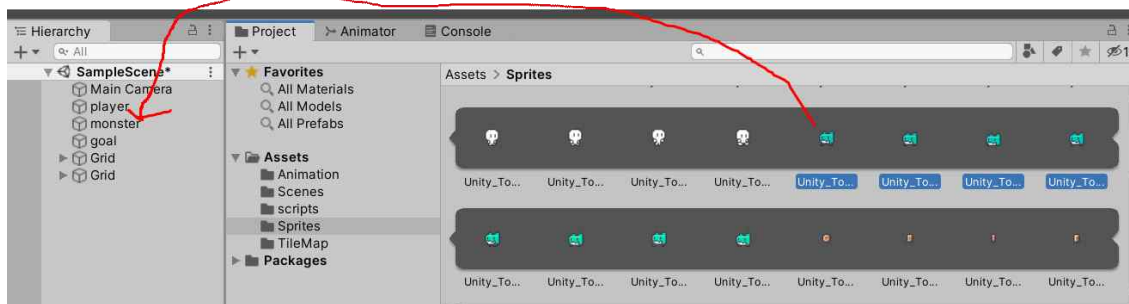
Animator 클래스의 변수 ani를 만든다.

update에서 현재 속도가 0이면 ani.SetBool("isWalking", false)

속도가 0이 아니면 다시 true로 하여서 바꾸어 준다.

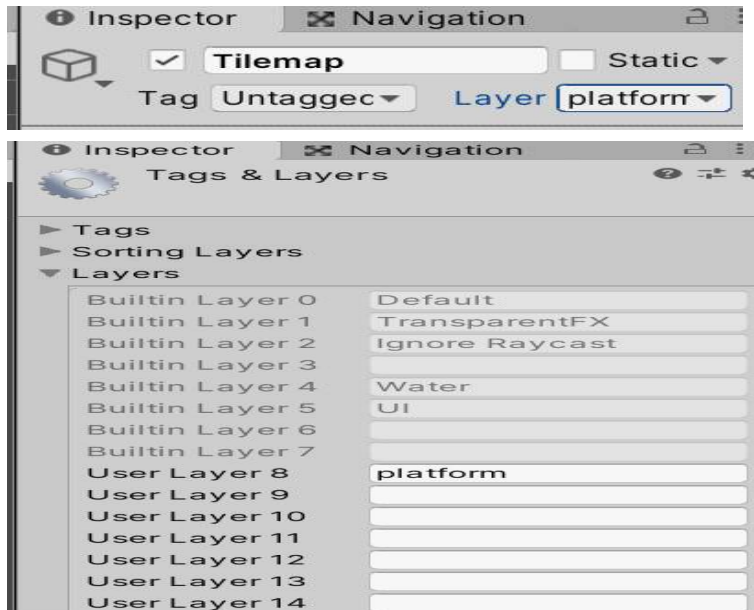
1. Default Animation

애니메이션에 사용할 그림들을 shift로 묶어서 해당 object에 드래그하면 끝



<Layer & Tag & Name> 20/06/29

1. Layer

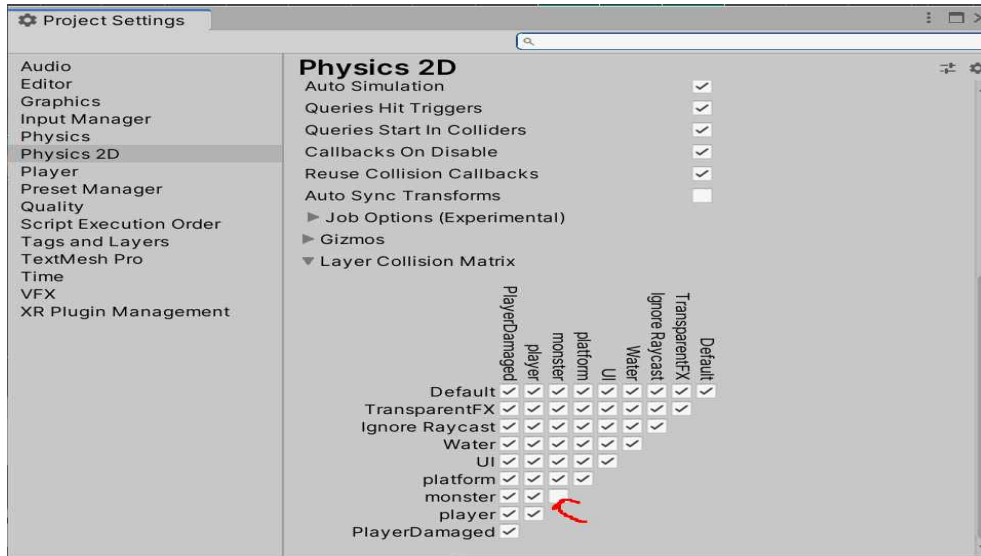


- Layer칸에 넣어 줄 Layer는 사용자가 직접 만들어서 많이 사용합니다. Tag또한 마찬가지 입니다.

1) 정의

- Layer = 슈트(옷) = 사용자 지정 충돌 상태
- GameObject에 입혀주는 슈트입니다. 슈트에 따라서 다른 오브젝트와의 충돌여부를 변경할 수 있습니다.
- 'player'라는 layer를 만들어도 enemy한테 입힐 수 있습니다.
- 예를 들어 플레이어의 layer가 player이면 몬스터한테 피격이 되는 상태의 layer이고, layer가 player_damaged이면 플레이어가 몬스터한테 피격을 당하지 않는 무적상태의 layer인 것입니다. 따라서 해당 상태가 될 때 마다 c#에서 오브젝트의 layer를 변경 해줘서 해당 오브젝트는 슈트를 갈아입게 됩니다.

2) 지정한 layer들끼리 충돌여부를 결정 짓기.



- Edit -> project_setting

- 설정한 두 레이어 끼리 충돌하지 않게 하려면 체크 해제 layer가 monster인 GameObject와 layer가 monster인 오브젝트는 충돌 하지 않도록 지정하였습니다.

- player_damaged로 player(GameObject)의 layer를 바꾼 후에 player_damaged에서 monster를 해제하면 player는 몬스터와 피격하지 않습니다.
무적 기능 구현에 유용합니다.

<스크립트에서 Layer 변경 -> Invinsible이라는 이름을 갖는 Layer로 변경 >

```
gameObject.layer = LayerMask.NameToLayer("Invinsible");
```

2. Tag

```
//충돌발생한 오브젝트가 "Border"일때
//Border중 4가지 방향의 조건문 형성
if (collision.gameObject.tag == "Border")
{
    if (collision.gameObject.name == "Top")
    {
        touch_top = true;
    }
    else if (collision.gameObject.name == "Bottom")
    {
        touch_bottom = true;
    }
    else if (collision.gameObject.name == "Right")
    {
        touch_right = true;
    }
}
```

- 태그의 목적은 명확하게 GameObject를 스크립트에서 확인하는 것입니다. 스크립트 코드를 작성할 때 찾으려는 태그에 속하는 GameObject를 찾을 수 있습니다

- 왼쪽의 캡처는 충돌하는 GameObject의 태그를 통해 찾는 것이고, 하단의 캡처는 Border라는 태그를 가진 게임오브젝트를 찾고 y축의 위치를 로그에 나타내는 것입니다.

- FindGameObjectWithTag("tag") : 태그가 "tag"인 처음 찾는 1개의 GameObject만 가져옵니다.

- FindGameObjectsWithTag("tag") : 태그가 "tag"인 모든 GameObject를 가져옵니다.

- 하나의 GameObject에는 한 개의 태그만이 가능합니다.

```
Debug.Log("six : " + GameObject.FindGameObjectWithTag("Border").GetComponent<Transform>().position.y);
```

- Unity Editor에서 태그이름을 만들지 않고 , Script에서 해당 태그이름으로 설정하면 ERROR가 발생합니다.

3. Name

- Tag와는 다르게 문자열로 검색하기 때문에 속도가 느려서 쓰지 않는 개념으로 생각해왔습니다. 하지만 Name이 필요한 경우가 존재합니다.

<Example : Prefab에서 생성한 Article 중에서 특정 위치에 존재하는 Article 제거>

```
GameObject player1_article = Instantiate(InitPlayerCharacter.Instance.player1_character, player_manager.rooms[clicked_room_number].room.transform.position, transform.rotation);  
player1_article.name = "Article" + clicked_room_number.ToString();
```

```
destroy_article_index = same_line_articles[i].ToString();  
destroy_article = GameObject.Find("Article" + destroy_article_index);  
Destroy(destroy_article);
```

- Prefab을 100개 만들고 태그를 모두 "Article"로 하면 100개의 GameObject 중에서 35번째의 GameObject를 태그로 찾는 방법은 존재하지 않습니다. 태그를 Article_1, Article_2, etc, Article_100으로 설정하면 될 것 같지만, 태그의 가장 큰 단점은 Unity World에서 해당 태그를 생성해주어야 한다는 것 입니다. 다시 말해, 없는 태그를 Script에서 설정해주면 ERROR가 발생합니다.
- 그러므로, Prefab 100개를 만들고, 만들 때 마다 이름을 모두 Article+번호로 작성해주면 모든 100개의 GameObject를 이름으로 구분할 수 있게 됩니다. 그 결과, 삭제할 때 원하는 이름을 적어주면 해당 이름을 가진 GameObject를 삭제할 수 있게됩니다.

<UGUI> 20/07/22

*** UnityEngine.UI를 Import 하고 진행해야 합니다. ***

*** 앞으로 Canvas 위에 존재하는 모든 UI Object (ex : text , image , button)를 **UI 요소**(Unity 공식 문서)라고 부르겠습니다. ***

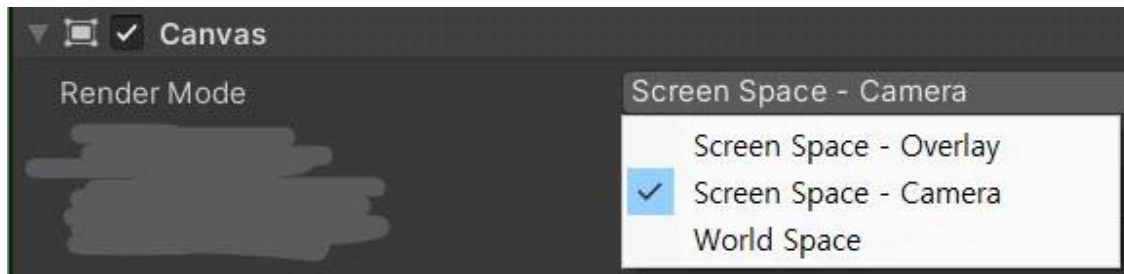
1. Canvas

1) 정의 및 특징

= UI 요소를 그리기 위한 플랫폼(도화지)

- **Canvas 위에 모든 UI 요소**가 존재하며 , 이들은 반드시 Canvas의 자식 GameObject여야 합니다.
- UI 요소 역시 GameObject이며 GameObject의 기능을 대부분 수행할 수 있습니다. (ex : 스크립트 추가)
- Canvas 위에 존재하지 않는 UI 요소는 보이지 않습니다.

2) Canvas(속성) -> Render Mode



① Canvas가 3차원 좌표에 존재하지 않은 경우

<Screen Space - Overlay>

= Canvas 위의 모든 UI 요소를 rendering 할 때 개발자가 구현한 GameObject들을 모두 rendering 하고 나서 rendering 합니다.

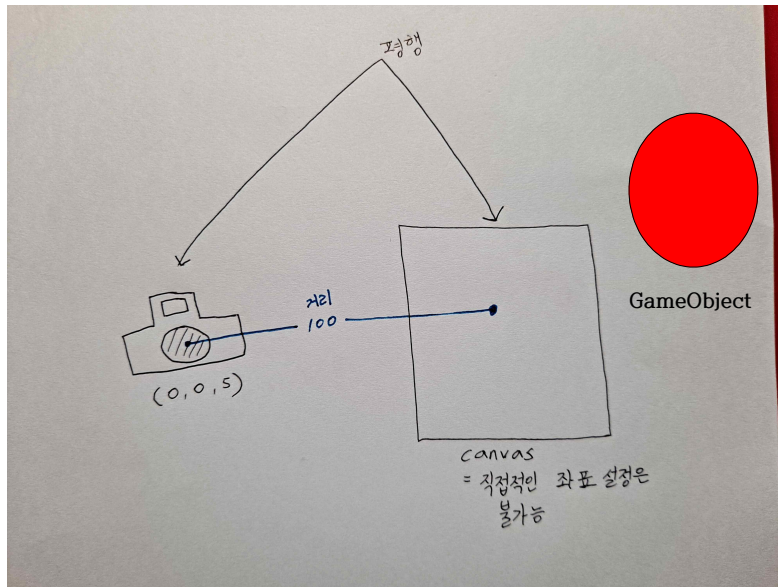
- 가장 늦게 rendering을 하므로 , **언제나 GameObject들 보다 위에 존재해서 Viewer가 모든 부분을 볼 수 있습니다. (UI 요소가 가려지지 않음)**

② Canvas가 3차원 좌표에 존재하는 경우

<Screen Space - Camera : 카메라>

= Canvas가 선택한 카메라에서 일정한 거리만큼 떨어져서 평행하게 앞쪽에 위치합니다.

- 카메라와 Canvas의 거리를 조절하면 그에 따라 Canvas가 GameObject의 앞에도 올 수 있고, 뒤에도 올 수 있습니다.
- 그러므로 Scene에서 GameObject가 Canvas의 UI 요소보다 카메라와 가까워서 UI 요소를 가리거나, GameObject가 Canvas의 UI 요소보다 멀어서 GameObject를 가리는 상황을 개발자가 조절할 수 있게 됩니다.



- Canvas가 카메라와의 평행한 거리만큼 떨어져 존재하므로 좌표계에서 좌표가 존재합니다.
- 하지만 직접적으로 좌표 설정을 할 수는 없습니다.
- Plane Distance : 카메라와 Canvas 사이의 평행한 거리 (ex : 100)

<World Space>

= Canvas를 GameObject로 취급해버립니다.

- Canvas를 일반적인 GameObject처럼 이동, 회전, 기타 작업이 모두 가능해집니다.
- VR에서 유용합니다.

3) Canvas Scaler -> UI Scale Mode



① Constant Pixel Size & Constant Physical Size

= 둘의 차이가 존재하지만, 둘 다 해상도에 대응하지 않고 고정된 UI의 크기를 가집니다.

② Scale With Screen Size

= 해상도에 따라 Canvas의 크기가 알아서 변경되므로, 모든 UI 요소의 크기가 알아서 변경됩니다.

- Canvas는 작은 화면에서는 작아지고, 큰 화면에서는 커집니다.

- Canvas의 크기에 따라 내부의 UI 요소의 크기도 자동으로 변경됩니다.

- 화면의 비율이 (640*360)일 때 해상도가 (1280*720)인 모니터에서는 깨지지 않고 올바르게 변경이 됩니다. 하지만 해상도의 비율이 다른 (1280*900)인 모니터에서는 깨지는 부분이 발생합니다. 이를 해결하기 위해서는 '일치 값'을 개발자가 자율적으로 조정해서 해결해야 합니다.

- UI가 세로로 나열되어 있다면 아래의 그림처럼 Height로 값을 옮겨줍니다.



- Reference : 레트로 2권 p487

4) EventSystem

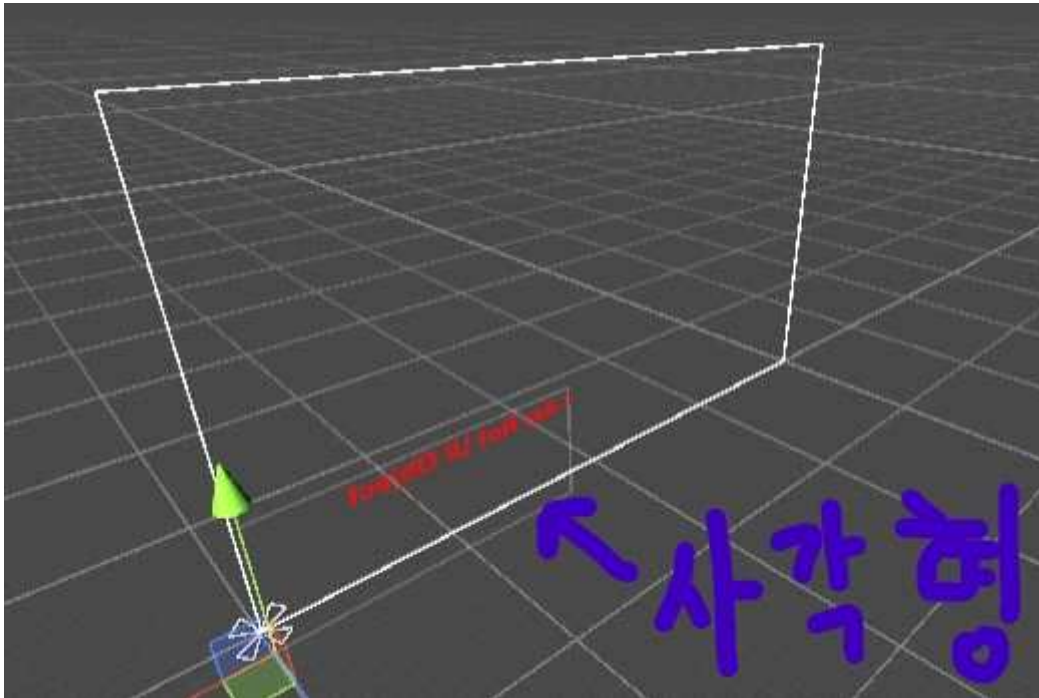
= UI 요소에 클릭이나 터치, 드래그 같은 상호작용을 이벤트 메시지로 전달합니다.

- EventSystem이 존재하지 않는다면 UI 버튼이나 슬라이드 바를 클릭하고 드래그 하는 등의 UI 상호작용을 동작 시킬 수 없습니다.

- EventSystem은 별다른 설정을 요구하지 않고 스스로 동작하기 때문에 개발자가 직접 수정할 일은 거의 없습니다.

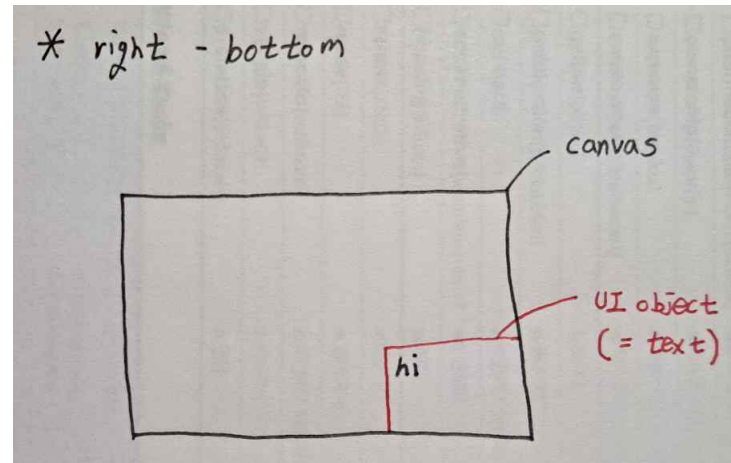
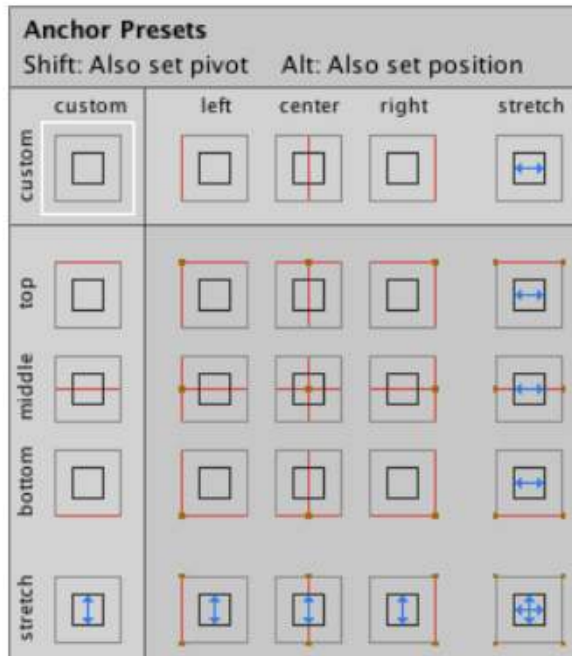
- EventSystem은 GameObject입니다.

2. UI Object의 기본 레이아웃



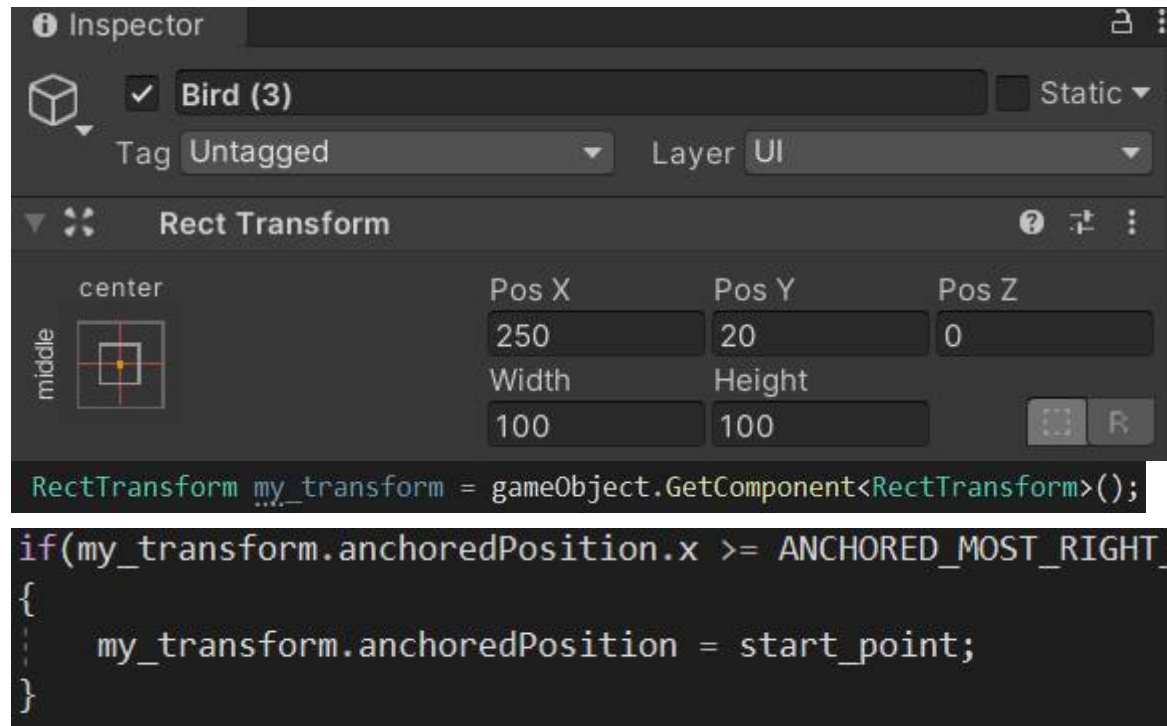
- UI Object의 예시로는 Text , Image , Button , Slider 등이 존재 합니다.
- 가장 많이 실수하는 것이 UI Object는 그림에서 보이는 Text를 담은 사각형입니다. Text(I Am Not UI Object)가 아닙니다.
- Canvas 위에 존재할 UI의 기본적인 레이아웃을 설정하는 탭 입니다.
- Pos : UI Object의 위치
- Width/Height : UI Object의 가로 길이와 세로 길이 이므로 , 이 크기가 작으면 큰 UI Object를 모두 표현할 수 없습니다.
- 다시 한 번 이야기 하지만 , Pos와 Width/Height는 UI Object(그림에서 사각형)를 Canvas 내에서 위치와 크기를 조절하는 것입니다.

<Anchor Preset>



- **Shift + Alt**를 이용하여 선택한 위치로 UI Object가 즉각적으로 이동하도록 합니다.
- UI Object를 해당 위치로 옮기면 Canvas 내부에서 선택된 위치로 이동합니다.
- 두 번째 그림처럼 Right-Bottom을 해도 hi(text)가 오른쪽 아래에 붙지 않는 이유는 UI Object가 오른쪽 아래로 이동하는 것일 뿐 , 그 안의 text는 해당 UI Object의 좌측 상단에서 부터 문자를 작성하기 때문입니다.

<Anchored Position> : 자주 실수하는 개념



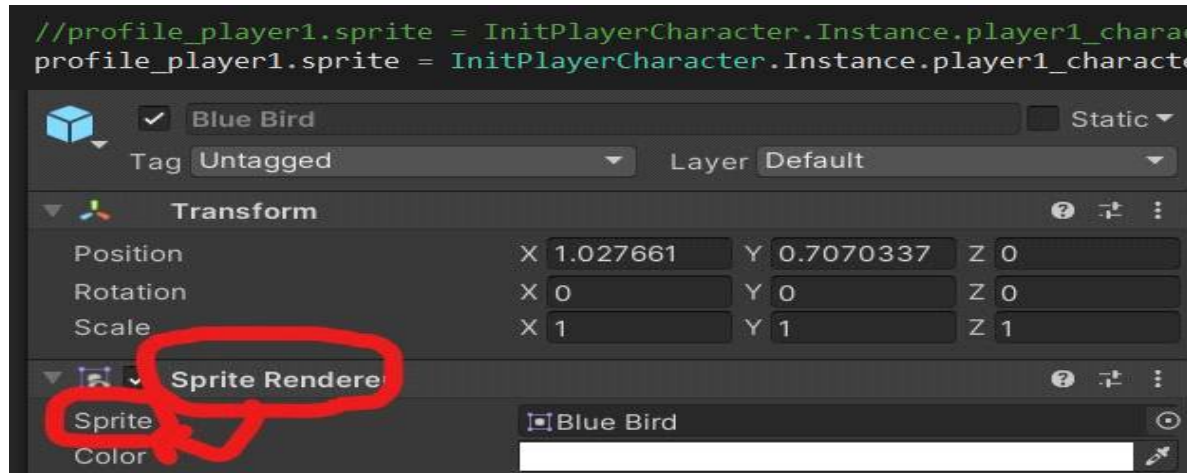
- Canvas에 존재하는 UI 요소에서는 `transform.position` 대신에 `anchoredPosition`을 사용합니다.
- Unity Editor에서 Rect Transform의 (PosX , PosY , PosZ)가 (my_transform.anchorPosition.x , my_transform.anchorPosition.y , my_transform.anchorPosition.z)입니다.
- 만약 UI 요소의 좌표를 `transform.position`으로 사용한다면 이상한 위치에 해당 UI 요소가 표기 됩니다.
- Vector 연산을 할 때 Vector2(2)로 연산을 해야 합니다. Vector3(3)와는 연산이 가능하지 않습니다.

3. Text , Image

- Text 와 Image는 GameObject이며 이들의 속성을 접근할 때 반드시 주의해야하는 사항이 존재합니다. 반드시 컴포넌트 -> 속성으로 속성을 가져와야 합니다.
- Text를 더 잘 보이게 하려면 그림자 효과를 추가해야 합니다. UI -> Effects -> Shadow를 이용해서 그림자를 추가합니다.

<Example : Image의 Sprite 접근하기>

```
//profile_player1.sprite = InitPlayerCharacter.Instance.player1_character.GetComponent<Sprite>();
profile_player1.sprite = InitPlayerCharacter.Instance.player1_character.GetComponent<SpriteRenderer>().sprite;
```



The screenshot shows the Unity Inspector for a GameObject named 'Blue Bird'. The 'Sprite Renderer' component is selected, and the 'Sprite' property is highlighted with a red circle. The 'Sprite' property is set to 'Blue Bird'. The 'Transform' component is also visible above it.

<Component와 속성을 잘 구분합니다.>

- Sprite Renderer Component의 하위 속성인 Sprite를 가져 와야 합니다.
- transform.position 대신에 그냥 position을 사용한 것과 동일한 실수입니다.

- 아래와 같이 Text도 동일한 형식으로 가져와야합니다. Text는 component고 text는 component의 멤버 입니다.

```
private Text battle_datas;
```

```
battle_datas = GameObject.FindGameObjectWithTag("RecordBoard").transform.GetChild(2).gameObject.GetComponent<Text>();
```

4. Button

- 버튼을 눌렀을 때 이벤트 발생(method 호출)이 일어나도록 구현되어 있습니다.
- 하위 Object UI에 Text가 존재합니다.
- UI가 겹쳐있을 때 다른 UI로 인해 버튼을 클릭할 수 없는 상황이 발생합니다. 이때는 다른 UI의 Raycast Target을 체크해제 하여 클릭의 대상으로 지정되지 않도록 합니다.
- 버튼 클릭 시에 호출되는 method는 public으로 지정해야 사용할 수 있습니다.



<버튼 클릭 시에 호출되는 method>

- ① 원하는 method를 담고 있는 Script가 연결된 GameObject를 드래그 합니다. Script를 드래그하면 method를 사용할 수 없습니다.
- ② 해당 script에서 원하는 method 선택

5. UI SetActive

image로 선언 되었으면

```
imgHpBar.gameObject.SetActive(true);
```

`imgHpBar.gameObject.SetActive(false);` 이런식으로 접근해서 활성화 비활성화 시켜보세요.

<Sound> 21/05/31

*** 각각의 Reference는 Unity Docs를 참고합니다. ***

*** Audio Clip은 Unity Asset Store에서 다운로드 합니다. ***

1. Audio Clip = 음악 파일

= Audio Source가 사용하는 오디오 데이터

- MP3 , PCM , Vorbis 형식을 선호합니다.

2. Audio Source = MP3 기기(H/W)

= Scene에서 Audio Clip을 재생합니다.

- Listener가 하나 이상의 리버브 존에 있을 경우 Audio Source에 잔향이 적용됩니다.

3. Audio Listener = 마이크

= Scene에서 주어진 Audio Source로 부터의 입력을 수신하여 컴퓨터 스피커를 통해 사운드를 재생합니다.

- Audio Listener를 메인 카메라 또는 플레이어를 나타내는 게임 오브젝트 중 하나에 추가해야 합니다. 둘 다 시도해 본 후 게임에 가장 잘 맞는 것을 골라야 합니다. 보통 메인 카메라에 Audio Listener를 추가하는 것이 가장 일반적입니다.

- Default로 Audio Listener는 메인 카메라에 컴포넌트로 존재합니다.

- 각 Scene에서 제대로 작동할 수 있는 Audio Listener는 1개 뿐입니다.

4. 여러 소리 넣기 : Empty GameObject에 여러 개의 Audio Source 넣기

- Reference : <https://midason.tistory.com/107>

하나의 AudioSource를 사용하면
게임을 하기 싫어질 만큼 귀가 괴로워진다.

보통은 하나의 오브젝트에 하나의 AudioSource를 가지고 있다.
몬스터는 몬스터의 음악이 있고
배경음도 지역에 따라 clip을 바꾸어 들어 주게 된다.

하지만 하나의 오브젝트에서
동시에 들려주어야 하는 것이 있다면 어떻게 할 것인가?

쉽게 예를 들어 설명하면
캐릭터가 있다.
점프를 할 때 점프 음을 튼다.
펀치를 날리면 타격음이 나와야 한다.
도중에 맞으면 잡음정도 하나 생겨야 한다.

이렇게 하나의 FBX로 만들어진 오브젝트의 경우
상태 값이 많아 사용될 AudioSource의 양도 많아 질텐데
하나의 AudioSource로는 감당이 안되며
동시에 처리가 힘들다.

그럼 어떻게 해야 되겠는가?

쉽게 AudioSource를 여러개 만드는 것이다.

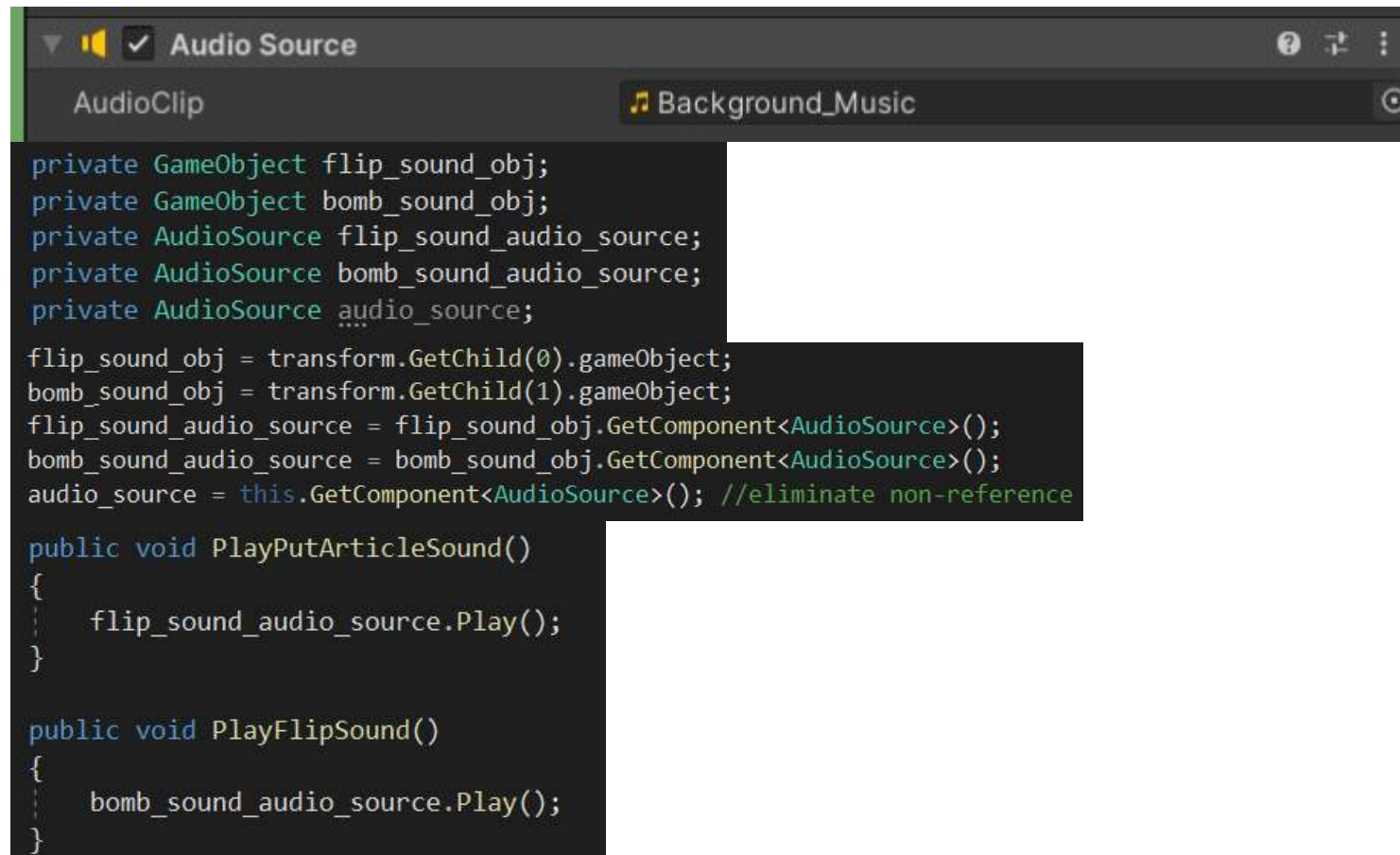
어디에 만드느냐?

오브젝트에 Create Empty를 하여 자식으로 더미를 만든다.
FBX로 불러온 것이라면 본의 위치를 잡는다.
점프음은 다리 부분에 더미를 만들어 AudioSource를 넣는다.
펀치는 주먹에 더미를 만들고 AudioSource를 추가한다.
이렇게 자식으로 더미를 만들어 AudioSource를 각각 추가한다.
그리고 캐릭터 컨트롤 스크립트에서
public으로 전역변수 AudioSource를
각각 변수에 할당 하여 컨트롤 하거나
child를 찾아 컨트롤 할 수 있다.
그러면 여러가지의 음원이 씹히지 않고
깔끔하게 따로따로 나올 수 있다.

AudioSource 를 설계 할 때에는
하나의 AudioSource에서 clip을 바꾸어 가며 컨트롤 할지
더미로 분할하여 다중으로 AudioSource를 생성해 컨트롤 할지
고려해봐야 함을 기억하자.



<Example : Audio Source 관련 Script>



- 코드에서 AudioClip을 변경하고 싶으면 오디오 소스.clip에서 변경
- Audio Source Component에 AudioClip 속성을 추가합니다.
- Audio Source는 Component이고 , 그 Component에는 여러 가지 method가 포함되어 있고 , 그 중 Play()를 실행시키는 스크립트 입니다.

<Scene> 20/08/25

1. Scene

- Scene은 Hierarchy에서 관리하는 것이 아닌 Assets폴더의 Scenes에서 관리합니다.
- Prefab, Script는 다른 Scene에서도 사용할 수 있습니다.
- 하나의 프로젝트에는 여러 개의 Scene이 있고, 하나의 Scene에는 여러 개의 GameObject가 존재합니다.
- Scene안에 Scene을 추가할 수 있습니다. 이것 Additive Loading이라고 합니다.
- Build Setting에서 0번 Index의 Scene이 게임 시작 시에 실행됩니다.

2. SceneManager로 Scene 관리하기

1) SceneManager

SceneManager

class in UnityEngine.SceneManagement

Static Methods

CreateScene	Create an empty new Scene at runtime with the given name.
GetActiveScene	Gets the currently active Scene.
GetSceneAt	Get the Scene at index in the SceneManager's list of loaded Scenes.
GetSceneByBuildIndex	Get a Scene struct from a build index.
GetSceneByName	Searches through the Scenes loaded for a Scene with the given name.
GetSceneByPath	Searches all Scenes loaded for a Scene that has the given asset path.
LoadScene	Loads the Scene by its name or index in Build Settings.
LoadSceneAsync	Loads the Scene asynchronously in the background.
MergeScenes	This will merge the source Scene into the destinationScene.
MoveGameObjectToScene	Move a GameObject from its current Scene to a new Scene.
SetActiveScene	Set the Scene to be active.
UnloadSceneAsync	Destroys all GameObjects associated with the given Scene and removes the Scene from the SceneManager.

- SceneManager는 개발자가 직접 구현하는 클래스(Script)가 아니라 UnityEngine에 존재하는 클래스입니다.

```
- using UnityEngine.SceneManagement;
```

사용하기 위해 반드시 using을 통해 SceneManager를 가져옵니다.

2) LoadScene

```
SceneManager.LoadScene("StartGame");
```

- Scene이 로드 될 때는 이전의 모든 Game Object가 Destroy 됩니다.
- Destroy를 막고 싶다면 DontDestroyOnLoad 함수를 이용하여 정의해야 합니다.

3) UnloadScene

SceneManager.UnloadSceneAsync

Declaration

```
public static AsyncOperation UnloadSceneAsync(int sceneBuildIndex);
```

Declaration

```
public static AsyncOperation UnloadSceneAsync(string sceneName);
```

Declaration

```
public static AsyncOperation UnloadSceneAsync(SceneManagement.Scene scene);
```

Declaration

```
public static AsyncOperation UnloadSceneAsync(int sceneBuildIndex, SceneManagement.UnloadSceneOptions options);
```

Declaration

```
public static AsyncOperation UnloadSceneAsync(string sceneName, SceneManagement.UnloadSceneOptions options);
```

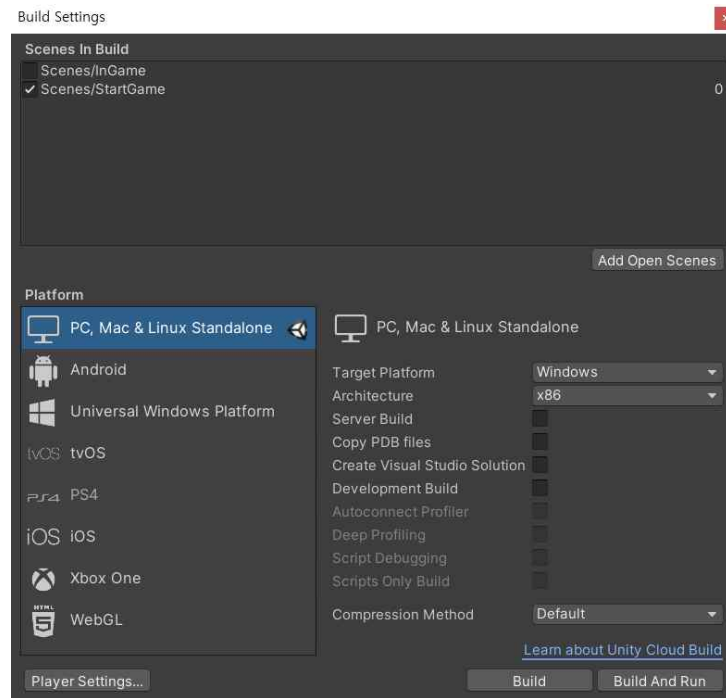
Declaration

```
public static AsyncOperation UnloadSceneAsync(SceneManagement.Scene scene, SceneManagement.UnloadSceneOptions options);
```

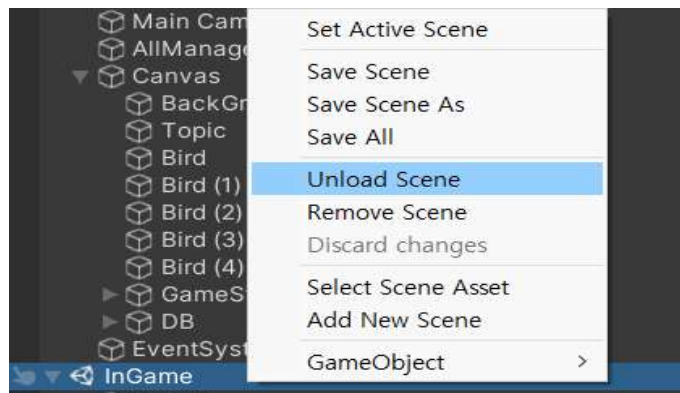
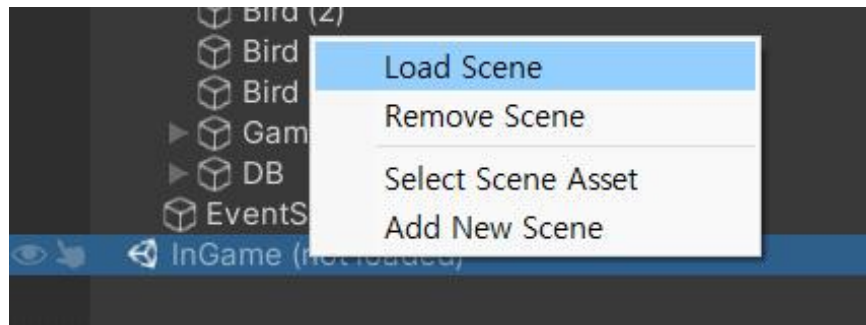
- 다른 Scene이 Load되면 알아서 사라지는데 필요한가? 나중에 이를 깨달으면 정리하자

3. Scene 관련 작업

1) File -> Build Settings



2) Unity Editor에서의 Load Scene &Unload Scene



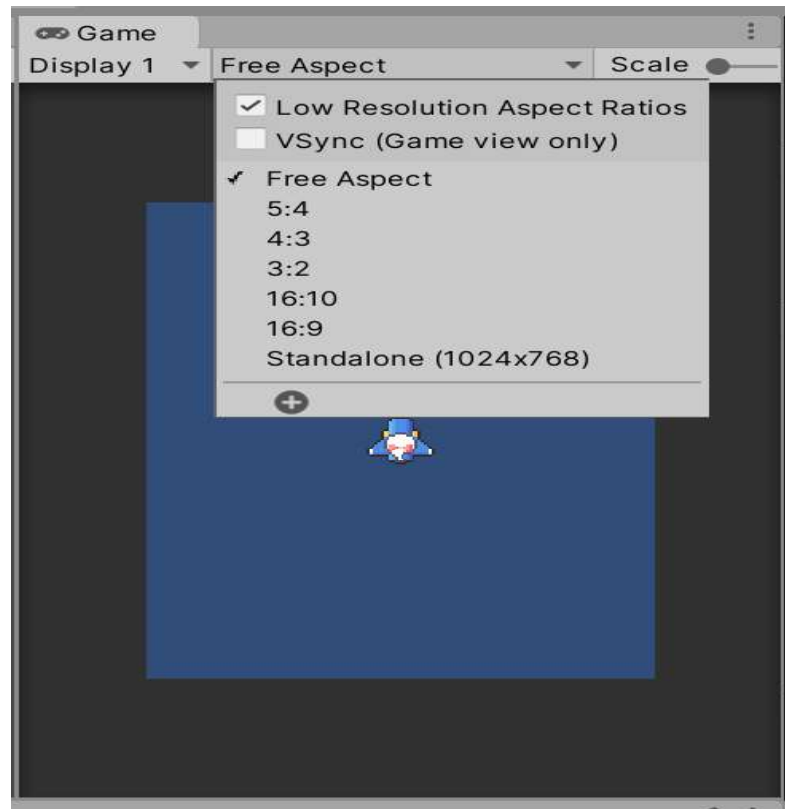
<슈팅 게임 만들기 1> 20/07/10

1. transform을 이용해서 물체 움직이기.

- transform.position을 vector3를 사용하여 위치를 변화시킵니다.
- 반드시 Time.deltaTime을 붙여야 합니다.

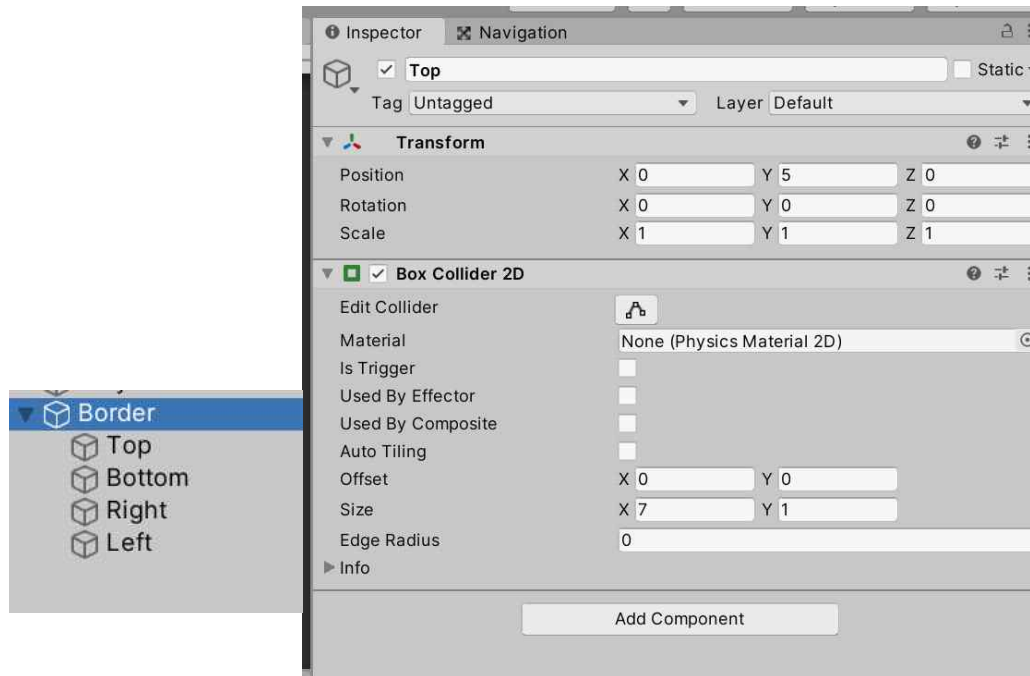
```
void Update()
{
    /*
     이번에는 플레이어 이동을 transform으로 이용합니다.
    */
    //입력
    float h = Input.GetAxisRaw("Horizontal");
    float v = Input.GetAxisRaw("Vertical");
    //플레이어 현재 위치 -> transform 이므로 vector3값이다.
    Vector3 cur_pos = transform.position;
    //플레이어 나중 위치
    Vector3 next_pos = new Vector3(h, v, 0) * speed * Time.deltaTime;
    //플레이어의 위치를 갱신합니다.
    transform.position = cur_pos + next_pos;
```

2. 해상도 조절



3. 플레이어의 움직임을 제한하기

1) GameObject 4개를 묶어서 1개의 GameObject 생성.



2) Trigger로 검사하기

```
//Trigger를 이용한다.  
//플레이어가 경계에 닿았을 때  
void OnTriggerEnter2D(Collider2D collision)  
{  
    //충돌발생한 오브젝트가 "Border"일때  
    //Border중 4가지 방향의 조건문 형성  
    if(collision.gameObject.tag == "Border")  
    {  
        if(collision.gameObject.name == "Top")  
        {  
            touch_top = true;  
        }  
        else if(collision.gameObject.name == "Bottom")  
        {  
            touch_bottom = true;  
        }  
        else if (collision.gameObject.name == "Right")  
        {  
            touch_right = true;  
        }  
        else  
        {  
            touch_left = true;  
        }  
    }  
}
```

```
//경계에 닿고 상황이 끝났을 때  
private void OnTriggerExit2D(Collider2D collision)  
{  
    //충돌발생한 오브젝트가 "Border"일때  
    //Border중 4가지 방향의 조건문 형성  
    if (collision.gameObject.tag == "Border")  
    {  
        if (collision.gameObject.name == "Top")  
        {  
            touch_top = false;  
        }  
        else if (collision.gameObject.name == "Bottom")  
        {  
            touch_bottom = false;  
        }  
        else if (collision.gameObject.name == "Right")  
        {  
            touch_right = false;  
        }  
        else  
        {  
            touch_left = false;  
        }  
    }  
}
```

- 트리거를 이용합니다. 플레이어 스크립트이므로, 플레이어와 다른 게임오브젝트가 충돌한다면 자동으로 감지하여 조건에 따라 이벤트를 발생시킵니다. 여기서는 Border랑만 부딪히는 조건만 존재하지만 추후에는 다른 GameObject와 충돌하는 조건이 많이 추가됩니다.

```

/*
 이번에는 플레이어 이동을 transform으로 이용합니다.
 */
//입력
//왼쪽화살표를 누르면 -1을 , 오른쪽 화살표를 누르면 1을 입력받습니다.
//그러므로 h에는 -1 or 1이 저장됩니다.
float h = Input.GetAxisRaw("Horizontal");
if((touch_right && h == 1) || (touch_left && h == -1))
{
    h = 0;
}
//아래쪽화살표를 누르면 -1을 , 위쪽 화살표를 누르면 1을 입력받습니다.
//그러므로 v에는 -1 or 1이 저장됩니다.
float v = Input.GetAxisRaw("Vertical");
if ((touch_top && v == 1) || (touch_bottom && v == -1))
{
    v = 0;
}
//플레이어 현재 위치 -> transform 이므로 vector3값이다.
Vector3 cur_pos = transform.position;
//플레이어 나중 위치
Vector3 next_pos = new Vector3(h, v, 0) * speed * Time.deltaTime;
//플레이어의 위치를 갱신합니다.
transform.position = cur_pos + next_pos;

```

4. 총알 삭제(Destroy)

```
참조 0개
private void OnTriggerEnter2D(Collider2D collision)
{
    //사라지는 경계에 닿는다면
    if(collision.gameObject.tag == "BorderBullet")
    {
        //게임 오브젝트 삭제
        Destroy(gameObject);
    }
}
```

<슈팅 게임 만들기 2> 20/07/13 ~ 07/20

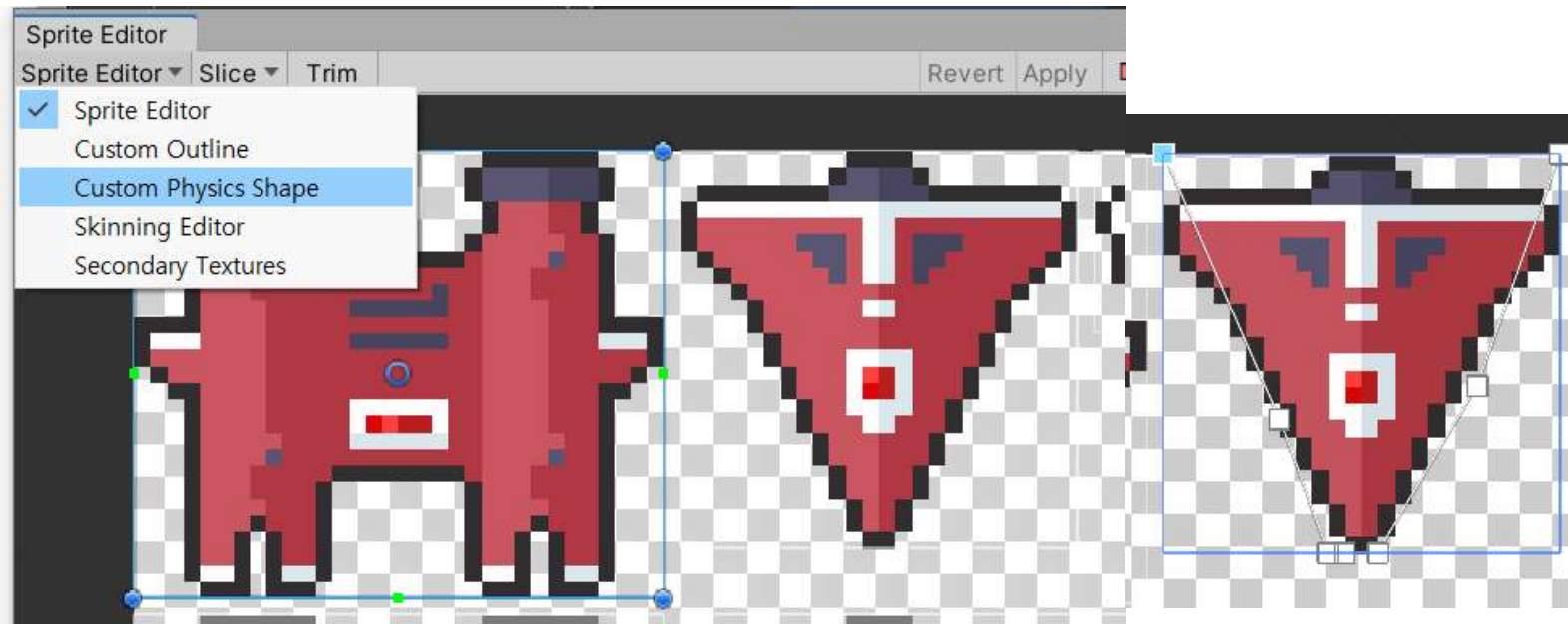
1. 파워 별로 총알 변경하기

```
//파워
//1단계 파워 : 1발 나감
if(power == 1)
{
    //인자 : (프리랩 , 생성될 위치(벡터) , 생성됐을 때 방향
    GameObject bullet = Instantiate(bullet_a, transform.position, transform.rotation);
    //bullet의 rigidbody를 가져옵니다.
    Rigidbody2D rigid = bullet.GetComponent<Rigidbody2D>();
    //위로 속도를 줍니다.
    rigid.AddForce(Vector2.up * speed, ForceMode2D.Impulse);
}

//2단계 파워 : 2발 나감
else if(power == 2)
{
    GameObject bullet_left = Instantiate(bullet_a, transform.position + Vector3.left*0.1f, transform.rotation);
    GameObject bullet_right = Instantiate(bullet_a, transform.position + Vector3.right*0.1f, transform.rotation);
    Rigidbody2D rigid_left = bullet_left.GetComponent<Rigidbody2D>();
    Rigidbody2D rigid_right = bullet_right.GetComponent<Rigidbody2D>();
    //위로 속도를 줍니다.
    rigid_left.AddForce(Vector2.up * speed, ForceMode2D.Impulse);
    rigid_right.AddForce(Vector2.up * speed, ForceMode2D.Impulse);
}

//3단계 이상 파워
else
{
    GameObject bullet_big = Instantiate(bullet_b, transform.position, transform.rotation);
    //bullet의 rigidbody를 가져옵니다.
    Rigidbody2D rigid_big = bullet_big.GetComponent<Rigidbody2D>();
    //위로 속도를 줍니다.
    rigid_big.AddForce(Vector2.up * speed, ForceMode2D.Impulse);
}
```

2. 사용자 지정 collider



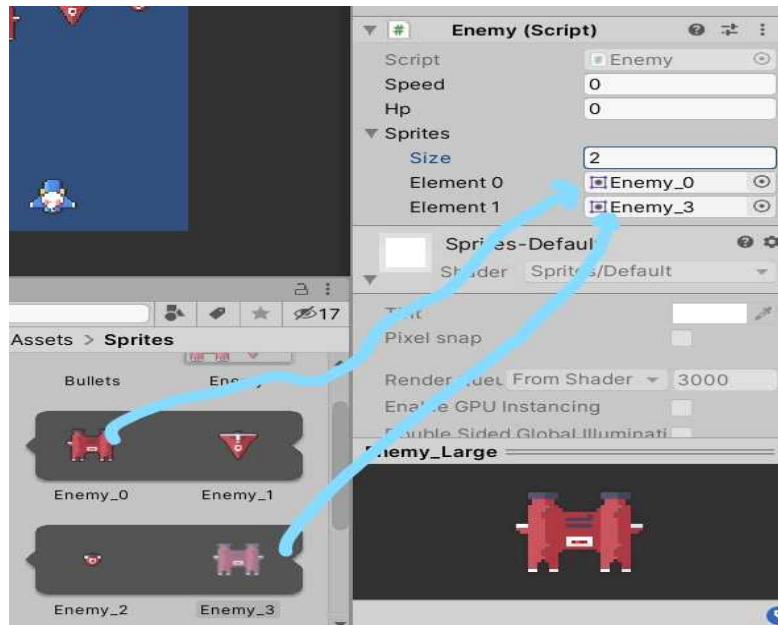
- = sprite의 inspector에 있는 sprite editor에 가서 custom physics shape로 바꿉니다. 그리고 선들을 변경해주면 collider가 지정됩니다.
- = 이렇게 바꾸고 **box collider**가 아닌 **polygon collider**를 적용해주면 제가 만든 collider가 적용 됩니다.

3. 적 비행체 만들기

1) public

= **public**은 다른 클래스에서도 해당 변수나 method를 사용할 수 있도록 해주는 지정자입니다. 실제로 적 비행체에서 총알의 데미지가 인자로 필요한데 이때 **public** 변수인 총알의 데미지를 가져와서 사용합니다.

2) public 변수와 배열을 Unity 에서 초기화



```
public class Enemy : MonoBehaviour
{
    public float speed;
    public int hp;
    public Sprite[] sprites;
    SpriteRenderer spriterenderer;
    Rigidbody2D rigid;
}
```

보통 C#에서 클래스 멤버 변수를 만들고 초기화를 하지 않고 **Unity 내부 inspector에서 초기화**를 합니다.

3) enemy끼리 충돌하지 않기 위해 is_trigger를 체크해줍니다.

4) collision.gameobject

= collision.gameobject 는 gameobject와 부딪힌 gameobject입니다.

- 다시 말해 player 스크립트의 gameobject인 player와 벽이 부딪혔다면 collision.gameobject는 벽이 됩니다.

5) Script (충돌 이벤트)

```
//충알에 맞으면
void OnHit(int dmg)
{
    //피 감소
    hp -= dmg;
    Debug.Log(hp);
    //흰색 이미지로 변경
    spriterenderer.sprite = sprites[1];
    //다시 원래 이미지로 변경
    Invoke("ReturnSprite" , 0.2f);

    //죽음
    if(hp <= 0)
    {
        Debug.Log("enemy 격추!");
        Destroy(gameObject);
    }
}

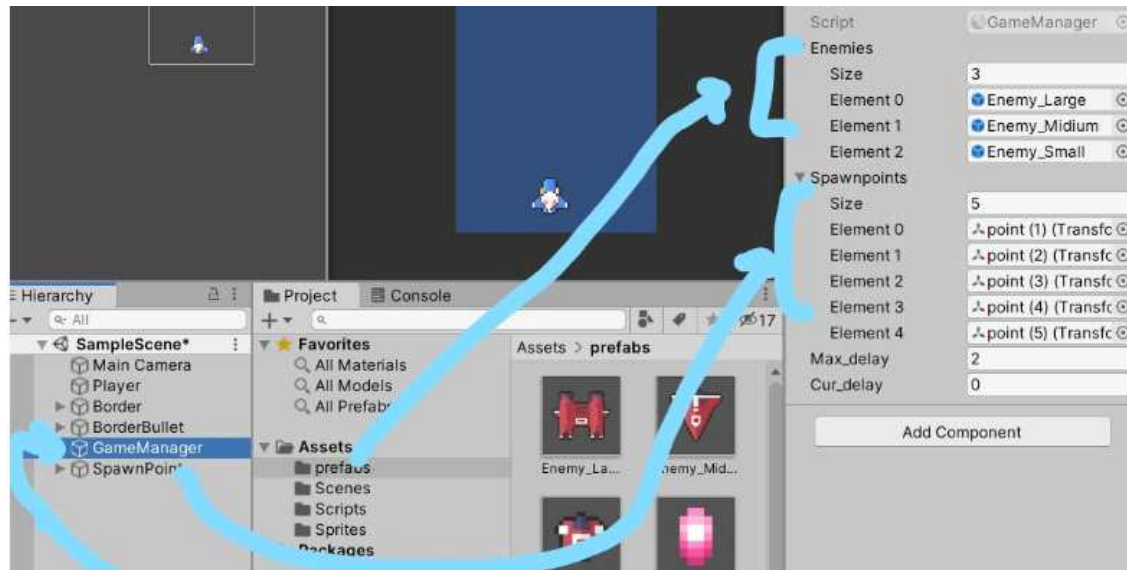
//다시 원래 이미지로 변경해주는 메소드
void ReturnSprite()
{
    spriterenderer.sprite = sprites[0];
}

//트리거로 충돌이벤트
void OnTriggerEnter2D(Collider2D collision)
{
    //1. 맵밖으로 이동하면 죽음
    if (collision.gameObject.tag == "BorderBullet")
    {
        Destroy(gameObject);
    }
    //2. 총알에 맞으면
    else if(collision.gameObject.tag == "PlayerBullet")
    {
        //Bullet 클래스를 가져 옵니다.
        //enemy가 gameobject이므로 이와 부딪힌 것을 collision.gameobject로 합니다
        //그것에 GetComponent를 하여 bullet에 저장합니다.
        Bullet bullet = collision.gameObject.GetComponent<Bullet>();
        OnHit(bullet.damage);
        //총알 없애기
        Destroy(collision.gameObject);
    }
}
```

- 트리거를 이용해서 해당 무언가 부딪힌다면 바로 호출되도록 합니다. 적 비행체가 총알에 맞았을 때 일어나는 현상들을 구현합니다.

4. 적 비행체(prefab)를 외부에서 나오도록 하기

1) GameManager라는 GameObject를 하나 만듭니다. (다른 클래스를 가장 많이 이용하는 GameObject 입니다.)



2) Script

```
//적 비행체 소환
void SpawnEnemy()
{
    int enemy_index = Random.Range(0, 3);
    int enemy_spawn_point = Random.Range(0, 5);
    Instantiate(enemies[enemy_index], spawnpoints[enemy_spawn_point].position,
        spawnpoints[enemy_spawn_point].rotation);
}
```

```
//적 비행체 배열
public GameObject[] enemies;
//적 비행체 소환 위치
public Transform[] spawnpoints;
//1초에 1번 소환하기 위해서 시간 생성
public float max_delay;
public float cur_delay;

private void Update()
{
    cur_delay += Time.deltaTime;
    if(cur_delay >= max_delay)
    {
        //적 비행체 소환 , max_delay를 랜덤값으로 하여 게임을 더 재밌게합시다
        SpawnEnemy();
        //(시작값 , 끝값) 사이에서 랜덤값으로 나옵니다.
        max_delay = Random.Range(0.5f, 3f);
        cur_delay = 0;
    }
}
```

- Unity에서 연결해 줄 오브젝트들을 미리 public 변수로 설정해 줍니다.

- 비행체와 비행체 소환위치를 모두 랜덤으로 해줍니다. 이때 Random 클래스를 이용합니다.

- 실수로 하고 싶다면 숫자를 실수인 f를 이용하면 됩니다.

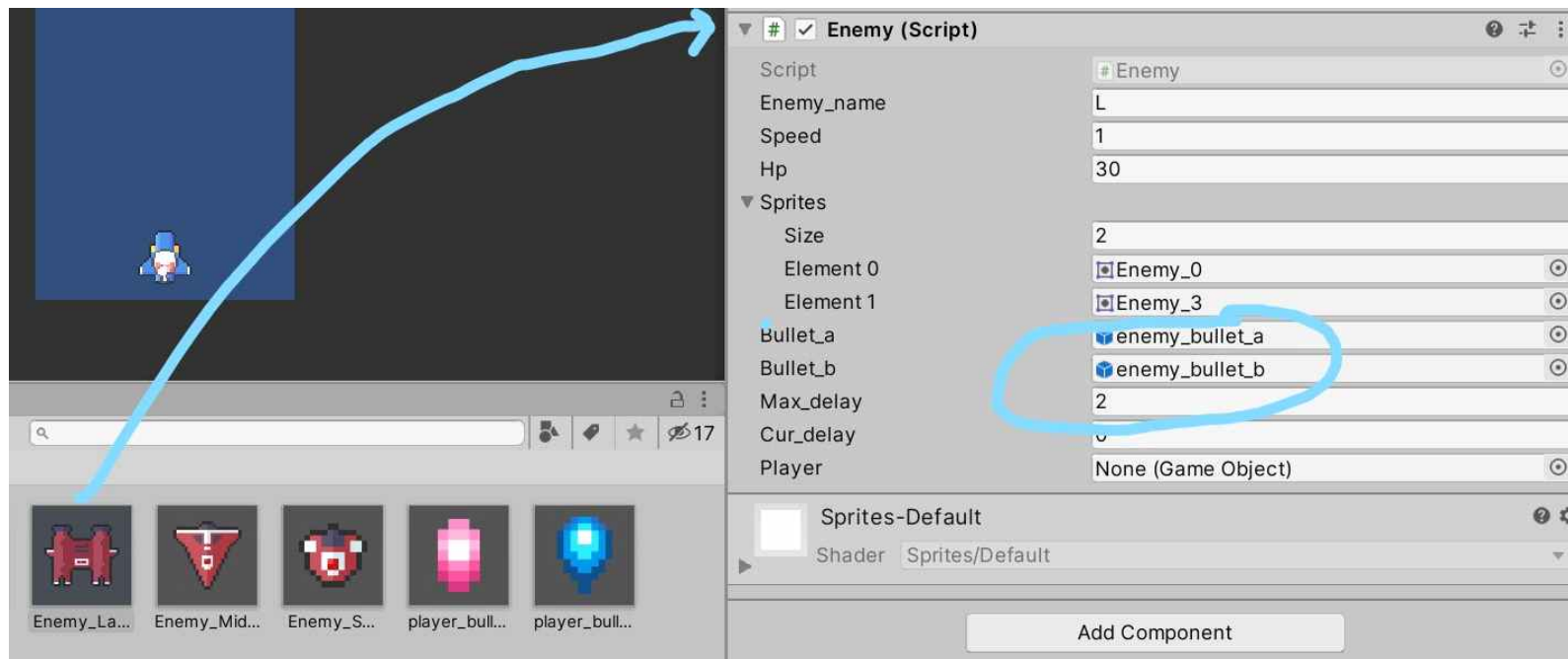
5. Enemy가 옆에서 등장하기

```
//생성된 걸 gameobject에 연결
GameObject enemy = Instantiate(enemies[enemy_index], spawnpoints[enemy_spawn_point].position,
    spawnpoints[enemy_spawn_point].rotation);
Rigidbody2D rigid = enemy.GetComponent<Rigidbody2D>();
//Enemy 클래스전체를 가져옵니다.
Enemy enemy_logic = enemy.GetComponent<Enemy>();
//vector의 y축을 -1로 하여 아래로 내려오게 합니다.
//우측 스폰
if(enemy_spawn_point == 5 || enemy_spawn_point == 6)
{
    //적 기체 방향 돌리기
    enemy.transform.Rotate(Vector3.back * 90);
    rigid.velocity = new Vector2(enemy_logic.speed * -1 , -1);
}
//좌측 스폰
else if (enemy_spawn_point == 7 || enemy_spawn_point == 8)
{
    //적 기체 방향 돌리기
    enemy.transform.Rotate(Vector3.forward * 90);
    rigid.velocity = new Vector2(enemy_logic.speed , -1);
}
//나머지 스폰
else
{
    rigid.velocity = new Vector2(0 , enemy_logic.speed * -1);
}
```

- Instantiate로 Prefab을 만듭니다.
- Prefab의 component 들을 가져옵니다.
- 조건문에 따라 Spawn 되도록 합니다.

6. 적이 총알 쏘기

- 적 총알은 기존에 만들어 놓은 player_bullet을 복사하고 수정하면 더 쉽게 만들 수 있습니다.
- 복사하는 것이므로 바꿀 것은 바뀌야 합니다. 하지만 player_bullet과는 로직은 같으므로 script는 동일하게 사용합니다.
- 기본적으로 적 또한 발사하고 재장전을 해야 하므로 Fire()와 Reload()를 만들어줍니다.
- Reload()는 이전과 로직이 같지만 Fire()는 내가 키를 누르지 않고 자동으로 발사 되어야 하므로 로직을 바꾸어 줍니다.
- enemy가 "L"(총열이 2개인 큰 적 기체)는 Vector 값을 조금 변경하여 두 군데에서 총알을 발사 하도록 합니다.
- 새롭게 클래스 변수를 많이 추가 하였으므로 모두 unity에서 다시 초기화를 해줍니다.



```

public void Fire()
{
    //창전 딜레이
    if (cur_delay < max_delay)
    {
        return;
    }

    //작은 총알 발사
    if(enemy_name == "S")
    {
        GameObject bullet = Instantiate(bullet_a, transform.position, transform.rotation);
        //bullet의 rigidbody를 가져옵니다.
        Rigidbody2D rigid = bullet.GetComponent<Rigidbody2D>();
        //플레이어 방향으로 쏘아합니다
        //어려운 부분입니다.
        //player.transform.position은 enemy가 spawn되었을 때 player의 위치입니다. 이걸 player에 연결하는 부분을 잘 이해하지 못하였습니다.
        //transform.position은 현재 스크립트와 연결된 gameobject인 enemy의 위치입니다.
        Vector3 to_player = player.transform.position - transform.position;
        rigid.AddForce(to_player.normalized * 3, ForceMode2D.Impulse);
    }
    else if(enemy_name == "L")
    {
        //위의 코드 복사
        //애는 두발 쏜다
        //오른쪽 총알
        GameObject bullet_r = Instantiate(bullet_b, transform.position + Vector3.right*0.3f, transform.rotation);
        Rigidbody2D rigid_r = bullet_r.GetComponent<Rigidbody2D>();
        Vector3 to_player_r = player.transform.position - (transform.position + Vector3.right*0.3f);
        // .normalized는 단위벡터로 만들어줍니다.
        rigid_r.AddForce(to_player_r.normalized * 3, ForceMode2D.Impulse);

        //왼쪽 총알
        GameObject bullet_l = Instantiate(bullet_b, transform.position + Vector3.left * 0.3f, transform.rotation);
        Rigidbody2D rigid_l = bullet_l.GetComponent<Rigidbody2D>();
        Vector3 to_player_l = player.transform.position - (transform.position + Vector3.left*0.3f);
        rigid_l.AddForce(to_player_l.normalized * 3, ForceMode2D.Impulse);
    }
    //중간 적은 총을 쏘지 않는다.
}

```

7. 총알 맞았을 때 피격 이벤트

```
//플레이어가 죽었으니까 리스폰하기  
//Invoke를 위해 실행 함수를 만듭니다.  
public void RespawnPlayer()  
{  
    Invoke("RespawnPlayerExe", 2f);  
}  
  
public void RespawnPlayerExe()  
{  
    player.transform.position = Vector3.down * 4.23f;  
    player.SetActive(true);  
}
```



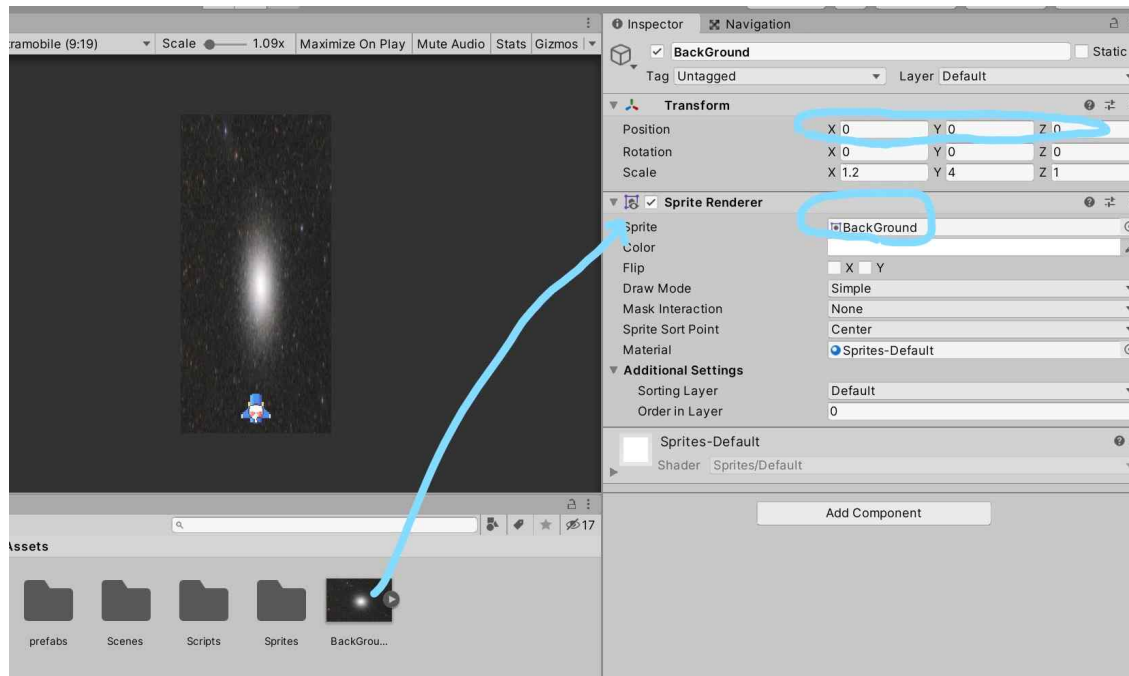
- 애는 게임 매니저에서 만들어 준 method입니다. Unity에서는 체크박스가 SetActive()를 담당합니다.

```
//게임 매니저  
public GameManager manager;
```

```
//플레이어가 적 기체나 총알에 맞았을 때  
//GameManager가 관리하는 함수를 이용합니다.  
else if (collision.gameObject.tag == "Enemy" || collision.gameObject.tag == "EnemyBullet")  
{  
    //1. 플레이어가 죽었으니 GameManager클래스에서 2초뒤에 리스폰하도록 만든 메소드를 호출합니다.  
    //다른 클래스의 메소드를 이용하므로 맨위에서 선언해주었습니다.  
    manager.RespawnPlayer();  
    //2. 플레이어 비활성화 = 게임에서 죽었으므로 사라집니다.  
    gameObject.SetActive(false);  
    Debug.Log("플레이어가 죽었습니다." + life);  
}
```

- GameManager클래스의 method를 이용해야 하므로 객체를 만들고 함수를 호출합니다.

8. 배경 넣기

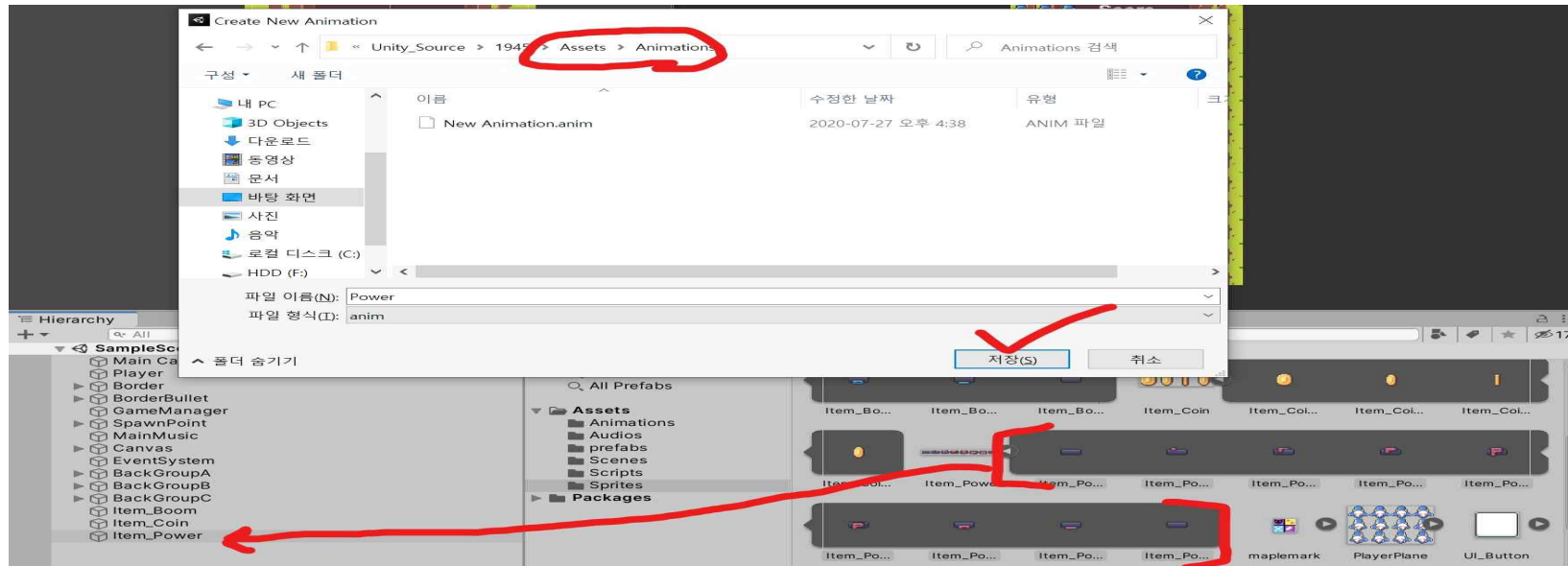


- 1) 이미지를 assets 폴더에 저장합니다.
- 2) transform의 position을 모두 0 0 0으로 하고 scale을 조정합니다.
- 3) sprite에 저장한 이미지를 넣어줍니다.
- 4) Order in layer를 게임 기물들 보다 작게 해줍니다.

<슈팅 게임 만들기 3> 20/07/27

1. Default를 이용한 Animation

- sprite들을 여러 개 묶어서 드래그하기만 하면 애니메이션이 재생됩니다.



2. 아이템 먹기

1) 점수 로직

플레이어가 아이템과의 충돌이 이루어질 때 발생 합니다. (Player 클래스)
충돌하면 해당 이벤트를 발생 시키고 바로 제거 시킵니다.

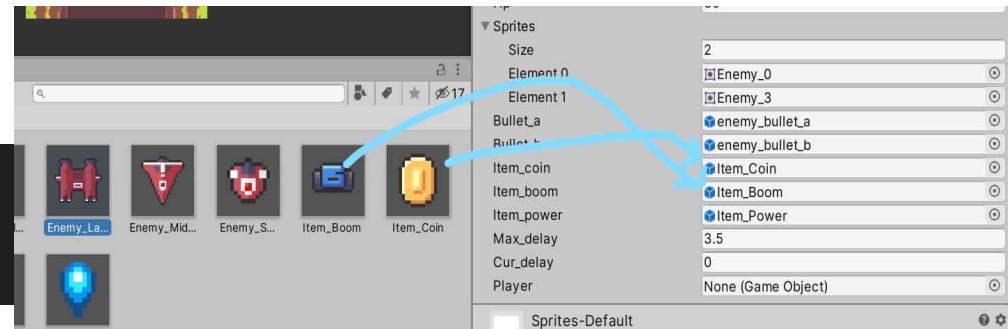
```
//아이템 먹었을 때
else if(collision.gameObject.tag == "Item")
{
    Item item = collision.gameObject.GetComponent<Item>();
    if(item.type == "Coin")
    {
        score += 1000;
        Destroy(collision.gameObject);
    }
    else if(item.type == "Power")
    {
        if (power != max_power)
        {
            power++;
            Destroy(collision.gameObject);
        }
        else
        {
            score += 3000;
        }
    }
    Destroy(collision.gameObject);
}
//아직 구현x
else if(item.type == "Boom")
{
    Destroy(collision.gameObject);
}
```

2) 아이템 스폰 로직

- 적 기체가 죽을 때 확률 기반으로 생성됩니다.(Enemy 클래스)
- prefab에서는 다른 prefab을 넣을 수 있습니다.

<스크립트>

```
//아이템  
public GameObject item_coin;  
public GameObject item_boom;  
public GameObject item_power;
```



```

int ran = Random.Range(0, 15);
if(ran < 10)
{
    Debug.Log(ran + " No Drop");
}
//coin
else if(ran < 13)
{
    Instantiate(item_coin, transform.position, item_coin.transform.rotation);
    Debug.Log(ran);
}
//power
else if(ran < 14)
{
    Instantiate(item_power, transform.position, item_power.transform.rotation);
    Debug.Log(ran);
}
//bomb
else if(ran < 15)
{
    Instantiate(item_boom, transform.position, item_boom.transform.rotation);
    Debug.Log(ran);
}

```

```

//총알에 맞으면
void OnHit(int dmg)
{
    //Player가 죽으면 SetActive를 2초간 끄기 때문에 Find를 하면 없는 게임 오브젝트를 찾는 것이므로 예러가 납니다.
    if (GameObject.Find("Player") == true)
    {
        hp -= GameObject.Find("Player").GetComponent<Player>().power;
    }
    Debug.Log(hp);
}

```

- 확률을 이용하여 아이템을 조건별로 생성합니다.

- 적 기체가 총알에 맞을 때 Player 스크립트의 power변수를 가져와서 hp에서 빼줍니다.

3. 필살기 만들기

```
//폭탄을 먹었을 때
else if (item.type == "Boom")
{
    if (boom_cnt == max_boom_cnt)
    {
        score += 3000;
    }
    else
    {
        boom_cnt++;
    }
    Destroy(collision.gameObject);
}
```

- 폭탄이 최대치를 넘으면 점수.
- 넘지 않으면 폭탄 장전.
- 언제나 이벤트 이후 폭탄 오브젝트는 삭제.

```
//폭탄 쏘기
public void Boom()
{
    if (Input.GetKeyDown(KeyCode.LeftShift) == false)
    {
        return;
    }

    //지금 폭탄을 쏘고 있지 않을 때만 발사
    if(is_boom_time == true)
    {
        return;
    }

    //폭탄 개수 0
    if(boom_cnt == 0)
    {
        return;
    }

    //1. 폭탄효과 발생 , 폭탄 개수 감소 , Boom_UI_update , Invoke로 5초뒤에 끄기
    boom_cnt--;
    boom_effect.SetActive(true);
    manager.UpdateBoom(boom_cnt);
    is_boom_time = true;
    Invoke("OffBoomEffect", 5f);
}
```

```
//2. "Enemy" 태그인 게임 오브젝트를 찾습니다.
//적 기체 죽사
GameObject[] enemies = GameObject.FindGameObjectsWithTag("Enemy");
for (int i = 0; i < enemies.Length; i++)
{
    //해당 게임 오브젝트의 스크립트를 가져옵니다.
    Enemy enemy_logic = enemies[i].GetComponent<Enemy>();
    enemy_logic.DeadByUltimate();
}

//3. 적 총알 삭제
GameObject[] bullets = GameObject.FindGameObjectsWithTag("EnemyBullet");
for (int i = 0; i < bullets.Length; i++)
{
    Destroy(bullets[i]);
}
boom_cnt--;
```

- 폭탄을 쏠 수 없는 조건문을 작성합니다.
- 그리고 주석에 맞게 폭탄의 기능을 수행합니다.
- 이제는 다른 스크립트의 변수 및 method를 사용함에 있어 어려움이 없습니다.

4. 필살기 UI 관리

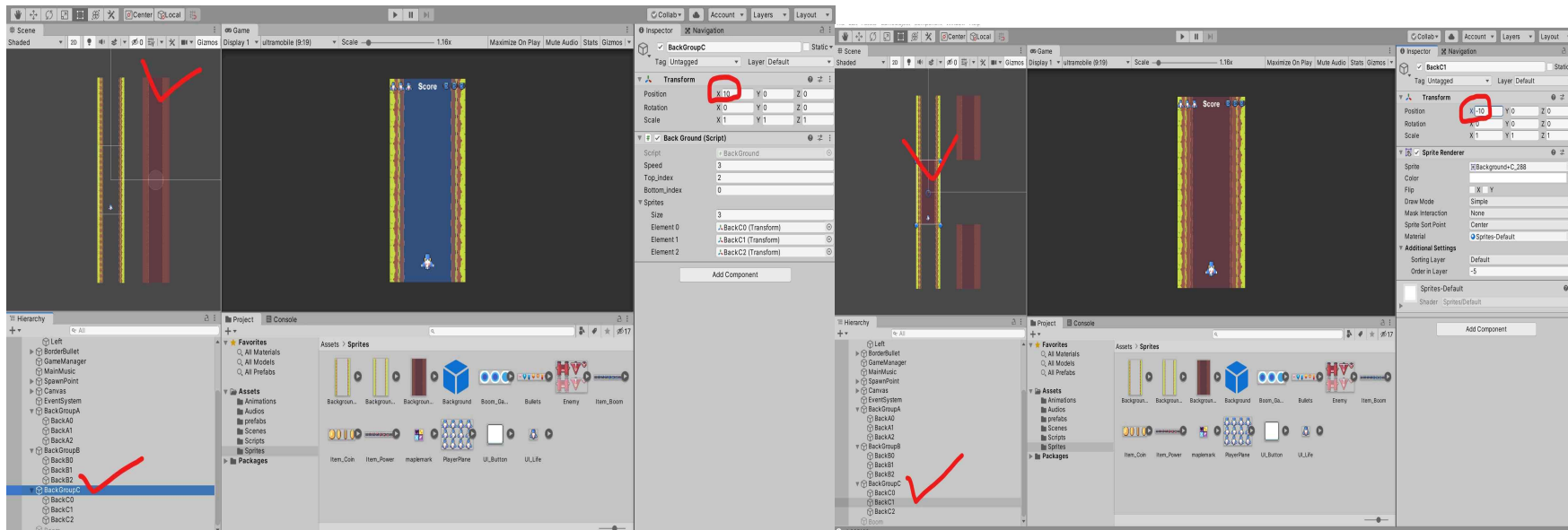
- 이전의 점수와 목숨 UI_Logic과 완전히 똑같습니다.
- 제작 할 때 강의 없이 혼자 제작했습니다.

```
//폭탄 먹는 메소드
public void UpdateBoom(int boom)
{
    Debug.Log("Boom Success");
    for (int i = 0; i < 3; i++)
    {
        boom_image[i].color = new Color(1, 1, 1, 0);
    }
    for (int i = 0; i < boom; i++)
    {
        boom_image[i].color = new Color(1, 1, 1, 1);
    }
}
```

<그룹으로 묶은 게임 오브젝트> 20/07/30

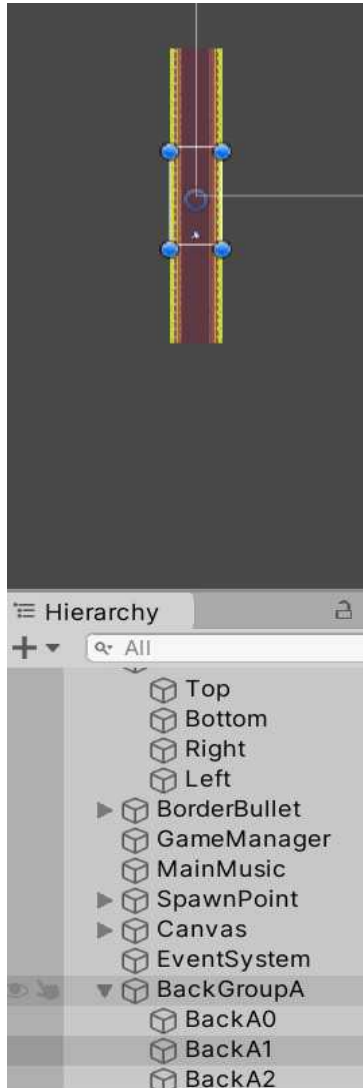
1. 그룹으로 묶인 GameObject의 위치

- 그룹전체의 position을 먼저 계산하고 -> 그룹 내부의 개개의 position을 계산합니다.
- 그룹의 position이 (-10,20,30) 이고 그룹내부의 어떤 개인이 (30,40,-20)이라면 그 개인의 포지션은(-10+30,20+40,30-20)입니다.
- 왼쪽 캡처는 그룹전체(BackGroupC)의 X값을 10으로 하였습니다. 그 결과 갈색 부분의 그림이 오른쪽으로 움직입니다.
- 오른쪽 캡처는 그룹(BackGroupC)중에서 Back1의 X값을 -10으로 하였습니다. 그 결과 Back1만 왼쪽으로 -10이 움직였습니다. 따라서 그룹전체가 +10을 하고 Back1만 -10을 했기에 Back1은 결과적으로 움직이지 않은 상태가 되었고 오른쪽 캡처처럼 정말 움직이지 않았습니다.



2. Position과 LocalPosition

- Position은 언제나 (0,0,0)을 기준으로 한 위치입니다. (절대 위치)
- LocalPosition은 부모 게임오브젝트의 위치를 기준으로 한 위치입니다. (부모 기반 상대 위치)
- 이 두 개념을 완벽히 이해하여야 물체의 위치에 대한 깊은 이해가 가능합니다.
- 그룹으로 묶인 게임오브젝트와 동일한 방법으로 이해할 수 있습니다.
- Queue 구조로 배경을 무한으로 만드는 것은 매우 쉬운 로직입니다.



BackGroupA는 갈색 부분 전체이고, BackA0~A2는 갈색 부분을 높이 기준으로 삼등분 한 것입니다. BackGroupA는 부모가 되고, BackA0~A2는 자식이 됩니다.

저는 BackGroupA를 스크립트에서 계속 아래로 내려가게 하였습니다. 당연히 그에 종속 된 BackA0~A2도 내려갑니다.

BackGroupA의 Position은 $(0, -y, 0)$ 으로 y 값은 계속 증가합니다. BackGroupA는 부모가 없으므로 LocalPosition은 Position과 동일하게 됩니다. 이 사실은 변하지 않습니다.

반면, BackA1의 Position은 아래의 코드로 인해 복잡한 로직을 갖게 됩니다.

```
if (sprites[bottom_index].position.y < -view_ht)
{
    Vector3 top_pos = sprites[top_index].localPosition;
    Vector3 bottom_pos = sprites[bottom_index].localPosition;
    sprites[bottom_index].transform.localPosition = top_pos + Vector3.up * view_ht;
```

sprites[]가 BackA0~A2이고, 이들 역시 BackGroupA가 내려가므로 내려가게 됩니다. 하지만 위의 조건문에서 A0~A2중 가장 아래의 오브젝트의 position(절대위치)이 $-view_ht$ 보다 작으면 그 것의 localPosition(상대위치)를 변화시킵니다.

즉, A0~A2의 절대위치도 위에서 같은 조건문이 없다면 계속 $(0, -y, 0)$ 이 될 것입니다. 하지만 조건문을 이용하여 위치를 변경해 주었고, 위치를 변경할 때 상대위치를 이용하면 더 간단하기에 그 값을 이용하여 변경하였습니다.

시뮬레이션을 통해 증명하면, BackGroupA(부모)의 절대위치는 어느 순간 $(0, -1000, 0)$ 이 될 것입니다. 시간이 지나면 더욱 아래로 내려갈 것 입니다.

하지만 조건문 덕분에 BackA0~A2(자식)의 절대위치 100년이 지나도 $(-24 \sim 24)$ 사이에 값으로 존재 할 것입니다. 그러나 상대 위치는 $\{0, -1000 + (-24 \sim 24 \text{ 사잇값})\}$ 이 될 것입니다.

개인적인 생각으로는 상대 위치로 표현한 식들을 절대 위치로도 표현할 수 있다고 생각이 듭니다.

```

float view_ht;
void Awake()
{
    //Camera.main에는 Camera클래스 관련 모든 메소드가 존재합니다.
    view_ht = Camera.main.orthographicSize * 2;
}

void Update()
{
    //이 스크립트에 연결되어 있는 게임오브젝트는 배경의 그룹입니다.
    //배경의 그룹전체를 아래로 이동하도록 합니다.
    Vector3 cur_pos = transform.position;
    Vector3 next_pos = Vector3.down * speed * Time.deltaTime;
    transform.position = cur_pos + next_pos;

    //Debug.Log(transform.position);
    Debug.Log("local : " + sprites[1].localPosition);
    Debug.Log("normal : " + sprites[1].position);
    //Debug.Log(sprites[bottom_index].position.y);
    if (sprites[bottom_index].position.y < -view_ht)
    {
        Vector3 top_pos = sprites[top_index].localPosition;
        Vector3 bottom_pos = sprites[bottom_index].localPosition;
        sprites[bottom_index].transform.localPosition = top_pos + Vector3.up * view_ht;

        //큐 구조로 Sprites들을 이동시켜서 무한 배경처럼 보이게 합니다.
        int tmp = top_index;
        top_index = bottom_index;
        if(tmp == 0)
        {
            bottom_index = sprites.Length - 1;
        }
        else
        {
            bottom_index = tmp - 1;
        }
    }
}

```

<Polo 모작 만들기> 20/08/11

2. 스크롤링

```
Queue<int> q = new Queue<int>(); //큐 이용(자료구조 이용)

void Start()
{
    ps = player.GetComponent<PlayerScript>();
    timespan = 0f;
    background_front_index = 0;
    background_end_index = 2;
    q.Enqueue(0);
    q.Enqueue(1);
    q.Enqueue(2);
}
```

- 스크롤링은 유한한 배경이미지를 transform의 변경을 통해 무한한 배경이미지로 만드는 기법입니다.
- 그리고 플레이어의 위치는 제한을 두고 , 배경만 움직여서 플레이어가 무한히 움직이게 느끼도록 눈속임을 줍니다.

1) 먼저 반복될 배경을 3개 정도 만듭니다.



2) 배경 모두를 같은 방향으로 이동시킵니다.

```
//두개의 배경을 스크롤링  
for(int i=0; i<background.Length; i++)  
{  
    background[i].transform.position -= new Vector3(0.03f, 0, 0);  
}
```

3) 어느 위치에 도착하면 맨 앞의 배경을 맨 뒤로 이동시킵니다. 이 때 Queue를 이용합니다.

```
Queue<int> q = new Queue<int>(); //큐 이용(자료구조 이용)
```

```
void Start()
```

```
{  
    ps = player.GetComponent<PlayerScript>();  
    timespan = 0f;  
    background_front_index = 0;  
    background_end_index = 2;  
    q.Enqueue(0);  
    q.Enqueue(1);  
    q.Enqueue(2);  
}
```

```
//큐를 이용한 스크롤링 로직
```

```
background_end_index = q.Peek();
```

```
q.Dequeue();
```

```
background_front_index = q.Peek();
```

```
q.Enqueue(background_end_index);
```

- Peek()은 front()와 같고 , Enqueue() push와 같습니다.
- Dequeue는 pop()과 같이 맨 앞의 원소를 제거하지만 , 그 후에 Peek()까지 리턴해줍니다.
- (다시 말해 , 맨 앞 원소를 제거하고 현재 큐의 맨 앞 원소를 리턴 합니다)

```
background_end_index = q.Peek();  
background_front_index = q.Dequeue();  
q.Enqueue(background_end_index);
```

이렇게 해도 될 것 같지만 **q.Dequeue()**를 2번째 식에서 먼저 처리하지 않으므로 결과가 이상하게 나옵니다.

4) 3번 과정을 무한루프 시킵니다.

<Open Source Study> 21/08/29 ~

1. 기초적인 내용

- 함수 이름을 매우 자세하게(길게) 작성합니다.

2. /// 주석

/// : 모노디벨롭에서 제공하는 자동완성 주석

```
/// <summary>
/// Plus the specified num1 and num2.
/// </summary>
/// <param name="num1">Num1.</param>
/// <param name="num2">Num2.</param>
int plus(int num1, int num2){
    return num1 + num2;
}
```

자동완성 주석을 남기길 원하는 함수 위에 ///를 입력하면 자동완성 주석이 뜨게 됩니다.

3. Inspector 기능

<https://computer-warehouse.tistory.com/8>

4. 인터페이스 , 가상함수 , 상속

- 교수들은 MonoBehaviour를 상속받지 않고 자신이 만든 인터페이스나 클래스를 상속 받음.

① Interface

```
1  namespace UOP1.StateMachine
2  {
3      interface IStateComponent
4      {
5          /// <summary>
6          /// Called when entering the state.
7          /// </summary>
8          void OnStateEnter();
9
10         /// <summary>
11         /// Called when leaving the state.
12         /// </summary>
13         void OnStateExit();
14     }
15 }
```

② Interface를 상속 받고 , 반드시 구현해야 하는 함수를 가상함수로 구현한 부모 클래스

```
public abstract class StateAction : IStateComponent
{
    internal StateActionSO _originSO;

    /// <summary>
    /// Use this property to access shared data from the <see cref="StateActionSO"/> that corresponds to this <see cref="StateAction"/>.
    /// </summary>
    protected StateActionSO OriginSO => _originSO;

    /// <summary>
    /// Called every frame the <see cref="StateMachine"/> is in a <see cref="State"/> with this <see cref="StateAction"/>.
    /// </summary>
    public abstract void OnUpdate();

    /// <summary>
    /// Awake is called when creating a new instance. Use this method to cache the components needed for the action.
    /// </summary>
    /// <param name="stateMachine">The <see cref="StateMachine"/> this instance belongs to.</param>
    public virtual void Awake(StateMachine stateMachine) { }

    public virtual void OnStateEnter() { }
    public virtual void OnStateExit() { }

    /// <summary>
    /// This enum is used to create flexible <c>StateActions</c> which can execute in any of the 3 available "moments".
    /// The StateAction in this case would have to implement all the relative functions, and use an if statement with this enum as a condition to decide whether to
    /// </summary>
    public enum SpecificMoment
    {
        OnStateEnter, OnStateExit, OnUpdate,
    }
}
```


③ 가상함수의 내용을 Override하여 구현한 자식 클래스

```
public class AnimatorParameterAction : StateAction
{
    //Component references
    private Animator _animator;
    private AnimatorParameterActionSO _originSO => (AnimatorParameterActionSO)base.OriginSO; // The SO this StateAction spawned from
    private int _parameterHash;

    public AnimatorParameterAction(int parameterHash)
    {
        _parameterHash = parameterHash;
    }

    public override void Awake(StateMachine stateMachine)
    {
        _animator = stateMachine.GetComponent<Animator>();
    }

    public override void OnStateEnter()
    {
        if (_originSO.whenToRun == SpecificMoment.OnStateEnter)
            SetParameter();
    }
}
```

<10/21 학습>

1. 왜 GetComponent()를 사용하면 느린가

GetComponent(문자열) 사용 방지

[GetComponent\(\)](#) 를 사용하는 경우 몇 가지 오버로드가 있습니다. 항상 형식 기반 구현을 사용하고, 문자열 기반 검색 오버로드를 사용하지 않는 것이 중요합니다. 장면에서 문자열로 검색하는 경우 Type을 기준으로 검색하는 것보다 훨씬 더 비쌉니다.

(좋음) GetComponent(Type 형식) 구성 요소

(좋음) T GetComponent<T>()

(나쁨) GetComponent(문자열)> 구성 요소

[GetComponent<T>\(\)](#) 및 [Camera.main](#) 과 같은 반복적인 함수 호출이 포인터를 저장하는 메모리 비용에 비해 더 비싸기 때문에 초기화 시 모든 관련 구성 요소 및 GameObjects에 대한 참조를 캐싱하는 것이 좋습니다. [Camera.main](#) 은 아래의 [FindGameObjectsWithTag\(\)](#) 만 사용합니다. 이는 "MainCamera" 태그를 사용하여 장면 그래프에서 카메라 개체를 검색하지만 많은 비용이 들어갑니다.

2. C# Struct vs Class

Classes Only:

- Can support inheritance
- Are reference (pointer) types
- The reference can be null
- Have memory overhead per new instance

Structs Only:

- Cannot support inheritance
- Are value types
- Are passed by value (like integers)
- Cannot have a null reference (unless Nullable is used)
- Do not have a memory overhead per new instance - unless 'boxed'

Both Classes and Structs:

- Are compound data types typically used to contain a few variables that have some logical relationship
- Can contain methods and events
- Can support interfaces

3. 이벤트 주도적 프로그래밍

- 책에도 내가 말한 방식이 정답인데 답이 뭘까... 입사한다면 물어보고 싶습니다

4. 게임의 발열

<https://merlins.tistory.com/entry/%EC%9C%A0%EB%8B%88%ED%8B%B0-%EB%B0%9C%EC%97%B4-%EC%9D%B4%EC%8A%88%EC%97%90-%EA%B4%80%ED%95%9C-%EC%A0%95%EB%A6%AC>

5. 게임 개발자가 내적과 외적 쓰는 경우

<http://rapapa.net/?p=2974>

6. 나는 완벽히 monobehaviour의 생명주기를 이해하고 있습니까?

- 잘 말한 것 같지만 사용해보지 않은 단계의 함수는 설명하지 못했습니다.

7. 17. 을 고치자

- occlusion culling frustrum culling을 합쳐서 이야기했는데 , 둘 다 사용해보지 않았기에 이론만 알았기 때문입니다.

<11/18 : Shader Coding>

1. 강의 해주신 선배님이 하고 싶은 말

- 숭실대 글로벌미디어 03학번 졸업생 강의 (unity engineer)
- 현업 구조는 개발자 : 그래픽 = 3 : 7
- 콘텐츠 , 서버 , 그래픽스 프로그래머가 게임회사에 존재하는데 그래픽스 프로그래머는 c#보다는 shader 코딩을 주로 합니다. 동료들이 원하는 shader 코드를 만들고 최적화 하도록
- 그래픽 디자이너와 게임 개발자는 많이 싸움 (요구하는 결과물이 다르기 때문입니다.)
- 자료구조 , 알고리즘은 필수적으로 공부해야 합니다.
- 요즘 학생들이 모두 똑똑하지만 면접에서 자신의 생각을 면접관에게 명확하게 전달하지 못하는 경우가 많습니다. 그러므로 그 연습을 합시다.
- 요즘에는 Shader 개발자와 TA 개발자가 높은 가치를 지니고 있습니다.
- 이직을 할 시에 연봉은 최소 10% 상승하고 , 일반적인 회사는 3%씩 상승합니다.

2. 기초 개념

1) Shader vs Material

① Shader

컴퓨터 그래픽스 분야에서 **셰이더(shader)**는 소프트웨어 명령의 집합으로 주로 그래픽 하드웨어의 렌더링 효과를 계산하는 데 쓰인다. 셰이더는 **그래픽 처리 장치(GPU)**의 프로그래밍이 가능한 **렌더링 파이프라인**을 프로그래밍하는 데 쓰인다.

셰이더는 표면상으로 무한해 보이는 효과를 만들기 위해 **영화 후처리**, **CGI**, **비디오 게임**에 널리 쓰인다. 단순한 광원 모델을 떠나, 더 복잡한 이용에는 영상의 **색조**, **채도**, **밝기**, **대비를 변경**하는 일과 **블러**, **라이트 블룸**, **입체 광원**, **심도 효과**를 위한 **노멀 매핑**, **보케**, **셀 셰이딩**, **포스터리제이션**, **범프 매핑**, **왜곡**, **크로마 키** (이른바 **블루스 크린/그린스크린 효과**), **테두리 검출**, **모션 감지**, **사이키델리아 효과** 제작 등을 포함한다.

- GPU에서 동작하는 **Material** , 조명정보 , 카메라 정보 같은 효과를 명령하는 것.

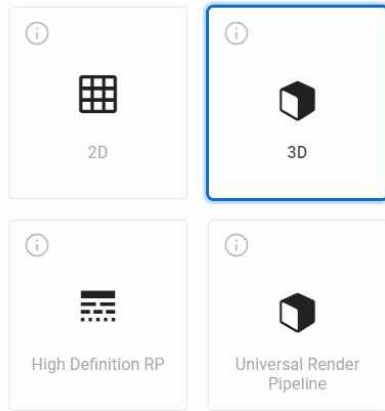
② Material

- 게임 오브젝트의 표면의 정보를 담고 있음. (오브젝트의 색깔 , 반짝임 데이터 구조)
- Material 안에 Shader를 넣을 수 있습니다.

2) Unity Project 만들기

Unity 2019.4.18f1 버전으로 새 프로젝트 생성

템플릿



- 2D/3D : 과거의 Rendering Pipeline을 이용하는 가벼운 project
- URP (Universal Rendering Pipeline) : 현대(최근 3~4년)에 사용하는 가벼운 중간 그래픽 성능의 project
- HDRP (High Definition Rendering Pipeline) : 현대에 사용하는 무거운 고급 그래픽 성능의 project
- HDRP는 언리얼의 고성능을 따라잡기 위해 고안한 Rendering Pipeline

3) Texture Mapping vs UV Mapping

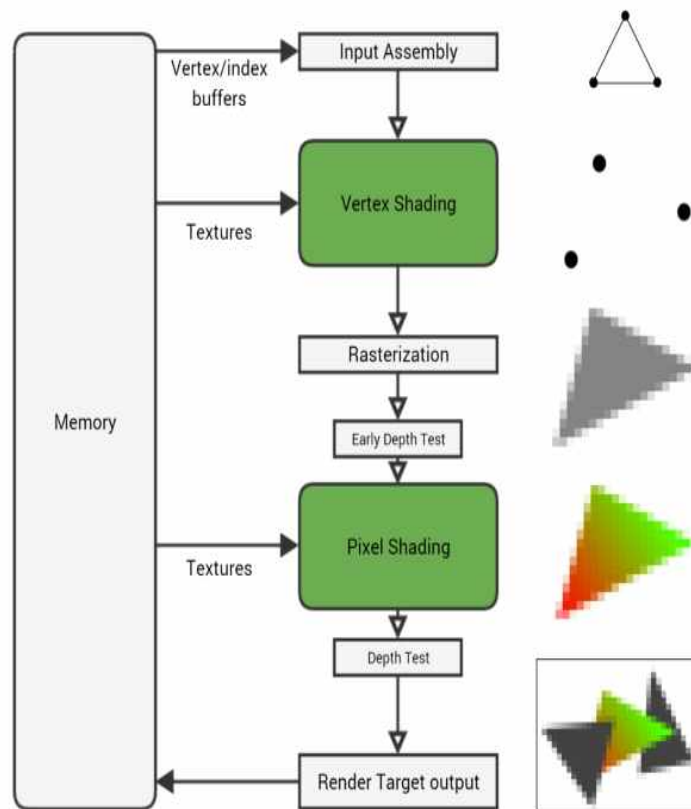
- Texture는 uv 정보가 있어야 입힐 수 있다.
- UV 도면 좌표 (2차원 전개도 같음)
- u,v 의 2D -> x,y,z의 3D
- texture mapping을 하려면 vertex shading OR Pixel shading에서 뭔짓을 해야함.

3. Rendering Pipeline

*** 컴퓨터 그래픽스 필기는 이론적인 내용이라면 강의를 기반으로 이론적인 내용과 실제로 내가 경험한 부분을 필기합니다. ***

1) 개요

Rendering Pipeline



① Input Assembly : Input 받기

② vertex shading : 삼각형이 들어오면 삼각형의 위치변환을 합니다. 여기서 vertex input으로 어떤 것을 받으면 vertex output으로 결과물을 내보냅니다. 그걸 pixel shading에서 받습니다.

③ Rasterization : 점을 찍으면 점들 끼리 연결하는 픽셀들을 만듭니다.

④ Pixel shading : RGB를 만들어 컬러를 결정하는 것

- shader language = GLSL

2) 심층 분석

①

-

3. 실습

- 실습 과정을 굳이 필기 하지는 않겠습니다.
- Shader 코딩을 진행해 보았습니다.
- Shader Graph로 만들어 오던 것을 직접 Shader의 코드로 작성합니다.
- Shader 객체 자체를 더블클릭하면 코드가 존재합니다.
- Shader 코딩으로 인해 Sphere

<11/22 : Team Project 학습>

1. 유니티에서 Json 사용하기

1) Json (=Java Script Object Notation)

- 지원하는 자료형이 매우 다양합니다.
- 보안이 강력합니다.
- 코드를 사용합니다.
- 가독성이 매우 훌륭하고 Serialization & Deserialization가 매우 간편합니다.
- 쉽고 빠르고 간단하기에 다양한 플랫폼에서 많이 사용하는 형식입니다.

Json 어떻게 생겼는가..?

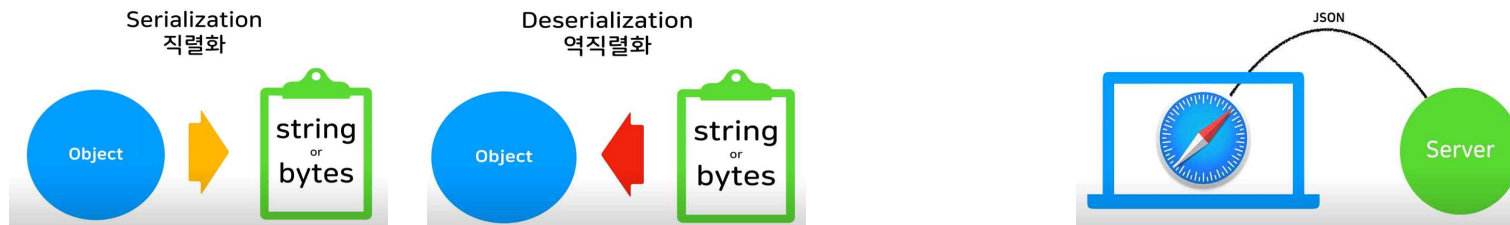
name(key) - value

```
{
  이름 : "NK_Studio"
  나이 : 25
}
```

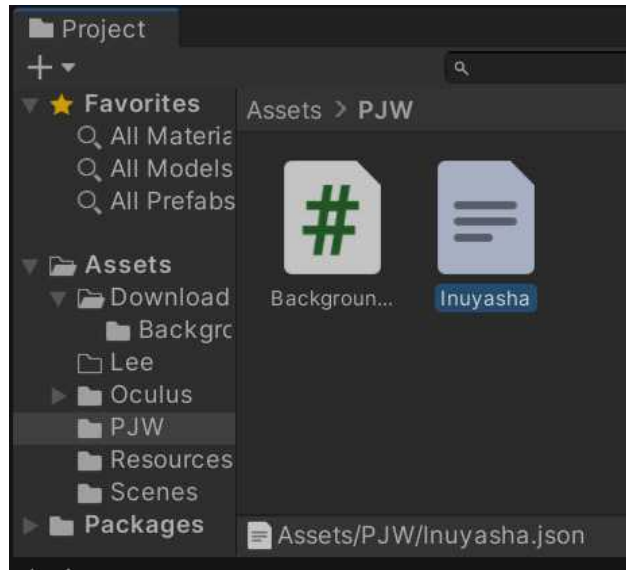
2) PlayerPrefs

- Unity에는 Json과 비슷한 형식의 코드가 필요없고 직관적인 간단한 함수인 PlayerPrefs가 존재합니다. 하지만 치명적인 단점 때문에 간단한 형식의 데이터에서만 사용합니다.
- 저장할 데이터의 자료형이 int , float , string 뿐입니다.
- n개의 데이터가 존재한다면 함수를 각각 n번 호출해야 합니다.
- 보안이 매우 취약합니다.

3) Serialization & De-Serialization (=직렬화 & 역 직렬화)



- Object는 Unity에서는 클래스의 객체를 의미합니다. 예를 들어 '박지원' 클래스를 만들고 멤버로 키, 몸무게, 성별 등을 저장하면 이 것이 그림에서의 Object가 됩니다.
 - 두 컴퓨터가 데이터를 주고 받을 때, 컴퓨터 마다 물리적으로 서로 다르기 때문에 통신이 되지 않을 수 있습니다. 그러므로 모든 컴퓨터가 수신 가능한 String or Bytes 형태로 변환하여 데이터를 송수신 합니다. 우리는 Json 형식을 추구하기 때문에 String or Bytes 대신 Json 형식으로 Serialization을 진행합니다. Json 형식으로 데이터를 받는다면, 다시 Object 형식으로 De-Serialization을 진행하여 변경합니다.
 - **Json Serialization : Object 형식의 데이터를 Json 형식의 데이터로 변경합니다.**
 - **Json De-Serialization : Json 형식의 데이터를 Object 형식의 데이터로 변경합니다.**
- 4) Path



```
string path = Path.Combine(Application.dataPath, "PJW", "Inuyasha.json");
```

Application.dataPath

```
public static string dataPath;
```

Description

Contains the path to the game data folder on the target device (Read Only).

The value depends on which platform you are running on:

Unity Editor: <path to project folder>/Assets

5) 실습

① Object 제작

```

//Json 형식으로 저장할 class를 생성합니다. (위의 그림에서 Object에 해당합니다.)
//Inspector 창에 나타나게 하기 위해 System.Serializable을 명시합니다.
[System.Serializable]
public class PlayerData
{
    public string name;
    public int age;
    public int level;
    public bool is_dead;
    public string[] items;
}

```

② Serialization (Object -> Json)

```

[ContextMenu("To json Data")]
void SaveDatatoJson()
{
    //1. JsonUtility API를 이용해서 내 오브젝트를 Serialization 합니다.
    // 매개변수의 true를 적지 않으면 default로 false가 되지만
    // 인간이 편하게 볼 수 있는 형식으로 작성하려면 true를 이용합니다.
    string json_data = JsonUtility.ToJson(player_data , true);
    //2. json 데이터를 저장할 경로 설정
    string path = Path.Combine(Application.dataPath , "PlayerData.json"); //저장할 경로
    //3. json 파일 생성
    File.WriteAllText(path, json_data);
}

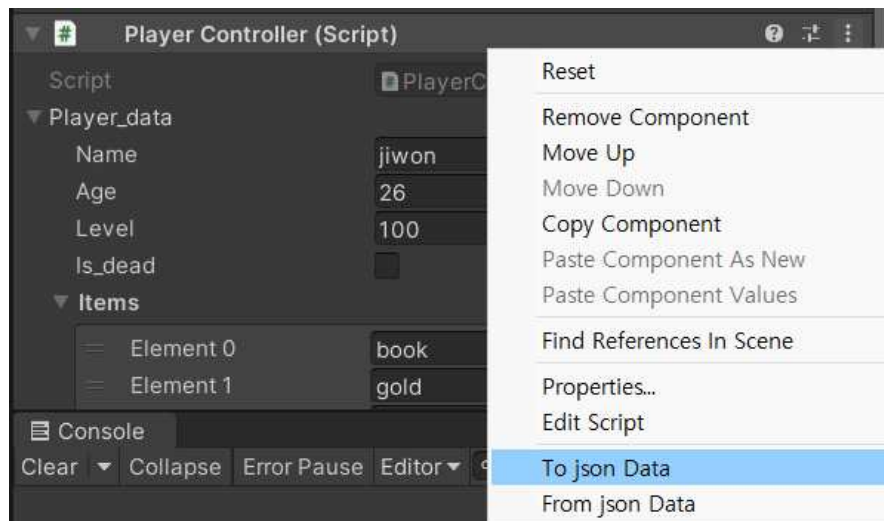
```

- 보통 해당 함수는 데이터를 저장하는 클래스와 별개의 클래스를 만들어서 그 클래스의 함수로 제작합니다.

③ De-Serialization (Json -> Object)

```
[ContextMenu("From json Data")]
void LoadDataFromJson()
{
    //1. 경로 찾기
    string path = Path.Combine(Application.dataPath, "PlayerData.json"); //저장할 경로
    //2. Json 데이터를 De-Serialization 하여 Object에 저장합니다.
    string json_data = File.ReadAllText(path);
    player_data = JsonUtility.FromJson<PlayerData>(json_data);
}
```

④ 적용



- Unity editor에서 'To json Data'를 통해 직렬화를 합니다.
- Unity editor에서 'From json Data'를 통해 역 직렬화를 합니다.
- json 데이터를 개발자가 수정하고 From json Data를 수행하면 즉시 수정 사항이 적용됩니다.

⑤ 참고 자료

ReadAllText(String)

텍스트 파일을 열고, 파일의 모든 텍스트를 읽은 다음에 파일을 닫습니다.

```
C# 복사  
  
public static string ReadAllText (string path);
```

매개 변수

path [String](#)
읽기 위해 열 파일입니다.

반환

[String](#)
파일의 모든 텍스트를 포함하는 문자열입니다.

JsonUtility

class in [UnityEngine](#) / Implemented in: [UnityEngine.JSONSerializeModule](#)

Description

Utility functions for working with JSON data.

Static Methods

FromJson	Create an object from its JSON representation.
FromJsonOverwrite	Overwrite data in an object by reading from its JSON representation.
ToJson	Generate a JSON representation of the public fields of an object.

WriteAllText(String, String)

새 파일을 만들고 지정된 문자열을 파일에 쓴 다음 파일을 닫습니다. 대상 파일이 이미 있으면 덮어 씁니다.

매개 변수

path [String](#)
쓸 파일입니다.

contents [String](#)
파일에 쓸 문자열입니다.

2. 기초 개념

- gameobject의 태그를 잘 살펴보자. 이상하게 설정해서 의도하지 않은 결과가 발생할 수 있음
- public - serialize - private
- 다른 지역에서 값 변화는 못하고 , 값 참조만 하는 건 역시 프로퍼티
- 데이터를 스크립트 끼리 주고 받는 순서가 중요함. 순서가 다르면 null을 전달하기도 함

프리팹 인스턴스 언패킹

프리팹 인스턴스의 콘텐츠를 일반 게임 오브젝트로 되돌리려면 프리팹 인스턴스를 언패킹해야 합니다. 이 작업은 프리팹을 생성(패킹)하는 작업의 정반대입니다. 단, 프리팹 에셋을 삭제하지 않으며 프리팹 인스턴스에만 영향을 줍니다.

계층 창에서 프리팹 인스턴스를 오른쪽 클릭하고 **Unpack Prefab** 을 선택하여 프리팹 인스턴스를 언패킹할 수 있습니다. 이제 씬의 게임 오브젝트는 이전 프리팹 에셋에 대한 링크를 더 이상 보유하지 않습니다. 프리팹 에셋 자체는 이 작업의 영향을 받지 않으며, 프로젝트에 다른 인스턴스가 있을 수 있습니다.

언패킹 전에 프리팹 인스턴스에 오버라이드를 적용한 경우 해당 프리팹 인스턴스는 결과로 생성되는 게임 오브젝트에 "베이크"됩니다. 즉, 값은 동일하게 유지되지만, 오버라이드할 프리팹이 없기 때문에 더 이상 오버라이드 역할을 하지 않습니다.

네스티드 프리팹이 있는 프리팹을 언패킹하는 경우 네스티드 프리팹은 프리팹 인스턴스로 유지되고 해당 프리팹 에셋에 대한 링크를 계속 보유합니다. 마찬가지로, 프리팹 베리언트를 언패킹하는 경우 루트에 새 프리팹 인스턴스, 즉 기본 프리팹의 인스턴스가 생성됩니다.

일반적으로 프리팹 인스턴스를 언패킹하면 해당 프리팹에 대한 프리팹 모드를 시작할 때와 동일한 오브젝트가 표시됩니다. 이는 프리팹 모드가 프리팹 내에 있는 콘텐츠를 표시하고, 프리팹 인스턴스를 언패킹하면 프리팹의 콘텐츠를 언패킹하기 때문입니다.

플레이 게임 오브젝트와 교체하고 모든 프리팹 에셋에 대한 링크를 완전히 제거하려는 프리팹 인스턴스가 있는 경우 **Hierarchy**를 마우스 오른쪽 버튼으로 클릭한 후 **Unpack Prefab Completely** 를 선택하십시오. 이는 프리팹을 언패킹하고 그 결과로 표시되는 모든 프리팹 인스턴스(네스티드 프리팹 또는 기본 프리팹)를 언패킹하는 작업에 해당합니다.

씬 또는 다른 프리팹 내에 존재하는 프리팹 인스턴스도 언패킹할 수 있습니다.

<static을 통해 모든 스크립트에서 접근 가능한 DATA 영역의 데이터 형성>


```
//Every-Script Can Use Music Data
public class MusicDataPjw : MonoBehaviour
{
    //now just using music_inuyasha
    public static List<Tuple<int, float>> music_inuyasha = new List<Tuple<int, float>>();
    public static List<Tuple<int, float>> music_letitgo = new List<Tuple<int, float>>();
    public static List<Tuple<int, float>> music_tmp = new List<Tuple<int, float>>();
}
```

- 다른 스크립트에서 데이터를 추가할 수 있음

<객체 복사>

```
public static List<Tuple<int, float>> music_inuyasha = new List<Tuple<int, float>>();
```

```
public List<Tuple<int, float>> selected_list = new List<Tuple<int, float>>();
```

```
selected_list = MusicDataPjw.music_inuyasha;
```

- 1번 그림은 스크립트_A에 static으로 만든 자료구조 입니다.
- 2번 , 3번 그림은 스크립트_B에서 객체를 형성하고 스크립트_A에 저장된 자료구조를 복사해서 사용하는 것 입니다.
- 올바르게 동작합니다.

<튜플 C#의 pair>

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace TupleTest
8 {
9     class Program
10     {
11         static void Main(string[] args)
12         {
13             //튜플로 List<int,string> 형식 선언
14             List<Tuple<int, string>> list = new List<Tuple<int, string>>();
15
16             list.Add(new Tuple<int, string>(1, "삼성전자"));
17             list.Add(new Tuple<int, string>(2, "현대"));
18             list.Add(new Tuple<int, string>(3, "아시아나"));
19             list.Add(new Tuple<int, string>(4, "BOE"));
20             list.Add(new Tuple<int, string>(5, "LG전자"));
21
22             //출력하기
23             for(int idx = 0; idx < list.Count; idx++)
24             {
25                 Console.WriteLine("{0} : {1}", list[idx].Item1, list[idx].Item2);
26             }
27         }
28     }
29 }
```

<맨날 힘든 개념 : nullreference>

```
for (int i = 0; i < GAYAGEUM_SCALES_COUNT; i++)
{
    ...
    unity_editor_current_scales[i] = new Queue<GameObject>();
}
```

```
Queue<GameObject>[] unity_editor_current_scales = new Queue<GameObject>[GAYAGEUM_SCALES_COUNT];
```

- C#은 배열을 만들고 , 배열의 각 요소도 메모리에 선언해 주어야 합니다. (너무 귀찮)

<객체 복사 ex : transform>

- transform을 어떤 객체에 복사하고 해당 객체가 움직여서 출력해보면 , 해당 원형 객체가 움직이면 복사 transform도 변함
- 근데 이게 tranform.position을 복사하면 안 됐던거 같음

<함수 나누기>

- 역시 함수로 나누고 설명하는 게 최고

<UI최적화>

<https://happysalmon.tistory.com/13>

<타인과 작업할 때는 클래스 이름이 겹칠 수 있고 , 그러면 오류가 나니까(git & unity) 스크립트나 클래스 이름뒤에 이니셜을 붙인다>

<자주 호출 되지 않는 거는 그냥 serialize 쓰자>

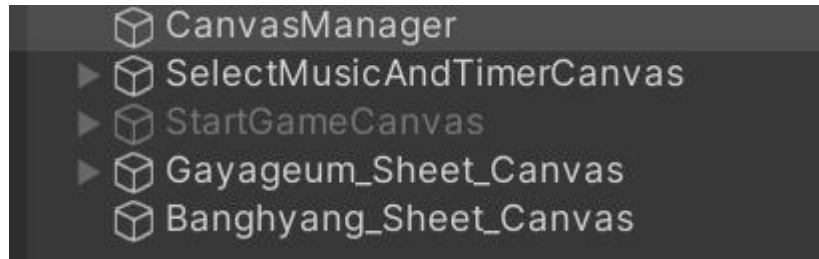
- 귀찮게 private 안해도 되니까 너무 편함

```
[SerializeField] private Button[] music_buttons;  
[SerializeField] private Text announcement_text;
```

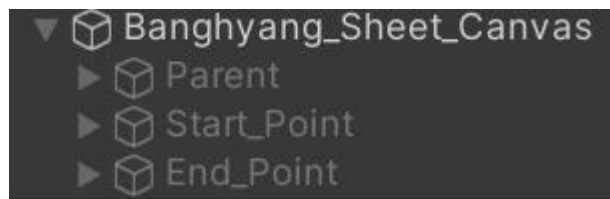
<전체에서 사용할 것은 하나의 싱글턴에>

<Manager Script>

- 빈 객체로 관련된 모든 Object를 관리하는 스크립트
- ex : CanvasManager로 하위 모든 canvas의 동작을 관리합니다.



<활성화>



자식만 죽이는 것도 나쁘지 않은 듯.

<제일 짝치는 상황>

- unity 실행 중에 뭐 바꿨는데 알아채지 못하고 당연히 바뀌었겠지 하고 작업하는데 에러 났는데 그게 사실은 바꾸지 않아서 생긴 오류일 때 개열 받음.

<플젝 아쉬웠던 점>

- 제작 기간이 3주인데 시험기간도 겹쳐서 주어진 시간이 너무 짧았음.
- 설계 후 개발이 필요한데 설계 시간이 너무너무 짧았습니다. 그래서 대충 설계하고 작업하다 보니 파트 분배도 모호하고(거의 안 됨) 내가 뭘 만 들어야 하는지 순서도 정해지지 않았습나다.
- 설계를 하지 않으니 존재하지 않는 것을 예측하면서 코딩해야 했기에 코딩의 퀄리티가 매우 낮아짐.
- 게임 개발을 4개월 만에 하니 초반에 속도가 나지 않았음
- 스크립트 분할 실패 (리팩토링을 해야 하는 이유) -> 같은 기능을 하는 함수 , 게임 전체에서 사용하는 자료구조 같은 것을 하나의 스크립트 파일 (manager)로 만들었다면 많이 편했을 것 같다고 생각. 설계를 오래하고 , 개인 개발을 할 때에도 시간이 많았다면 이를 모두 고려하며 했을 것. 그

러다보니 for문 9번 도는 것도 함수로 만들지 않고 무작정 코딩을 하고 , 쓸데 없이 코드가 길어졌습니다.

- 파트 분배가 되지 않아 같은 것을 만들어 온 적도 있음.
- 시간이 부족하다 보니 DB , Network를 계획했지만 실패.

<좋았던 점>

- 개발 2명 , 디자인 2명이라 그래픽을 전담해주는 팀원이 존재했고 , 개발 들어서 파트 분배가 모호했으나 중반부터는 나름 잘 되어서 진행이 원활 했다.
- 그래픽이 훌륭하니 퀄리티 있는 게임이 개발되었으며 , 학교에서 대면으로 자주 만나다 보니 답답했던 것을 이해할 수 있었다.
- 리듬게임이라는 새로운 장르를 팀플로 진행해 보아 신선했으며 , 기존에 만들던 스타일을 제작했다면 더 잘했겠지만 발전이 없었을 것.
- 디자인 팀이 폴더명과 파일명을 잘 정리해주면 개발에서 그냥 가져다 쓰면 되니까 편했다. docs도 있으면 좋을 듯.

<serialize 효과>

serialize로 연결해준 애는 해당 오브젝트가 꺼져있어도 코드내에서 호출 가능 (죽은 자식 호출법 그거 해결)

<폴더>

- unity의 프로젝트에서 폴더를 이동시켜도 큰 문제는 없다. (경로 관련만 아니면)

<png to sprite>

Carpe-Denius



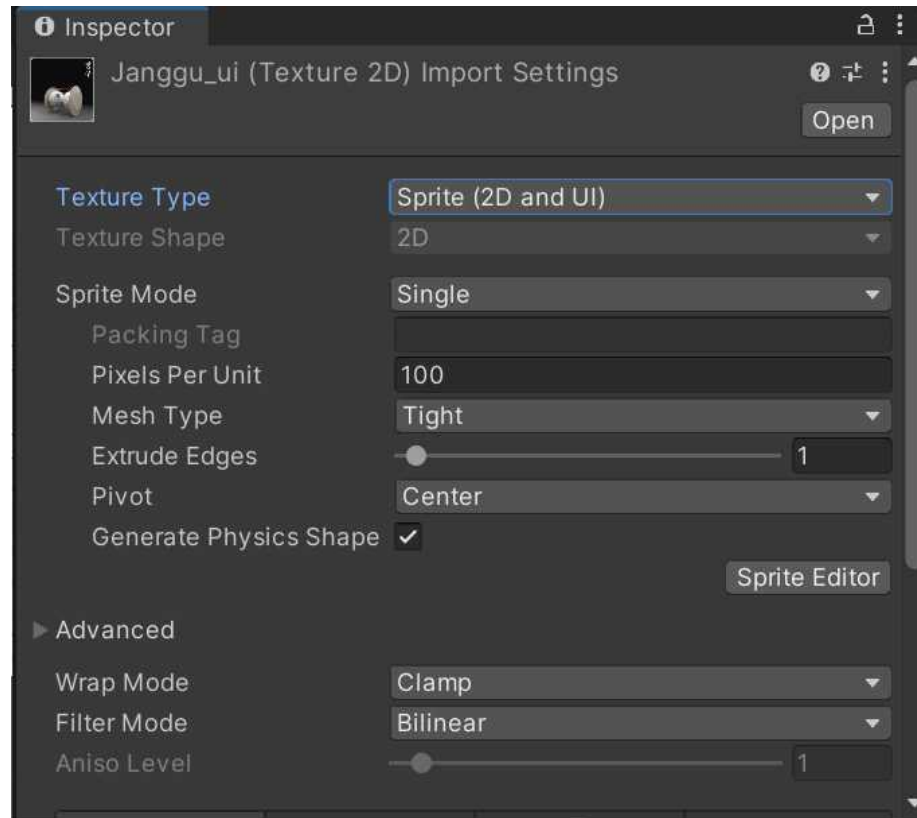
Joined:
May 17, 2013
Posts: 837

Choose image instead of rawimage and in your project folder click on your texture. There is a dropdown at the top of the inspector, "texture" is preselected, click on it and select Sprite (2d or UI).
After that, you should be able to drag your texture (sprite...) into your "Image"-gameobject.

Carpe-Denius, Jan 7, 2015

#9

shadowFire4444, druthers1993, Bboyun and 1 other person like this.



<비슷한 역할을 하는 스크립트 , 그러나 매개변수와 세세한 자료구조가 다름>

원래는 가야금과 방향 악보 부분을 동작하는 스크립트를 하나로 구현하려 했음
근데 가야금을 했는데 이게 굳이?
가야금 스크립트를 방향 스크립트에 복사하고 이용하면 되더라.
그리고 2개긴 하지만 if문을 돌리면 2배로 시간이 드는 거라 판단해서
1대1 대응으로 스크립트를 연결하면 좋지 않을까.

12/12 해결 : 3개의 악기를 전체로 관여하는 RhythmGamePlayManager같은 걸 만들고 여기서 호출하는 방식을 채택했어야 함.

<unity button UI>

- 코드로 작성하지 않고 Unity Editor에서 적용할 수 있는 내용이 생각보다 많습니다.

<씬 전환 해도 동작하는 배경음악>

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class StartMusic : MonoBehaviour
{
    GameObject BackgroundMusic;
    AudioSource backmusic;

    void Awake()
    {
        BackgroundMusic = GameObject.Find("BackgroundMusic");
        backmusic = BackgroundMusic.GetComponent<AudioSource>(); // 배경음악 저장해둠
        if (backmusic.isPlaying) return; // 배경음악이 재생되고 있다면 패스
        else
        {
            backmusic.Play();
            DontDestroyOnLoad(BackgroundMusic); // 배경음악 계속 재생하게(이후 버튼매니저에서 조작)
        }
    }
}
```

<버튼 unity inspector>

Button

?

Interactable

☒

Transition

Color Tint

Target Graphic

—

Normal Color

Highlighted Color

Pressed Color

Selected Color

Disabled Color

Color Multiplier

1

Fade Duration

0.1

Navigation

Automatic

Visualize